UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master's Degree in Computer Science

AMM Scheduler:

Design and Implementation of a Runtime Scheduling System for Asymmetric Heterogeneous Computing

Supervisor:

Prof. Marco Danelutto

Candidate:

Simone Stanganini

ACADEMIC YEAR 2024/2025

Contents

Chapter 1

Introduction

1.1 Context

Until a few years ago, performance improvements depended mainly on increasing CPU clock speeds. Today the approach has changed completely, shifting the focus towards hardware specialization.

Historically, this specialization was driven by distinct applicative domains. Two of the most common examples are that GPUs were designed for rendering pixels, while DSPs handled only signal processing. However, in recent years, the landscape of high performance computing has witnessed a fundamental change; with the explosive growth of Artificial Intelligence, a large number of modern applications from computer vision to LLMs are impacted by the performance achieved in the execution of a core kernel: **matrix multiplication**.

Consequently, hardware architectures have evolved to optimize this specific operation. Modern computers from servers to embedded SoCs, don't rely on a single processor anymore. Instead, they integrate a heterogeneous suite of execution units, all capable of performing linear algebra, but with different characteristics:

- **CPU**: Optimized for sequential tasks, complex logic and low latency.
- **GPU**: Available as integrated (iGPU) or discrete (dGPU), in a lot of cases both at the same time, designed for massive parallel computing.
- **NPU** (Neural Processing Unit): A recently introduced unit, specialized in speeding up matrix operations for AI workloads, trading flexibility for extreme efficiency.

This variety of hardware has created a gap with the software. While hardware offers different resources for different workloads, traditional software tends to be static. It

usually assigns tasks to a single device, often just the CPU or GPU, and ignores the other available resources.

The result is an inefficient usage of the system. We could get more throughput with the same efficiency, or be more efficient at the same speed.

This is precisely where our project fits in. Since the computational core of modern applications has converged on matrix operations, the current challenge is orchestration: we need a system capable of dynamically mapping these matrix multiplication tasks to the right accelerator, exploiting the entire heterogeneous topology of the system to maximize performance.

1.2 Objectives

The main goal of this thesis is therefore the design and implementation of a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures.

The work started with an essential preliminary phase, I focused on integrating and studying different compute backends, such as CUDA, SYCL & OpenVINO. This step was useful to create a uniform abstraction layer over the hardware and to verify the technical feasibility of the system.

After this initial phase, the work moved on to the actual development of the scheduler, the core of this project, focused on the asymmetric orchestration logic, designed to decide in real-time which resources will be used for each specific ideal workload such that utilization and efficiency are maximized.

Finally, the system was validated with experimental results on a complex hybrid architecture. This experimental setup included an Intel multicore CPU (with integrated GPU and NPU) and a discrete NVIDIA GPU. The tests used to verify the operation were streams of matrices of the same size, streams of differently sized matrixes, and even splitting larger tasks across multiple accelerators.

1.3 Thesis Structure

The rest of this work is organized as follows:

- **Chapter 2**: Introduces the hardware technologies (CPU, GPU, NPU), the software backends used (CUDA, SYCL, OpenVINO, OpenBLAS) and the software stack used for this project (C, C++, Cmake).
- **Chapter 3**: Describes the logical design, divided into two phases: the creation of the benchmarking environment and the subsequent design of the scheduler.
- **Chapter 4**: Details the code implementation, it covers the wrappers for hardware abstraction, the essential data structures for the scheduler (ringbuffer and performance map), and the techniques used for task splitting, as well as the project compilation procedures and details.
- **Chapter 5**: Analyzes the experimental results, validating the effectiveness of the scheduler on different workloads.
- **Chapter 6**: Draws conclusions on the work done and suggests possible future developments.

Chapter 2

Tools and Technologies

This chapter presents the hardware and software resources used to develop and validate our scheduler. The choice to work on a hybrid and highly heterogeneous architecture was made to test the scheduler in a real-world scenario, and in this scenario, devices with different performance characteristics and programming models have to work together.

2.1 Experimental Platform

All development and benchmarking activities were done on a mobile workstation Lenovo Yoga Pro 9i gen 9. This platform represents a modern high-performance "Edge" computing node well. It integrates all the types of accelerators we considered into a single physical system: multicore CPU, NPU, integrated GPU, and discrete GPU. Having these components available at the same time allowed to emulate, within a single node, the orchestration issues of heterogeneous distributed systems.

2.1.1 The SoC: Intel Core Ultra 9 185H

The core of the system is the Intel Core Ultra 9 185H processor, based on the "Meteor Lake" architecture [2]. This component marks a significant change from traditional monolithic CPUs. It uses a disaggregated design (tiled architecture) that combines different specialized processing units:

Figure 2.1: Intel Ultra Logo

- **Host CPU**: This cpu is made of 16 hybrid cores, 6 Performance cores, 8 Efficient cores, and 2 Low Power cores for efficiency. In this project this unit acts both as host and accelerator. It runs the scheduler code and does some computation through the OpenBlas library (see Sec. 2.5).
- **NPU**: The Ultra 9 includes a native Neural Processing Unit. This is a low power accelerator designed specifically for neural network inference. Its compute acceleration is facilitated by Neural Compute Engines, which consist of hardware acceleration blocks for AI operations like matrix multiplication and convolution, alongside streaming hybrid architecture vector engines for general computing tasks [4]. In this project the NPU is exploited via the OpenVINO (see Sec. 2.4) backend.
- **iGPU**: Intel Arc Graphics, integrated into the SoC. This graphics unit based on Xe-LPG architecture [1], offers parallel computing capabilities with direct access to system memory. In this project this GPU has been exploited via with the SYCL library (see Sec. 2.3) as the backend to leverage the full performance of this accelerator.

2.1.2 NVIDIA GeForce RTX 4060

Figure 2.2: Nvidia Rtx Logo

To complement the Intel SoC the system is equipped with a discrete NVIDIA GeForce RTX 4060 Laptop GPU. This device is based on the Ada Lovelace architecture [8] and acts as the high-throughput accelerator of the system.

Unlike the integrated GPU this card features 3072 CUDA Cores for general parallel processing and 96 Fourth-Generation Tensor Cores. The Tensor Cores are specifically designed to accelerate dense matrix multiplications with 16 or even 8 bit operations.

The device utilizes 8 GB of GDDR6 VRAM with a 128-bit memory interface, this dedicated memory provides high bandwidth but requires explicit data transfer over the PCIe bus.

Within the scheduler this device is managed via the CUDA backend (see Sec. 2.2).

2.2 CUDA Toolkit

Figure 2.3: Nvidia Cuda Logo

To exploit the computational capabilities of the discrete NVIDIA GPU the system utilizes CUDA (Compute Unified Device Architecture). This parallel computing platform and programming model, developed by NVIDIA, enables dramatic increases in computing performance by harnessing the power of the GPU. It allows developers to accelerate compute intensive applications using C, C++, and Fortran, and is widely adopted in fields such as deep learning, scientific computing, and more in general high performance computing (HPC) [10].

In this project CUDA is the tool chosen to extract maximum throughput from the hardware (see Sec. 2.1.2). The implementation relies on specific libraries and mechanisms to optimize the execution of matrix multiplication streams.

- **cuBLAS and Tensor Cores**: to perform linear algebra operations we adopts cuBLAS (CUDA Basic Linear Algebra Subroutines). This library is the GPU-accelerated version of the standard BLAS specification and provides highly optimized implementations of matrix operations, specifically, cuBLAS allows the utilization of Tensor Cores, specialized hardware units designed to accelerate mixed-precision matrix multiplication (Float16 and Int8). This significantly increases the theoretical throughput compared to standard floating-point units [9].
- **CUDA Streams**: To handle the latency introduced by data transfers between the CPU and the GPU the system utilizes CUDA Streams. A stream is a sequence of operations that execute in issue-order on the GPU, by employing multiple streams it is possible to achieve computation/communication overlapping: while one stream is transferring data over the PCIe bus another stream can execute a computational kernel [13]. This mechanism, known as "latency hiding", is fundamental for real-time processing and ensures that the GPU compute units remain active as much as possible.
- **NVIDIA Management Library (NVML)**: For the analysis of energy efficiency the project integrates the NVIDIA Management Library (NVML). This serves as a monitoring tool that provides direct access to the GPU hardware sensors, it allows the retrieval of real-time metrics such as power usage (in milliJoule) and temperature without requiring external physical probes [11].

2.3 SYCL

While CUDA provides maximum performance for NVIDIA hardware, it requires a proprietary programming model, and to address the need for portability and to fully exploit the Intel hardware present in the system, this project adopts the SYCL standard through the Intel oneAPI toolkit.

- **The SYCL Standard**:

Figure 2.4: SYCL Logo

SYCL is an open, royalty-free standard managed by the Khronos Group. It enables high-performance computing on heterogeneous accelerators (such as CPUs, GPUs, and FPGAs) using standard ISO C++ [15]. As defined in the official specification, SYCL provides an abstraction layer that allows developers to write code for heterogeneous processors using a "single-source" style. This means that host code on the CPU and the device code on the accelerator are written in the same file, utilizing standard C++ templates and lambda functions. This approach eliminates the complexity of managing separate source files for different architectures.

- **Intel oneAPI**:

Figure 2.5: OneApi Logo

SYCL requires a specific software development kit (SDK) to be compiled and executed. For this thesis, the Intel oneAPI Base Toolkit was selected [3]. oneAPI is Intel's unified programming model designed to simplify development across diverse architectures. It implements the SYCL standard via the `icpx` compiler, and provides a comprehensive set of libraries optimized for a large set of devices. The choice of this toolkit allows the project to target the available Intel hardware efficiently. Moreover this choice guarantees that the codebase could seamlessly target accelerator from othe vendor like AMD without having to rewrite the code.

2.4 Openvino

Figure 2.6: OpenVino Logo

Developed by Intel, this open-source toolkit is designed to optimize and deploy deep learning models across various types of hardware, from edge devices to cloud servers [5].

The main function provided by the library is that it acts as a translator, it takes an AI model and converts it into a specific format that the target hardware understands. This is very useful because it allows the program to run on different components like the CPU, NPU or the integrated graphics without needing to rewrite the code for each specific accelerator.

The toolkit automatically handles the low level details to ensure the task runs as fast as possible in the chosen device. This efficiency is achieved through the model optimizer, as described in the documentation, this tool analyzes the computational operations and simplifies them (for example, by merging multiple mathematical steps into one) before execution.

The result is a hardware-agnostic format called intermediate representation, IR, which allows the runtme to map the operations efficiently to the available resources [5].

In this project OpenVINO was essential for targeting the specific NPU found in the system (see Sec.2.1.1). Since the NPU cannot run standard C++ code directly, OpenVINO acts as the bridge between the scheduler and the hardware, specifically, it was implemented to execute Matrix Multiplication tasks on the NPU. This allowed the system to offload these calculations to the neural accelerator, taking advantage of its high energy efficiency.

2.5 OpenBlas

Figure 2.7: OpenBlas Logo

To enable efficient processing on the Host CPU, the project integrates OpenBLAS. OpenBLAS is an optimized, open-source implementation of the BLAS (Basic Linear Algebra Subprograms) API.

Historically, the project emerged as a successor to GotoBLAS2, a highly regarded library developed by Kazushige Goto. After the original development ceased, the OpenBLAS community took over to maintain the code and expand support for modern architectures [16].

The library achieves high performance through a mechanism called runtime processor detection. OpenBLAS first identifies the CPU architecture and automatically selects the most optimized functions for that specific hardware. These functions are often implemented in assembly language to leverage SIMD instructions, such as AVX2 or AVX-512, which allow the processor to handle multiple data points simultaneously (vector computing). Additionally, the library automatically handles multi-threading to distribute the computational load across all available processor cores[12].

In this project this library allows the Host CPU to be treated as an additional accelerator. Rather than limiting the CPU solely to orchestration tasks, OpenBLAS enables the scheduler to use some of the processor cores as a worker node. More details on this will be provided in the Implementation Chapter 4.

2.6 C/C++

Figure 2.8: C and C++ Logo

The C and C++ programming languages represent two of the most influential and widely used tools in the history of computer science. Known for their performance, versatility, and low-level control, they are employed in a vast range of sectors, from operating systems to high-performance applications.

Below is a brief overview of both languages:

The C Language: Developed in the 1970s by Dennis Ritchie, it was one of the first high-level languages in history. It is renowned for its simplicity, efficiency, and direct access to memory, making it ideal for writing highly optimized code [6].

The C++ Language: C++ is an extension of C developed in the 1980s by Bjarne Stroustrup. Among its key features is the support for Object-Oriented Programming (OOP), which allows for the creation of more structured and modular software compared to procedural C [14]. It provides a rich Standard Library (STL) and is the industry standard for performance-critical applications.

In the context of this project, C plays a critical architectural role. It was utilized to define the Application Binary Interface (ABI) between the scheduler and the various hardware wrappers. This choice was driven by the need for a stable and simplified interface. Since the project components are built with different compilers, C++ was unsuitable due to its lack of a standardized ABI, which causes binary incompatibility. C, instead, ensures consistent linkage across these environments. Instead C++ serves as the primary development language. Its adoption was mandatory as it is the native language required by the SDKs of all the utilized frameworks. It is used to implement the complex logic of the scheduler and the data structures necessary for task management. Further details on this will be provided in the implementation Chapter 4.

2.7 Cmake

Figure 2.9: Cmake Logo

To manage the compilation process of such a complex architecture, the project utilizes CMake (Cross-Platform Make). CMake is an open-source family of tools designed to build, test, and package software. It was originally created by Kitware in 2000 to address the need for a powerful build environment capable of supporting cross-platform development [7].

CMake does not build the software directly, instead it acts as a generator: it reads configuration files named CMakeLists.txt, and generates standard build files for the native toolchain of the operating system, such as Makefiles for Linux or Visual Studio projects for Windows. This abstraction allows developers to describe how the project should be built in a way that is independent of the specific compiler being used.

In this thesis its primary function is to manage the complexity of dynamic linking: since the project depends on large external frameworks (CUDA, OpenVINO, OneAPI, OpenBlas), CMake is configured to locate and link these as shared libraries. This ensures that the executable remains lightweight and that dependencies are resolved correctly at runtime.

Additionally, it enables a modular structure where specific hardware backends can be included or excluded via simple configuration flags.

Chapter 3

Logical Design

This chapter describes the logical structure of the project, the design process was divided into two main phases:

- The first part of the chapter (see Sec. 3.1) presents the independent benchmarks. Before developing the centralized scheduler, standalone applications were created for each accelerator. This phase was essential to verify the feasibility of the project and to understand how to effectively control the specific hardware using the selected libraries.
- The second part (see Sec. 3.2) provides a comprehensive overview of the AMM Scheduler. This section describes the high level operation of all the components of the scheduler, it illustrates the architectural design of the main project and explains how the various logical blocks interact to enable the scheduler's functionality, detailing the specific operational logic that governs the system.

3.1 Preliminary Phase

Before designing the centralized scheduler, the project required a preliminary phase focused on the independent analysis of each software stack. Three standalone applications were developed, targeting the NVIDIA GPU, the Intel iGPU, and the Intel NPU respectively. The primary objective of this phase was to gain a deep understanding of the specific libraries (CUDA, SYCL, and OpenVINO) and their interaction with the hardware.

It is important to note that the OpenBLAS backend for the CPU was not included in this preliminary phase. This component was introduced directly during the development of the scheduler (described in the next section 3.2) with a specific goal: to maximize the degree of heterogeneity of the system, by treating the CPU as an additional accelerator rather than just a coordinator. Moreover unlike the GPU and NPU, which required a specific study of their asynchronous drivers and memory models, the CPU integration relies on standard synchronous function calls. Consequently, a dedicated standalone analysis deemed unnecessary. The technical details of this integration will be discussed in the implementation Chapter 4.

3.1.1 Studying the Common Patterns

A crucial goal of this stage was to gain a deep, hands-on understanding of the specific libraries. Since technologies like CUDA, SYCL, and OpenVINO manage hardware in fundamentally different ways, so a first approach phase was necessary to see how their specific mechanisms works and the drivers interactions.

This distinction is particularly evident in the case of OpenVINO. While CUDA and SYCL follow a traditional *kernel launch* paradigm, the NPU is designed exclusively for deep learning graphs, not for general-purpose linear algebra. Consequently, to perform a standard matrix multiplication (GEMM) on this accelerator, a workaround was required; instead of invoking a mathematical function, the operation had to be encapsulated within a synthetic neural network layer, effectively "tricking" the driver into treating a raw calculation that we wanted to perform as an inference task (more on this in section 4.2.2).

Despite these architectural differences, this preliminary study revealed a recurring pattern, the execution flow for every accelerator whether running a kernel or a simulated inference relies on four identical logical steps:

- **Context Initialization**: setting up the driver handles like context, queue, and so on.
- **Memory Management**: allocating buffers and managing host-to-device transfers.

- **Kernel Execution**: triggering the asynchronous computation.
- **Synchronization**: waiting for the hardware to complete the task.

Recognizing this common structure for each library was fundamental for the design of the Abstraction Layer used in the scheduler.

3.1.2 Benchmarking

During this phase, benchmarking routines were implemented within each of the standalone applications. However, their purpose was distinct from the final experimental results presented in later chapters. In this context, the measurements of the execution time and power consumption were intended to act as a development tool to:

- Verify the correctness of the implementation (e.g., ensuring the code was actually running on the accelerator and not falling back to the CPU).
- Analyze the runtime behavior of the drivers, and identify architectural bottlenecks (such as PCIe transfer overheads or compilation latencies).

However, we must point out that the performance profile observed in these isolated environments proved to be quite different from the behavior, observered when using the full scheduler. The concurrent execution of different workloads and the increased stress on system resources exposed new bottlenecks that were not visible during the standalone tests. Consequently, the backend implementations significantly evolved during the integration phase, and specific structures were introduced to mitigate these latencies. These optimizations and the specific issues encountered under high-load scenarios will be analyzed in detail in the implementation Chapter (4).

3.2 AMM Scheduler

The core of the project is the AMM Scheduler, that stands for Asymettric Matrix Multiplication Scheduler.

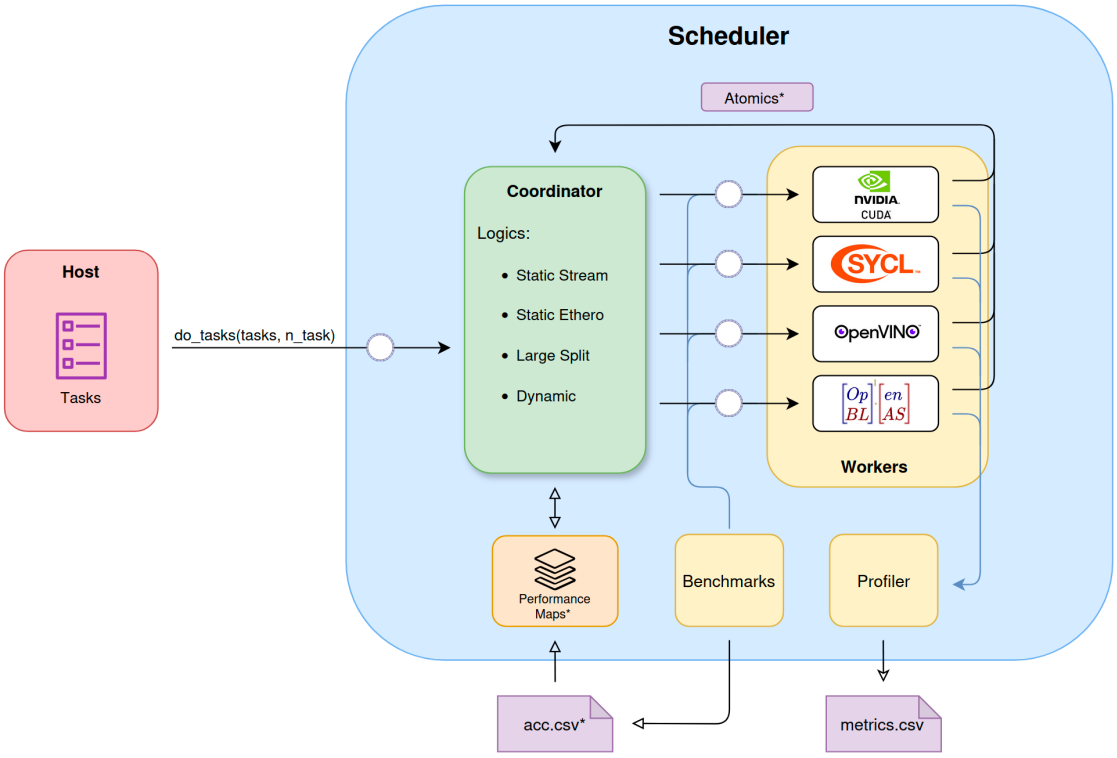

Figure 3.1: AMM Scheduler Components

As illustrated in Fig. 3.1, the architecture implements a producer and consumer pattern.

In this model, the host application acts as the producer, it generates computational tasks for the scheduler, that behave as the consumer, processing these requests asynchronously. The execution logic follows the farm pattern: a central coordinator retrieves the tasks and distributes them to the available worker nodes. To enable parallel execution, the coordinator and each worker operate as independent threads.

As shown in the diagram, Workers communicate with the Coordinator using atomic variables. This mechanism signals when a Worker is idle or updates the count of completed tasks, this ensures that the scheduler's wait function can correctly determine when all tasks are finished.

Now we will examine each component in the diagram and explain a bit deeper its behavior:

Figure 3.2: Ring Buffer Rappresentation

- **Ring Buffers**: The key to this asynchronous pattern is the ring buffer (rappresented in Fig. 3.2 as the arrows marked with a circle), which decouples the Host from the Coordinator, designed ad-hoc for the scheduler. This structure enables extremely fast communications with minimal overhead, using atomic variables instead of locks. As the graph in Fig 3.1 illustrates, this mechanism also implements the communication between the Coordinator and the Workers, minimizing the overhead and maximizing speed during task distribution. More on the code implementation in Sec. 4.3.5.

Figure 3.3: Coordinator

- **Coordinator**: The Coordinator (represented in Fig. 3.3) acts as the brain of the scheduler. It decides how to dispatch tasks and implements different strategies to demonstrate various resource management techniques.

  It features distinct scheduling logics, based on the type of workload that the host have to compute. The main logics that shape the design of the scheduler are the static ones.

  The first logic, Static stream, handles static homogeneous workloads (tasks of identical matrices). By using a Performance Map (a custom data structure containing heuristics and execution time estimates) the coordinator predicts how long a task will take on a specific accelerator and based on this, it performs a static dispatch of the tasks in a batch style way (see Sec. 4.3.9, Static Partitioning)

  The second logic, Static Hetero, work in a similar way but for static heterogeneous workloads (matrices of different sizes), the coordinator uses the same performance map to calculate the optimal distribution, ensuring the workload is balanced across all accelerators (see Sec. 4.3.11, Static Hetero Partitioning).

To evaluate these strategies, we also implemented a dynamic tasks dispatch logic. This approach assigns tasks to the first available Worker (i.e. idle) and serves as a baseline to verify the efficiency of our static methods (see Sec. 4.3.12, Dynamic).

Finally, the Coordinator supports the Task Splitting logic. If a matrix exceeds the memory capacity of a single accelerator, the logic breaks it down into smaller sub-tasks such that they may be distributed across multiple accelerators (see Sec. 4.3.10, Large Matrix Split).

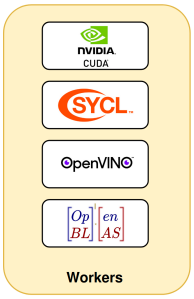

Figure 3.4: Workers

- **Workers**: Workers (represented in Fig. 3.4) are threads that utilize the abstraction layers defined in the preliminary phase (see Sec. 3.1). Their internal logic is straightforward: they retrieve available tasks and execute them on their assigned accelerator. All low-level details regarding the accelerator libraries are completely hidden from the Worker prospective (for implementation see Sec. 4.3.2, Worker).

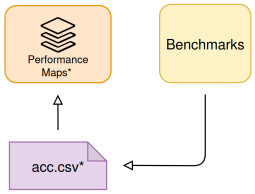

Figure 3.5: Benchmark and Performancemaps Rapresentation

- **Benchmarks and Performance Maps**: Internally, the scheduler utilizes a data structure called the Performance Map (represented in Fig. 3.5) to determine the execution time of every single task for each individual accelerator. The structure contains heuristics and data loaded from CSV files (specifically one file for each accelerator). These files are created by a benchmark routine developed directly within the scheduler. If the csv for one or more accelerators is missing so that there are no data for the performance maps, the routine activates automatically to generate the necessary information stored in the csv files, it utilizes the accelerator libraries, testing them by executing precise tasks

in a specific way to gather this data as accurately as possible. Furthermore, the Performance Map is an extremely fast structure: it is equipped with caching and highly optimized internal data structures to minimize overhead on the Coordinator's logic, which is the only component that accesses this structure (see Sec. 4.3.6, Performance Map).

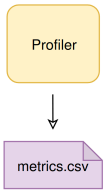

Figure 3.6: Profiler

- **Profiler**: To conduct an extensive and precise study of how the scheduler behaves across all workloads, a component called the Profiler (represented in Fig. 3.6) was developed directly within the scheduler. It is responsible for profiling all operational strategies and recording all execution times for both the scheduler and the workers. It returns all these metrics in an organized and readable manner in the form of CSV file. These metrics include the power consumption of the entire system and of each individual accelerator, as well as the specific overhead for every single phase of the scheduler's logic and a lot of more information that will be described later (see Sec. 4.3.7, Profiler).

Chapter 4

Implementation

In this chapter, after having seen a general explanation of how the scheduler works in the previous sections, we are going to look at and explain all its components in detail.

In the first section, we will explain the structure of the project files and the reason why they are organized this way, and we will also give a view of the entire compilation process of the scheduler.

Subsequently, we will explain each of the abstraction layers used to target the accelerators, and then to conclude, we will see in detail all the components that make the scheduler logic work.

As mentioned in Chap. 3, logical design, the code developed during the preliminary phase is not analyzed in this section. This avoids repetition, as the evolved version of this code, which is integrated into the scheduler, will be explained in detail later. However, all the original code is available in Appendix A.2.

4.1 Project Structure and Compilation

The figure below provides a view of the project's file and folder structure:

Project Tree

```
|-- bin/
|-- build/
|-- cuda/
|   |-- runner.cu
|   |-- cuda_wrapper.h
|   |-- CMakeLists.txt
|-- sycl/
|   |-- runner.cpp
|   |-- sycl_wrapper.h
|   |-- CMakeLists.txt
|-- openvino/
|   |-- runner.cpp
|   |-- ov_wrapper.h
|   |-- CMakeLists.txt
|-- scheduler/
|   |-- main.cpp
|   |-- scheduler.hpp
|   |-- performancemap.hpp
|   |-- sharedbuffer.hpp
|   |-- cpu.hpp
|   |-- profiler.hpp
|   |-- tasks.hpp
|   |-- tests.hpp
|   |-- config.hpp
|   |-- CMakeLists.txt
|-- CMakeLists.txt
|-- make_run.sh
|-- merge_clangd.py
|-- setup.txt
```

Figure 4.1: Project Dir Tree

The codebase of this project was designed with a specific goal: modularity. Since the project works with very different hardware architectures, writing all the code in a single environment is not feasible. Each accelerator requires its own specific compiler and libraries: NVIDIA GPUs need the `nvcc` compiler, Intel GPUs need the `icpx` compiler for SYCL, and the NPU relies on OpenVINO libraries.

To solve this compatibility issue, the architecture was decomposed into independent dynamic libraries. Each hardware backend (CUDA, SYCL, OpenVINO) is compiled into a separate shared object file (`.so`), and each of these libraries expose a standard C ABI (Application Binary Interface). This design is mandatory because the C ABI acts as a universal language that allows the main scheduler to communicate with the accelerators without needing to understand their specific C++ implementations or complex driver dependencies, moreover the real problem was with the version of the compilers utilized with each specific library, so the standard approach is to unify all the interfaces with the help of the C language.

As shown in Fig. 4.1, the project includes a dedicated folder for each library (`sycl/`, `openvino/`, and `cuda/`) containing its specific implementations, finally, the `scheduler/` directory containing all the internal dependencies used by the scheduler logic, which will be explained in Sec. 4.3.

4.1.1 Compilation Process

The tool CMake was used in this project to manage this complex build process, the build is not performed all at once.

Below is the entire CMakeLists.txt file in the root directory:

```
cmake_minimum_required(VERSION 3.15)
project(amm_scheduler)

include(ExternalProject)

set(LIB_OUTPUT_DIR ${CMAKE_SOURCE_DIR}/bin)
file(MAKE_DIRECTORY ${LIB_OUTPUT_DIR})

# CUDA SYCL OPENVINO
option(USE_CUDA "CUDA ON" OFF)
option(USE_SYCL "SYCL ON" OFF)
option(USE_OPENVINO "OpenVINO ON" OFF)
option(USE_OPENBLAS "OpenBLAS ON" OFF)
option(USE_OPENCV "OpenCV ON" OFF)

set(SCHEDULER_DEPS "") # depends variable

set(OPT_FLAGS "-O2 -DNDEBUG")

# CUDA
if(USE_CUDA)
    message(STATUS "CUDA Backend ON")
    ExternalProject_Add(build_cuda
```

```
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/cuda
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            -DCMAKE_CUDA_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )

    list(APPEND SCHEDULER_DEPS build_cuda)
else()
    message(STATUS "CUDA Backend OFF")
endif()

# SYCL and OPENVINO are the same as cuda

# scheduler
ExternalProject_Add(build_scheduler
    SOURCE_DIR ${CMAKE_SOURCE_DIR}/scheduler
    CMAKE_ARGS
        -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
        -DCMAKE_BUILD_TYPE=Release
        -DLIBS_DIR=${LIB_OUTPUT_DIR}
        -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}

        -DCUDA_SRC_DIR=${CMAKE_SOURCE_DIR}/cuda
        -DSYCL_SRC_DIR=${CMAKE_SOURCE_DIR}/sycl
        -DOPEN_SRC_DIR=${CMAKE_SOURCE_DIR}/openvino

        -DENABLE_CUDA=${USE_CUDA} # USE_CUDA
        -DENABLE_SYCL=${USE_SYCL} # USE_SYCL
        -DENABLE_OPENVINO=${USE_OPENVINO} # USE_OPENVINO
        -DENABLE_OPENBLAS=${USE_OPENBLAS} # USE_OPENBLAS
        -DENABLE_OPENCV=${USE_OPENCV} # USE_OPENCV

        -DCMAKE_EXPORT_COMPILE_COMMANDS=ON # for clangd
    DEPENDS ${SCHEDULER_DEPS}
    #BUILD_ALWAYS 1
)

add_custom_target(UpdateLSP
    ALL
    COMMAND python3 ${CMAKE_SOURCE_DIR}/merge_clangd.py
    DEPENDS build_scheduler
)
```

Listing 4.1: CMakeLists.txt root directory

As the code in the Fig. 4.1 show, the `ExternalProject_Add` feature of cmake is utilized to keep the compilation steps isolated. The main `CMakeLists.txt` file in the root directory acts as a manager. It does not compile the code directly, instead, it checks the configuration flags (such as `USE_CUDA`, `USE_SYCL` and `USE_OPENVINO`) and triggers a separate build process for each active module. The process is the following:

1. First, CMake enters the accelerators sub-directories, `cuda/`, `sycl/`, `openvino/`,

and builds them as standalone projects.

2. Each sub-project uses its specific compiler to generate a dynamic library, e.g., `libcuda_lib.so`, `libsycl_lib.so`).

3. Finally, the `scheduler/` directory is compiled. Because of the standard C ABI, the scheduler does not need to know how the hardware works or which compiler was used for the accelerators, it simply links against these shared libraries at runtime.

This separation ensures that the specific code for one hardware component does not interfere with the others. For example, the scheduler compiler does not need to handle CUDA Kernels, but rather it only sees standard C functions.

Based on the flags passed to CMake during compilation, specific preprocessor definitions are passed to the C++ code. This allows the program to enable or disable dependencies on specific libraries directly inside the source code.

Consequently, the scheduler logic knows exactly which accelerators are available. This approach makes the system flexible, if a specific accelerator or library is missing on the machine where the project has to be compiled, it can be disabled simply by using a flag.

The following snippet in Listing 4.2 from the scheduler's build configuration shows how the flags trigger the injection of preprocessor definitions and the linking of specific libraries:

```
# CUDA
if(NOT DEFINED ENABLE_CUDA)
    message("[SCHEDULER] CUDA off on scheduler")
    set(ENABLE_CUDA OFF)
endif()

# SYCL
if(NOT DEFINED ENABLE_SYCL)
    message("[SCHEDULER] SYCL off on scheduler")
    set(ENABLE_SYCL OFF)
endif()

# OPENVINO
if(NOT DEFINED ENABLE_OPENVINO)
    set(ENABLE_OPENVINO OFF)
endif()

#OPENBLAS
if(NOT DEFINED ENABLE_OPENBLAS)
    message("[SCHEDULER] OpenBLAS off on scheduler")
    set(ENABLE_OPENBLAS OFF)
endif()
```

Listing 4.2: Part of CMakeLists.txt inside the scheduler directory

The command `target_compile_definitions` passes a macro (e.g., `ENABLE_CUDA`) to the compiler, which the C++ code effectively uses to enable or disable code blocks via `#ifdef` directives. Simultaneously, `target_link_libraries` ensures that the scheduler only links against the dynamic libraries that are actually present.

The compilation process for the accelerator backends follows a standard pattern. As an example, the configuration file for the OpenVINO library is presented in Listing 4.3:

```
cmake_minimum_required(VERSION 3.20)
project(OpenVinoPart LANGUAGES CXX)

message("\n--------------- OPENVINO COMPILING \n")

find_package(OpenVINO REQUIRED)

add_library(ov_lib SHARED runner.cpp)

set_target_properties(ov_lib PROPERTIES PUBLIC_HEADER ov_wrapper.h)

# C++17 needed OpenVINO
target_compile_features(ov_lib PRIVATE cxx_std_17)

target_link_libraries(ov_lib PRIVATE openvino::runtime)

install(TARGETS ov_lib
    LIBRARY DESTINATION lib
    PUBLIC_HEADER DESTINATION include
)
```

Listing 4.3: CMakeLists.txt inside OpenVINO directory

First this script locates the necessary dependencies on the host system using `find_package`, then, it defines a shared library named `ov_lib` using the source file `runner.cpp`. The `SHARED` keyword is fundamental because it instructs the compiler to generate a dynamic library instead of a standard executable.

Finally, the script links the specific runtime (in this case, `openvino::runtime`) and specifies the installation rules. The `install` command ensures that the resulting binary and the public header (`ov_wrapper.h`) are placed in the correct output directories, making them accessible to the main scheduler.

The CUDA and SYCL modules use an identical structure, differing only in the specific compilers and libraries linked.

Finally, when the compilation finishes, all the resulting files are placed in a `bin/` directory. This folder contains the final executable named scheduler, the csv that the scheduler will create at runtime, and all the dynamic libraries it needs.

To speed up the development process, a script named `make_run.sh` was created. This script automates the entire workflow that consist of cleaning the bin and build dir, runs CMake with the correct options to enable all accelerators, compiles the

project using all available CPU cores, and finally executes the program.

Additionally, the project includes a Python script named `merge_clangd.py`. Since multiple independent builds are used, the LSP (Language Server Protocol) of the code editing tools often don't work with these many libraries. This script collects the compilation commands from all the different modules and merges them into a single file, ensuring that code completion and error highlighting work correctly across the entire project. This file will be executed after all the compilation steps are done.

4.2 Accelerators Abstraction Layers

Before diving into the architectural details in the following sections, it is important to clarify the supported data types. The scheduler is fully capable of managing both 32-bit and 16-bit matrix operations. Furthermore, thanks to the modular and flexible design of the codebase, the architecture is already capable to support 8-bit matrices as well. However, as 8-bit operations are not supported by all target accelerators and were not required to achieve the objectives of this thesis, their implementation has not been finalized across all the different backends.

4.2.1 Cuda

The CUDA abstraction layer is designed to manage the NVIDIA discrete GPU available in the system. As previously discussed, the interaction between the C++ scheduler and the NVIDIA compiler (`nvcc`) is mediated by a C interface.

The header file `cuda_wrapper.h` defines the Application Binary Interface exposed to the scheduler (see Listing 4.4). It declares the necessary functions for initialization, memory cleanup, and the execution of matrix multiplication across different data types, 32-bit floating point, 16-bit floating point, and 8-bit integer.

```
#ifdef __cplusplus
extern "C" {
#endif

    void cuda_init();
    void cuda_free();
    void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);

#ifdef __cplusplus
}
#endif
```

Listing 4.4: cuda_wrapper.h interface

The implementation of these functions is in the `runner.cu` file, but before analyzing the initialization and execution logic, we will examine the global data structures and helper macros defined in the file, these elements manage the persistent state of the device and ensure error reporting.

```
#define CHECK_CUDA(func) {                \
    cudaError_t e = (func);               \
    if(e != cudaSuccess)                  \
        printf ("%s %d CUDA ERROR: %s\n", __FILE__,  __LINE__,
                cudaGetErrorString(e)); \
}
```

```
7
8  #define CHECK_CUBLAS(func) {                          \
9      cublasStatus_t e = (func);                        \
10     if(e != CUBLAS_STATUS_SUCCESS)                    \
11         printf ("%s %d CUBLAS ERROR: ", __FILE__, __LINE__); \
12 }
13 #define N_STREAM 2
14
15 cublasHandle_t handle;
16 cudaStream_t streams[N_STREAM];
17 void *d_A[N_STREAM];
18 void *d_B[N_STREAM];
19 void *d_C[N_STREAM];
20 int i = 0; /* stream id*/
```

Listing 4.5: cuda_runner.cu data and helper

To ensure that the execution progresses without errors, the code utilizes two macros: `CHECK_CUDA` and `CHECK_CUBLAS` (see Listing 4.5). These are used to wrap every API call to the CUDA runtime and the cuBLAS library, respectively. If an operation fails the macro immediately prints the specific error code along with the file name and line number where the failure occurred.

As for as state management is concerned, the system relies on CUDA Streams, a macro `N_STREAM` defines the number of parallel execution queues. Streams, enable the GPU to allows the overlap of memory transfers with computation, or the concurrent execution of multiple kernels. To support this concurrency, the device resources are replicated for each stream. The memory pointers (`d_A`, `d_B`, `d_C`) are defined as arrays, meaning that each stream possesses its own independent pre-allocated memory buffers. Finally, a global integer `i` acts as counter, used to cycle through the available streams when dispatching new tasks.

The design focuses on minimizing the runtime overhead by performing expensive operations only once during the startup. The `cuda_init` function is responsible for preparing the GPU environment before any task is processed. Its primary goal is to eliminate the latency associated with running the dynamic memory allocation instrunctions. Allocating memory on the GPU using `cudaMalloc` is a slow operation compare to the others, infact if the system were to allocate and free memory for every single matrix multiplication task, the overhead would significantly impact the performance. To solve this, the initialization function adopts a simple pre-allocation strategy.

In Listing 4.6 the implementation of the `cuda_init` function:

```
void cuda_init() {
    CHECK_CUBLAS(cublasCreate(&handle));

    /* reset stream counter */
    i = 0;

    size_t MAX = (size_t)4096 * 4096 * sizeof(float);

    for (int j = 0; j < N_STREAM; ++j) {
        CHECK_CUDA(cudaStreamCreate(&streams[j]));
        CHECK_CUDA(cudaMalloc(&d_A[j], MAX));
        CHECK_CUDA(cudaMalloc(&d_B[j], MAX));
        CHECK_CUDA(cudaMalloc(&d_C[j], MAX));
    }
}
```

Listing 4.6: cuda_runner.cu init

At startup, the `cuda_init` function first initializes the cuBLAS library handle, which is required to invoke the linear algebra routines. Then, it iterates through `N_STREAM` number of streams (simply 2 stream is enough). For each stream, it creates a CUDA stream object to manage execution queues and allocates three distinct memory buffers on the device (`d_A`, `d_B`, `d_C`). The size of these buffers is calculated to hold the largest matrix dimension supported by the system (MAX size), ensuring that any incoming task fits within the pre-allocated space without requiring reallocation.

It is important to observe that, although the hardware accelerator supports dimensions exceeding this limit, a maximum square matrix size of 4096×4096 was established (for each accelerator). This design choice was made to avoid introducing unnecessary architectural complexity, since it wasn't in this thesis objectives to maximize the memory saturation, and this limit provides a sufficient workload to stress the scheduler without incurring into the typical limitations due to the memory management layer.

Complementary to the initialization, the `cuda_free` (see Listing 4.7) function ensures the proper release of all GPU resources when the scheduler shuts down.

```
void cuda_free() {
    for (int j = 0; j < N_STREAM; ++j) {
        CHECK_CUDA(cudaFree(d_A[j]));
        CHECK_CUDA(cudaFree(d_B[j]));
        CHECK_CUDA(cudaFree(d_C[j]));
        CHECK_CUDA(cudaStreamDestroy(streams[j]));
    }
    CHECK_CUBLAS(cublasDestroy(handle));
}
```

Listing 4.7: cuda_runner.cu free

The function iterates through the created streams, freeing the device memory buffers using `cudaFree` and destroying the stream objects with `cudaStreamDestroy`. Finally, it destroys the global cuBLAS handle to release the library resources.

To maintain compatibility between the C ABI and CUDA C++ features, the execution logic is split into two levels, since the public C interface cannot recognize CUDA specific types it relies on generic `void*` pointers.

To handle this the system implements a layer of simple bridge functions, these functions accept the generic pointers and cast them to the correct concrete type (e.g., `float*` or `__half*`), and immediately forward the call to the internal function (see Listing 4.8).

```
void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K) {
    cuda_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
}

void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K) {
    cuda_gemm_16bit_p((__half*)A, (__half*)B, (__half*)C, M, N, K);
}
```

Listing 4.8: cuda_runner.cu type bridging functions

The actual computational logic is implemented in the private functions, such as `cuda_gemm_32bit_p`. This functions orchestrates the entire lifecycle of a single GPU task. (see Listing 4.9)

```
void cuda_gemm_32bit_p(float* A, float* B, float* C, int M, int N, int K) {
        cublasSetStream(handle, streams[i]);

        CHECK_CUDA(cudaMemcpyAsync(d_A[i],
            A, M * K * sizeof(float), cudaMemcpyHostToDevice, streams[i]));
        CHECK_CUDA(cudaMemcpyAsync(d_B[i],
            B, K * N * sizeof(float), cudaMemcpyHostToDevice, streams[i]));

        float a = 1.0f, b = 0.0f;

        cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                 N, M, K,
                 &a,
                 d_B[i], CUDA_R_32F, N,
                 d_A[i], CUDA_R_32F, K,
                 &b,
                 d_C[i], CUDA_R_32F, N,
                 CUBLAS_COMPUTE_32F,
                 CUBLAS_GEMM_DEFAULT_TENSOR_OP);

        CHECK_CUDA(cudaMemcpyAsync(C,
            d_C[i], M * N * sizeof(float),
            cudaMemcpyDeviceToHost, streams[i]));
        /* blocking for latency metrics */
        CHECK_CUDA(cudaStreamSynchronize(streams[i]));
        /* change stream for the next call */
        i = (i + 1) % N_STREAM;
```

```
}
```

Listing 4.9: cuda_runner.cu 32-bit logic

The function begins by associating the global cuBLAS handle with the current stream (`streams[i]`), this ensures that all subsequent library calls are queued into this specific stream.

The execution begins by asynchronously copying the inputs A and B to the GPU buffers using the `cudaMemcpyAsync`, next, the matrix multiplication is triggered via `cublasGemmEx`, this function is chosen over standard BLAS calls because it offers more flexibility for high-performance computing.

It enables the `CUBLAS_GEMM_DEFAULT_TENSOR_OP` flag, explicitly instructing the GPU to utilize tensor cores whenever is possible (with 16 bit and 8 bit, in more modern GPUs from the Ampere architecture, even 32 bit).

Notably, the input operands are swapped in the function call. This operation ensure the matematical correctness compensating the layout difference between C++ and cuBLAS that are using two different storing method of the matrix, respectively row-major and column-major. With this simple method the function multiply two matrices without requiring an expensive physical transpose

Once computed, the result is copied back to the host. The process concludes with `cudaStreamSynchronize`, which blocks the thread to ensure data validity and accurate timing, followed by an update of the stream index for the next stream.

In listing 4.10 we outline the implementation for the 16-bit cuda gemm:

```
void cuda_gemm_16bit_p(__half* A, __half* B, __half* C,
                       int M, int N, int K) {
    cublasSetStream(handle, streams[i]);

    CHECK_CUDA(cudaMemcpyAsync(d_A[i], A,
        M * K * sizeof(__half), cudaMemcpyHostToDevice, streams[i]));
    CHECK_CUDA(cudaMemcpyAsync(d_B[i], B,
        K * N * sizeof(__half), cudaMemcpyHostToDevice, streams[i]));

    float a = 1.0f, b = 0.0f;

    cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                 N, M, K,
                 &a,
                 d_B[i], CUDA_R_16F, N,
                 d_A[i], CUDA_R_16F, K,
                 &b,
                 d_C[i], CUDA_R_16F, N,
                 CUBLAS_COMPUTE_32F,
                 CUBLAS_GEMM_DEFAULT_TENSOR_OP);

    CHECK_CUDA(cudaMemcpyAsync(C, d_C[i],
        M * N * sizeof(__half), cudaMemcpyDeviceToHost, streams[i]));
    CHECK_CUDA(cudaStreamSynchronize(streams[i]));
```

```
25
26      i = (i + 1) % N_STREAM;
27  }
```

Listing 4.10: cuda_runner.cu 16-bit logic

The logic stay the same of the 32 bit version, but it introduces changes to handle half-precision data, first the pointer types are replaced with the specific `__half` type, consequently, the `cublasGemmEx` function is configured with the `CUDA_R_16F` flag for the input and output data. However, it is worth pointing out that the computation type is kept as `CUBLAS_COMPUTE_32F`. This is a standard practice to ensures that while the data storage is compressed to 16 bit. The internal mathematical computation happens in 32 bit to prevent overflows.

The implementation for 8 bit integer matrices follows the exact same architectural pattern, differing only in data types, `int8_t` and flags `CUDA_R_8I`. The code can be found in full in Appendix A.1.3.

4.2.2 OpenVINO

The OpenVINO abstraction layer is designed to target the Intel NPU, just like the CUDA backend, the interaction between the C++ scheduler and the OpenVINO runtime is mediated by a standard C interface to ensure the ABI compatibility.

```
1  #ifdef __cplusplus
2  extern "C" {
3  #endif
4
5      void ov_init();
6      void ov_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
7      void ov_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
8      void ov_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);
9      void ov_free();
10
11 #ifdef __cplusplus
12 }
13 #endif
```

Listing 4.11: ov_wrapper.h interface

The header file `ov_wrapper.h` (see Listing 4.11) defines the interface exposed to the scheduler. It's the identical structure used for the other accelerators, declaring functions for initialization, cleanup, and matrix multiplication for 32-bit, 16-bit, and 8-bit data types.

The implementation resides in the `runner.cpp` file. While the function signatures are similar to the CUDA implementation, the internal logic is fundamentally different due to how OpenVINO function, unlike CUDA, which have all pre-compiled

kernels, OpenVINO works by building a so call computational graph and compiling it for the specific target device at runtime.

Compiling a model is an expensive and time consuming operation, it cannot be performed for every single task. To solve this, the system implements a simple caching mechanism.

```
ov::Core core;

using namespace std;

/* cache for compiled models */
static std::map<tuple<int, int, int>, ov::InferRequest> cache_32bit;
static std::map<tuple<int, int, int>, ov::InferRequest> cache_16bit;
static std::map<tuple<int, int, int>, ov::InferRequest> cache_8bit;

const size_t MAX_MAP_SIZE = 20;
size_t map_size_32bit=0;
size_t map_size_16bit=0;
size_t map_size_8bit=0;
```

Listing 4.12: ov runner.cpp chaching

The global variable `core`(see Listing 4.12), represents OpenVINO runtime, it will be utilized for discovering available devices and compiling the models.

To implement the caching mechanism, the system utilizes standard C++ maps for associating a specific set of matrix dimensions (represented by a tuple (M,N,K)) with a already compiled ov::InferRequest. This means that if the scheduler requests a matrix multiplication with dimensions that have been seen before, the system can retrieve the pre-compiled request immediately, skipping the expensive compilation step for already seen matrices.

Finally, to manage memory usage and prevent the cache from growing indefinitely in case there are many different matrices, a simple eviction policy is enforced using the `MAX_MAP_SIZE` constant and associated counters and if the number of cached models exceeds this limit, the cache is cleared to make room for new entries.

```
void ov_init_p() {
    bool available = false;

    for (auto &d : core.get_available_devices())
        if (d.find("NPU") != string::npos)
            available = true;

    if (!available) {
        cout << "[OPENVINO] NPU not present, aborting \n";
        exit(EXIT_FAILURE); /* shut down something is wrong */
    }
}
```

Listing 4.13: ov runner.cpp init

The init (Listing 4.13 function simply queries the OpenVINO Core to verify the presence of the NPU device. If the hardware is not found, the system aborts to prevent runtime failures during execution.

The core logic of the OpenVINO backend is found in the caching functions, such as `cache_32bit`. This function is responsible for constructing the neural network graph that represents a matrix multiplication.

```
void cache_32bit(tuple<int, int, int> key) {
    if (cache_32bit.find(key) == cache_32bit.end()) {
        map_size_32bit++;
        if (map_size_32bit >= MAX_MAP_SIZE) {
            cache_32bit.clear();
            map_size_32bit = 1;
        }
        auto A_ov = std::make_shared<ov::op::v0::Parameter>
            (ov::element::f32, ov::Shape{(size_t)get<0>(key),
            (size_t)get<2>(key)});
        auto B_ov = std::make_shared<ov::op::v0::Parameter>
            (ov::element::f32, ov::Shape{(size_t)get<2>(key),
            (size_t)get<1>(key)});

        auto matmul = std::make_shared<ov::op::v0::MatMul>(A_ov, B_ov);
        auto result = std::make_shared<ov::op::v0::Result>(matmul);

        auto model = std::make_shared<ov::Model>(result,
                                    ov::ParameterVector{A_ov, B_ov});

        ov::CompiledModel compiled_model = core.
                                        compile_model(model, "NPU");
        ov::InferRequest request = compiled_model.create_infer_request();

        cache_32bit[key] = request;
    }
}
```

Listing 4.14: ov runner.cpp caching function

The graph construction in Listing 4.14 consist in three steps, first two input nodes are created with the specific shape and precision in this case 32-bit float. Second, a `MatMul` node is instantiated taking these parameters as inputs, defining the GEMM Operation. Finally, the graph is wrapped into an `ov::Model` object and compiled for the NPU device. This step optimizes the graph for the specific hardware architecture, and the resulting request object is stored in the cache map.

Once the model is cached, the execution of the task is handled by the `ov_gemm_32bit_p` function.

```
void ov_gemm_32bit_p(void *A, void *B, void *C, int M, int N, int K){
    auto key = std::make_tuple(M, N, K);
    cache_32bit(key);

    ov::InferRequest& infer_request = cache_32bit[key];

```

```
    ov::Tensor tensor_A(ov::element::f32, ov::Shape{(size_t)M,
                                                    (size_t)K}, A);
    ov::Tensor tensor_B(ov::element::f32, ov::Shape{(size_t)K,
                                                    (size_t)N}, B);
    ov::Tensor tensor_C(ov::element::f32, ov::Shape{(size_t)M,
                                                    (size_t)N}, C);

    infer_request.set_input_tensor(0, tensor_A);
    infer_request.set_input_tensor(1, tensor_B);
    infer_request.set_output_tensor(0, tensor_C);

    infer_request.infer();
}
```

Listing 4.15: ov runner.cpp gemm 32 bit

This function (Listing 4.15) first calls the cache function to ensure the request object is available. Then, it wraps the raw pointers provided by the scheduler (A, B, C) into `ov::Tensor` objects. Notably this constructor uses the existing memory pointers without allocating new buffers or copying data (zero-copy). Finally, the tensors are bound to the input/output ports of the model, and the `infer()` method is called to trigger the execution on the NPU.

Finally, the code exposes the public C ABI by wrapping the internal C++ functions (Listing 4.16). This structure mirrors the approach used in the CUDA backend, ensuring that the scheduler sees a uniform interface across all accelerators.

```
extern "C" {
    void ov_init() {ov_init_p();}

    void ov_gemm_32bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_32bit_p(A, B, C, M, N, K);
    }

    void ov_gemm_16bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_16bit_p(A, B, C, M, N, K);
    }

    void ov_gemm_8bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_8bit_p(A, B, C, M, N, K);
    }

    void ov_free();
}
```

Listing 4.16: ov runner.cpp wrappers

A notable difference compared to the CUDA implementation is the `ov_free` function, which is empty. This is because the OpenVINO runtime relies on modern C++ features, specifically RAII and smart pointers, so when the program terminates or the objects go out of scope, the destructors for the `ov::Core` and the cached requests are invoked automatically, releasing the NPU resources explicit cleanup.

4.2.3 SYCL

The SYCL abstraction layer is designed to manage Intel GPUs. In this project, SYCL is utilized in conjunction with the Intel oneAPI Math Kernel Library (oneMKL) to perform high-performance matrix operations.

Just like the previous modules, the interaction with the scheduler is mediated by a pure C interface to ensure ABI compatibility. The header file `sycl_wrapper.h` defines the standard functions for initialization, execution (32-bit and 16-bit), and cleanup.

The implementation logic begins with the initialization phase (see Listing 4.17):

```
void init() {
    try {

        /* persisten streams */
        for(int i=0; i<N_STREAMS; i++) {
            streams.push_back(std::make_unique<SYCLStream>(MAX));
        }

    } catch (sycl::exception const& e) {
        std::cerr << "[SYCL ERROR] Init: " << e.what() << "\n";
        exit(1);
    }
}
```

Listing 4.17: sycl runner.cpp init

The primary task of the `init` function is to populate the global vector named `streams` with instances of `SYCLStream`. By iterating `N_STREAMS` times, it creates multiple independent execution contexts, allowing the scheduler to submit tasks in a round-robin manner.

Since SYCL is a higher-level C++ abstraction, the resource management is encapsulated in a dedicated struct named `SYCLStream` (see Listing 4.18).

```
#define N_STREAMS 2

using namespace std;

struct SYCLStream {
    sycl::queue q;
    float *d_A_f, *d_B_f, *d_C_f;
    sycl::half *d_A_h, *d_B_h, *d_C_h;

    SYCLStream(size_t max_elements) :
                q(sycl::gpu_selector_v,
                sycl::property::queue::in_order()) {
        d_A_f = sycl::malloc_device<float>(max_elements, q);
        d_B_f = sycl::malloc_device<float>(max_elements, q);
        d_C_f = sycl::malloc_device<float>(max_elements, q);

        d_A_h = sycl::malloc_device<sycl::half>(max_elements, q);
```

```
        d_B_h = sycl::malloc_device<sycl::half>(max_elements, q);
        d_C_h = sycl::malloc_device<sycl::half>(max_elements, q);

        if (q.get_device().get_info<sycl::info::device::name>()
                != "Intel(R) Arc(TM) Graphics") {
            std::cout << "Intel GPU NOT PRESENT\n";
            exit(EXIT_FAILURE);
        }
    }

    ~SYCLStream() {
        sycl::free(d_A_f, q); sycl::free(d_B_f, q); sycl::free(d_C_f, q);
        sycl::free(d_A_h, q); sycl::free(d_B_h, q); sycl::free(d_C_h, q);
    }
};

//static unique_ptr<sycl::queue> global_q;
static std::vector<std::unique_ptr<SYCLStream>> streams;
static int current_stream = 0;
static size_t MAX = (size_t)4096 * 4096;
```

Listing 4.18: sycl runner.cpp global data

The `SYCLStream` struct acts as a container that holds the state necessary for execution.

First, it contains a `sycl::queue`, which is the object used to submit commands to the device. The queue then is initialized with `in_order`, which simplifies the execution by ensuring that commands submitted to this queue are executed sequentially.

In the constructor, the device memory is pre-allocated using `sycl::malloc_device`, following the same optimization strategy used in the CUDA backend to avoid runtime allocation overhead. A specific check is also performed to verify that the code is running on the Intel GPU specifically, if the GPU is not found, the program aborts.

Finally, the struct includes a destructor, so that when the stream object is destroyed, all the memory buffers associated with it are explicitly freed, preventing memory leaks.

The execution logic is implemented in the following functions:

```
void sycl_gemm_32bit_p(float *A, float *B, float *C, int M, int N, int K) {
    SYCLStream& s = *streams[current_stream];

    s.q.memcpy(s.d_A_f, A, M * K * sizeof(float));
    s.q.memcpy(s.d_B_f, B, K * N * sizeof(float));

    float alpha = 1.0f, beta = 0.0f;

    oneapi::mkl::blas::row_major::gemm(s.q,
                                       oneapi::mkl::transpose::nontrans,
                                       oneapi::mkl::transpose::nontrans,
                                       M, N, K, alpha,
                                       s.d_A_f, K, s.d_B_f, N,
                                       beta, s.d_C_f, N);
```

```
15
16      s.q.memcpy(C, s.d_C_f, M * N * sizeof(float));
17      s.q.wait();
18
19      current_stream = (current_stream + 1) % N_STREAMS;
20  }
21
22  void sycl_gemm_16bit_p(sycl::half *A, sycl::half *B, sycl::half *C, int M,
        int N, int K) {
23      SYCLStream& s = *streams[current_stream];
24
25      s.q.memcpy(s.d_A_h, A, M * K * sizeof(sycl::half));
26      s.q.memcpy(s.d_B_h, B, K * N * sizeof(sycl::half));
27
28      sycl::half alpha = 1.0f, beta = 0.0f;
29      oneapi::mkl::blas::row_major::gemm(s.q,
30                                         oneapi::mkl::transpose::nontrans,
31                                         oneapi::mkl::transpose::nontrans,
32                                         M, N, K, alpha,
33                                         s.d_A_h, K, s.d_B_h, N,
34                                         beta, s.d_C_h, N);
35
36      s.q.memcpy(C, s.d_C_h, M * N * sizeof(sycl::half));
37      s.q.wait();
38
39      current_stream = (current_stream + 1) % N_STREAMS;
40  }
```

Listing 4.19: sycl runner.cpp execution

The `sycl_gemm_32bit_p` function (see Listing 4.19) orchestrates execution of the tasks, first, it selects the current stream and copies the input data from the host to the pre-allocated device buffers using `memcpy`.

Next, it invokes the matrix multiplication using the oneMKL library, thanks to this library the matrices A and B in this case can be passed directly with the `nontrans` flag, without needing to swap operands.

Finally, the result is copied back to the host, and `s.q.wait()` is called to block the execution until all operations in the queue are complete.

The 16-bit implementation is structurally identical but operates on the `sycl::half` data type. This allows the hardware to utilize half-precision units on the GPU, improving throughput.

```
1  extern "C" {
2      void sycl_init() {
3          init();
4      }
5
6      void sycl_gemm_32bit(void *A, void *B, void *C, int M, int N, int K) {
7          sycl_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
8      }
9
10     void sycl_gemm_16bit(void *A, void *B, void *C, int M, int N, int K) {
11         sycl_gemm_16bit_p((sycl::half*)A, (sycl::half*)B, (sycl::half*)C,
```

```
12                                  M, N, K);
13     }
14
15     void sycl_free() {
16         streams.clear();
17     }
18 }
```

Listing 4.20: sycl runner.cpp wrapper

The code concludes with the C-compatible wrappers (see Listing 4.20. These functions simply cast the generic `void*` pointers to the appropriate C++ types (float* or sycl::half*) and forward the call.

The `sycl_free` function simply calls `streams.clear()`. Because the system uses smart pointers (std::unique_ptr) and the SYCLStream struct has a defined destructor, clearing the vector automatically triggers the destruction of all stream objects.

4.2.4 OpenBLAS

Finally, the project includes a backend to target the host CPU. Unlike the other accelerators which required complex compilation and separate dynamic libraries to ensure compatibility, the CPU implementation was introduced directly into the scheduler codebase.

As already said in this thesis, this library was the last addition to the project, and since the scheduler and the OpenBLAS library share the same compiler, there were no compatibility issues to solve, infact, the implementation is located directly in the header file `cpu.hpp` inside the `scheduler/` directory.

The complete implementation is presented in Listing 4.21:

```
1  inline void cpu_init() {
2      openblas_set_num_threads(N_CORES);
3      omp_set_num_threads(N_CORES);
4  }
5
6  inline void cpu_gemm_32bit(void* A, void* B, void* C,
7                             int M, int N, int K) {
8
9      float* A_ = (float*) A;
10     float* B_ = (float*) B;
11     float* C_ = (float*) C;
12
13     cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
14                 M, N, K,
15                 1.0f,
16                 A_, K,
17                 B_, N,
18                 0.0f,
19                 C_, N);
20 }
```

```
21
22 inline void cpu_gemm_16bit(void* A, void* B, void* C,
23                             int M, int N, int K) {
24
25     uint16_t* pA = (uint16_t*)A;
26     uint16_t* pB = (uint16_t*)B;
27     uint16_t* pC = (uint16_t*)C;
28
29     std::vector<float> A_f32(M*K);
30     std::vector<float> B_f32(K*N);
31     std::vector<float> C_f32(M*N);
32
33     /* large overhead but we need it, not all cpus have 16 bit */
34     #pragma omp parallel for
35     for (int i = 0; i < M * K; ++i) {
36         A_f32[i] = half_to_float(pA[i]);
37     }
38
39     for (int i = 0; i < K * N; ++i) {
40         B_f32[i] = half_to_float(pB[i]);
41     }
42
43     cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
44                 M, N, K,
45                 1.0f,
46                 A_f32.data(), K,
47                 B_f32.data(), N,
48                 0.0f,
49                 C_f32.data(), N);
50
51     #pragma omp parallel for
52     for (int i = 0; i < M * N; ++i) {
53         pC[i] = float_to_half(C_f32[i]);
54     }
55 }
```

Listing 4.21: OpenBLAS CPU Implementation

The initialization function, `cpu_init`, is straightforward, it set the number of thread to use using `openblas_set_num_threads` and for OpenMP `omp_set_num_threads`. This ensures that the CPU utilizes all the available physical cores defined by the `N_CORES` macro.

For the 32 bit execution, the function `cpu_gemm_32bit` acts as a direct wrapper around the standard BLAS routine `cblas_sgemm`. It casts the generic pointers to `float*` and invokes the library function `CblasRowMajor`, which handles the input layout without transposition.

The 16-bit implementation (cpu_gemm_16bit), however, requires a workaround. Since the OpenBLAS library does not support half-precision arithmetic, the function performs a manual conversion. The logic is simple: first convert to Float,

1. Convert to float: the input 16 bit matrices are converted to standard 32 bit floats.

2. Compute: matrix multiplication is performed using the already available 32-bit routine.

3. Convert to half: The result is converted back to 16 bit.

To mitigate the performance cost of these conversions, the loops are parallelized using OpenMP (`#pragma omp parallel for`).

To handle the bitwise conversion between 16 bit and 32 bit formats without introducing dependencies, two standard IEEE 754 compliant functions were included in the project, these are well-known algorithms already widely used in the open-source community:

```
inline float half_to_float(uint16_t h) {
    uint32_t s = (h >> 15) & 0x00000001;
    uint32_t e = (h >> 10) & 0x0000001f;
    uint32_t m = h & 0x000003ff;

    if (e == 0) {
        if (m == 0) {
            uint32_t res = s << 31;
            float f; memcpy(&f, &res, 4); return f;
        } else {
            while (!(m & 0x00000400)) { m <<= 1; e -= 1; }
            e += 1; m &= ~0x00000400;
        }
    } else if (e == 31) {
        if (m == 0) { // Inf
             uint32_t res = (s << 31) | 0x7f800000;
             float f; memcpy(&f, &res, 4); return f;
        } else { // NaN
             uint32_t res = (s << 31) | 0x7f800000 | (m << 13);
             float f; memcpy(&f, &res, 4); return f;
        }
    }

    e = e + (127 - 15);
    m = m << 13;
    uint32_t res = (s << 31) | (e << 23) | m;
    float f; memcpy(&f, &res, 4); return f;
}
```

Listing 4.22: IEEE 754 Bitwise Conversion Helpers

The `float_to_half` function (available in the Appendix A.1.2, in `task.hpp`) performs the inverse operation.

4.3 AMM Scheduler

The core of this project is the `AMScheduler` class. While the underlying system involves thread management, synchronization, and hardware interactions, the design goal was to encapsulate all this complexity behind a simple interface.

The scheduler is implemented as a C++ class defined in `scheduler.hpp`, is implemented as a **RAII** pattern. This ensures that the lifecycle of the execution environment including worker threads, shared buffers, and hardware contexts is tied to the lifecycle of the scheduler object itself. However, it is important to point out that while standard C++ smart pointers are used for the threads and maps, explicit memory management is maintained for performance critical data structures, such as the task arrays and the internal shared buffers allowing the system to minimize overhead and ensuring that the developer has full visibility into the exact behavior of the memory.

The public interface is minimal, consisting of a constructor, a destructor, and two primary methods for task submission and synchronization (see Listing 4.23):

```
class AMScheduler {

public:

    /* Constructor: initializes threads, benchmarks if needed and
    performancemaps */
    AMScheduler(Logic logic) {
        init_threads();
        init_benchmarks();
        init_maps();
    }

    /* Destructor: stops threads and cleans up resources */
    ~AMScheduler() {
        stop_threads();
    }

    /* Non blocking: pushes tasks to the coordinator */
    void do_tasks(task* tasks, size_t n) {
        pending_tasks += n;
        for (int i=0; i<n; i++) {
            tasks[i].start_time = chrono::high_resolution_clock::now();
            shared_buffers[BT::CORDINATOR]->put(&tasks[i]);
        }
    }

    /* Blocking: waits until all pending tasks are completed */
    void wait();

    /* Prints execution statistics */
    void print_stats(task* tasks, int num_tasks);
};
```

Listing 4.23: Scheduler Public Interface

The constructor takes a `Logic` parameter, which decide the scheduling strategy to be used. Upon instantiation, the scheduler perform three operations: it initializes the worker threads and their contexts, loads the benchmark data if presents, and prepares the performance maps. The destructor ensures a clean shutdown of the system, it invokes the `stop_threads` function, which signals all active threads to terminate, waits for them to join, and all all the backends clean functins.

The `do_tasks` method serves as the entry point for workload. Before pushing the task into the system, it records the current timestamp into each task. This timestamp is crucial for the profiler to accurately measure the end-to-end latency of each task. Once timestamped, the task pointer is pushed into the coordinator's shared buffer.

The Listing 4.24 is an example of how the scheduler is utilized in the main.cpp file:

```
void test_scheduler() {

    /* 1. Instantiate Scheduler (starts threads) */
    AMScheduler scheduler(Logic::DYNAMIC);

    /* 2. Submit work */
    scheduler.do_tasks(Array_Of_Input_Tasks, Number_of_input_tasks);

    /* ..... */

    /* 3. Wait for completion */
    scheduler.wait();

    /* 4. Analyze profiler results */
    scheduler.print_stats(Array_Of_Input_Tasks, Number_of_input_tasks);
}
```

Listing 4.24: Example usage

The usage of the scheduler is extremely simple, the developer only needs to instantiate the class passing the logic and submit the array of tasks. The scheduler handles all the complexity of thread management and hardware dispatching internally. For the explaination of the tasks creation see Sec. 4.3.4

4.3.1 Thread Initialization

The `init_threads` function is responsible for setting up the environment based on the available hardware (see Listing 4.25):

```
1  void init_threads() {
2
3      /* from the fastest to the slowest */
4      /* init here and not in the threads */
5  #ifdef ENABLE_CUDA
6          init_acc(BT::CUDA);
7          bts.push_back(BT::CUDA);
8  #endif
9  #ifdef ENABLE_SYCL
10             init_acc(BT::SYCL);
11             bts.push_back(BT::SYCL);
12 #endif
13 #ifdef ENABLE_OPENVINO
14             init_acc(BT::OPENVINO);
15             bts.push_back(BT::OPENVINO);
16 #endif
17 #ifdef ENABLE_OPENBLAS
18             init_acc(BT::OPENBLAS);
19             bts.push_back(BT::OPENBLAS);
20 #endif
21
22     /* initialize atomics for dynamic strategy */
23     if(strategy == Logic::DYNAMIC) {
24         #ifdef SLEEP
25         cout << "[SCHEDULER] ERROR remove "#define SLEEP" with Dynamic
       logic"
26         exit(EXIT_FAILURE);
27         #endif
28         for (int i=0; i<bts.size(); i++) {
29             busy[bts[i]] = new atomic<bool>;
30             *busy[bts[i]] = false;
31         }
32     }
33
34     for (BT i : bts) {
35         shared_buffers[i] = make_unique<SharedBuffer>(BUFFER_LENGHT);
36         threads[i] = make_unique<thread>(&AMScheduler::worker_thread, this,
37                                                 i, shared_buffers[i].get());
38     }
39
40     if (bts.size() == 0) {
41         PRINT("[SCHEDULER] ATTENTION, no accelerator\n");
42         exit(EXIT_FAILURE);
43     }
44
45     /* coordinator thread */
46     shared_buffers[BT::CORDINATOR] =
47                         make_unique<SharedBuffer>(BUFFER_LENGHT);
48
49     /* ref because the thread function try to copy all the parameters */
50     threads[BT::CORDINATOR] = make_unique<thread>(
51                                 &AMScheduler::coordinator_thread,
52                                 this, ref(shared_buffers));
```

```
53 }
```

Listing 4.25: Init Thread

This function first checks which accelerators were enabled during compilation using preprocessor macros passed with CMake. For each enabled backend, it calls the specific initialization function for each backend, inside the dynamic libraries, and adds the backend type to the active list `bts`. After it iterates through the active backends to create the worker threads. For each accelerator, a dedicated `SharedBuffer` is instantiated to hold its incoming tasks, and a standard C++ `std::thread` is spawned to run the `worker_thread` function.

Finally, the function initializes the **Coordinator Thread**. This thread is equipped with its own input buffer and is responsible for fetching tasks from the user and dispatching them to the workers (for implementation see Sec. 4.3.8). The reference to the shared buffers map is passed to the coordinator to allow it to push tasks to any worker inside the logics.

4.3.2 Workers

While the Coordinator thread manages the complexity of deciding where to send the data, the worker threads are designed to be extremely lightweight and simple. Their only purpose is to retrieve a task from their buffer, execute it, and signal the completion.

The implementation of the worker function (see Sec. 4.26):

```
1  void worker_thread(BT bt, SharedBuffer *buffer) {
2      task* task = nullptr;
3      PROF(profiler.init_last_work(bt));
4
5      if (strategy == Logic::DYNAMIC) {
6
7          /* Dynamic execution */
8          while (threads_keep_running) {
9
10             task = buffer->get();
11             if (!task) {continue;}
12
13             PROF(profiler.start_worker(bt));
14             handle_task(bt, task);
15             PROF(profiler.end_worker(bt));
16
17             *busy[bt] = false;
18             pending_tasks--;
19         }
20
21     } else {
22
23         /* Static execution */
24         while (threads_keep_running) {
```

```
25
26              task = buffer->get();
27
28              /* if there isn't a task we check if we have to shutdown */
29              if (!task) {continue;}
30
31              PROF(profiler.start_worker(bt));
32              handle_task(bt, task);
33              PROF(profiler.end_worker(bt));
34              pending_tasks--;
35          }
36
37      }
38  }
```

Listing 4.26: Worker Thread Implementation

The implementation is a continuous loop that retrieves a task from the shared buffer and processes it. The function `buffer->get()` is the entry point. As detailed in the Shared Buffer section (see Sec. 4.3.5), if the macro `SLEEP` is defined, this function blocks the thread and puts it to sleep until a new task arrives. This prevents the worker from consuming CPU cycles while waiting. If `SLEEP` is not defined, the thread spins (busy waiting) to react faster.

The behavior is slightly different based on the scheduler strategy:

- **Static:** The worker simply processes its queue as fast as possible without signaling.
- **Dynamic:** The worker notify the Coordinator when it finishes a task. It sets the atomic flag `*busy[bt]` to false so the Coordinator knows this accelerator is ready for a new task.

The `handle_task` function acts as a wrapper, isolating the scheduler from the specific backend implementations offered by the dynamic libraries linked at runtime.

```
1  /* task handler for the workers, choose the right backend_type */
2  inline void handle_task(BT backend_type, task *task) {
3      switch (backend_type) {
4      case BT::CUDA:
5  #ifdef ENABLE_CUDA
6          if (task->type == Type::FLOAT)
7              cuda_gemm_32bit(task->A,task->B,task->C,
8                              task->M, task->N, task->K);
9          if (task->type == Type::HALF)
10             cuda_gemm_16bit(task->A,task->B,task->C,
11                             task->M, task->N, task->K);
12 #endif
13         break;
14
15     case BT::SYCL:
16 #ifdef ENABLE_SYCL
17         if (task->type == Type::FLOAT)
```

```
18              sycl_gemm_32bit(task->A,task->B,task->C,
19                              task->M, task->N, task->K);
20          if (task->type == Type::HALF)
21              sycl_gemm_16bit(task->A,task->B,task->C,
22                              task->M, task->N, task->K);
23  #endif
24          break;
25
26      /* .. same for OpenVINO and OpenBLAS .. */
27
28      default:
29          cerr << "[SCHEDULER] ERROR handle task\n";
30          exit(EXIT_FAILURE);
31      }
32
33      task->end_time = chrono::high_resolution_clock::now();
34  }
```

Listing 4.27: Task Handler

This function (see Sec. 4.27) performs three simple steps, it checks the `backend_type` enum to decide which accelerator to use, then checks if the task requires 32-bit or 16-bit precision and calls the corresponding C-wrapper function from the dynamic libraries.Immediately after the computation finishes, it saves the current time into `task->end_time`, this gives the precise execution time for the metrics.

4.3.3 Benchmarks Routines

Once the threads are running, the coordinator thread needs specific data to make decisions inside the static logics (see Sec. 4.3.9). This data comes from the data profiling saved in CSV files create by `init_benchmarks` function and later uploaded inside the `PerformanceMap` structure (see Sec. 4.3.6) by the `init_maps` function.

```
1  /* upload the banchmark files and generate them if not presents */
2  void init_benchmarks() {
3      for (BT i : bts) {
4
5          string filename_f;
6          string filename_h;
7
8          filename_f = get_benchmark_filename(i, Type::FLOAT);
9          filename_h = get_benchmark_filename(i, Type::HALF);
10
11         if (filesystem::exists(filename_f))
12             PRINT("[SCHEDULER] File: " <<
13                     filename_f <<  " opened.\n");
14         else
15             benchmark(i, Type::FLOAT ,filename_f);
16
17         if (filesystem::exists(filename_h))
18             PRINT("[SCHEDULER] File: " <<
19                     filename_h <<  " opened.\n");
20         else
```

```
21              benchmark(i, Type::HALF ,filename_h);
22
23      }
24  }
25
26  /* create the map for each accelerator */
27  void init_maps() {
28      for (BT i : bts) {
29          string filename_f;
30          string filename_h;
31          filename_f = get_benchmark_filename(i, Type::FLOAT);
32          filename_h = get_benchmark_filename(i, Type::HALF);
33
34          if (filesystem::exists(filename_f)
35              && filesystem::exists(filename_h)) {
36
37              /* performance map creation */
38              bts_map[i] = make_unique<PerformanceMap>(STEP_SIZE,
39                                                       i, filename_f, filename_h);
40
41          } else
42              /* ... error prints and exit ... */
43      }
44  }
```

Listing 4.28: Init Benchmarks and Maps

The `init_benchmarks` (Listing 4.28) function ensures that the data exists. It iterates through the list of active accelerators and checks for the existence of specific CSV files for both 32 and 16 bit data types. If a file is found, the system proceeds, if a file is missing, the system automatically triggers the benchmarking routine to generate the data from scratch.

Subsequently, again within the constructor, the `init_maps` function is called. Its role is to load the data from the disk into the memory. It reads the CSV files generated in the previous step and populates the `PerformanceMap` objects for each accelerator. This ensures that when the scheduler is running, it can query the expected execution time of a matrix multiplication in constant time O(1) without needing to read from the disk again. If a file is missing at this stage, the system considers it a critical error and shuts down, as the scheduler cannot function without its performance models. If a file is missing at this stage, the system shuts down, as the scheduler cannot function without its performance models.

The accuracy of the scheduler depends entirely on the quality of the data collected during the benchmarking phase (and small tweaks inside performanceMaps). This process is managed by the `benchmark` and `benchmark_acc` functions.

```
1  /* call benchmark_acc for each test matrix*/
2  inline void benchmark(BT bt, Type type, string filename) {
3      vector<int> dims;
4
5      for (int d = STEP_SIZE; d <= STEP_SIZE*STEP_TOTAL; d += STEP_SIZE)
6          dims.push_back(d);
```

```
7
8       for (int m : dims)
9           for (int n : dims)
10              for (int k : dims)
11                  benchmark_acc(bt, type, filename, m, n, k);
12
13      PRINT("[SCHEDULER] Benchmark ===\n");
14      PRINT("Results saved in bin/csv/" << filename << "\n");
15  }
```

Listing 4.29: Benchmark Function

The `benchmark` (see Listing 4.29) function acts as the driver. It defines the search space for the matrix dimensions (M, N, K) based on a `STEP_SIZE` (defined in the file `config.hpp`). It uses three nested loops to iterate through all possible combinations of dimensions, calling the measurement function for each configuration.

The measurement logic is implemented in `benchmark_acc` (see Listing 4.30):

```
1  /* benchmark the accelerator and write in the csv*/
2  inline void benchmark_acc(BT bt, Type type, string filename,
3                            int M, int N, int K) {
4      chrono::high_resolution_clock::time_point start_time;
5      chrono::high_resolution_clock::time_point end_time;
6      chrono::duration<double, std::milli> duration;
7
8      const int N_RUNS = 4;
9      const int N_TASKS = N_RUNS + 2;
10
11     double total_ms = 0.0;
12     double jit_ms = 0.0;
13     double no_jit_ms = 0.0;
14
15     /* tasks init */
16     task* tasks = init_tasks(N_TASKS, M, N, K, type);
17
18     /* warmup and jit detection */
19     start_time = chrono::high_resolution_clock::now();
20     handle_task(bt, &tasks[0]);
21     end_time = chrono::high_resolution_clock::now();
22     duration = end_time - start_time;
23     jit_ms = duration.count();
24
25     start_time = chrono::high_resolution_clock::now();
26     handle_task(bt, &tasks[1]);
27     end_time = chrono::high_resolution_clock::now();
28     duration = end_time - start_time;
29     jit_ms = jit_ms - duration.count();
30
31     /* runs */
32     for (int i = 2; i < N_TASKS; i++) {
33         start_time = chrono::high_resolution_clock::now();
34         handle_task(bt, &tasks[i]);
35         end_time = chrono::high_resolution_clock::now();
36         duration = end_time - start_time;
37         total_ms += duration.count();
38     }
```

```
39
40      clean_tasks(tasks, N_TASKS);
41
42      double avg_ms = total_ms/N_RUNS;
43
44      ofstream file(filename, ios::app);
45
46      if (file.is_open()) {
47          if (filesystem::file_size(filename) == 0)
48              file << "Accelerator,DataType,M,N,K,Avg_Time_ms,Jit_Time_ms\n";
49
50          string acc_str = get_acc_string(bt);
51
52          file << acc_str << "," << type << "," << M << "," << N <<
53                  "," << K << "," << avg_ms << "," << jit_ms << "\n";
54
55          file.close();
56      } else {
57          cerr << "[SCEDULER] ERROR Cannot open file: " << filename << "\n";
58      }
59  }
```

Listing 4.30: Benchmark Accelerator Function

Measuring the performance of accelerators like GPUs or NPUs is not trivial because these frameworks often perform a just in time compilation (JIT). This means that the very first time a specific matrix operation is executed, the driver must compile the kernel for the hardware, resulting in a significantly higher execution time compared to subsequent runs.

To handle this and isolate the overhead, the function uses a specific strategy; it allocates an array of tasks instead of reusing a single task object. This is done to prevent unrealistic CPU cache hits that might occur if the exact same memory pointers were reused constantly.

The measurement proceeds in three steps: the first task is executed. This run includes the time required for data transfer, computation, and any kernel compilation or driver initialization overhead. The second task is executed immediately after. Since the kernel is already compiled, this run represents the execution time without the JIT expenses, then the system calculates the **JIT time** by subtracting the duration of the second run from the duration of the first run.

The remaining tasks are executed in a loop, their durations are summed and divided by the number of runs to obtain a stable average execution time (`Avg_Time_ms`).

Finally, the results including the accelerator name, data type, matrix dimensions, average time, and JIT time are appended to the corresponding CSV file. This distinction between JIT time and execution time is crucial for the scheduler: during the first few iterations of an algorithm, the scheduler must account for the JIT overhead, and all of this complexity is hidden in the `PerformanceMap`.

4.3.4 Tasks

To allow the scheduler to manage workloads efficiently without being tied to a specific data type, the project introduces an abstraction called `Task`.

The definition of the task structure is below:

```
enum Type : int { FLOAT, HALF, UINT8 };

typedef struct {

    void *A;
    void *B;
    void *C;

    Type type; /* 1 float, 2 half, 3 8bit */
    int M;
    int N;
    int K;

    /* benchmarking */
    chrono::high_resolution_clock::time_point start_time;
    chrono::high_resolution_clock::time_point end_time;

} task;
```

Listing 4.31: Task Structure

The structure utilizes generic pointers (void *) for the matrices A, B, and C so it allows the same code to handle all the data type supported uniformly. The specific type of the data is determined by the Type enum field.

Furthermore, the task structure plays a crucial role in profiling, containing two high-resolution timestamp fields, `start_time` and `end_time`, the scheduler can records the exact moment a task is submitted, and the worker thread records the moment the computation finishes. This allows for precise calculation of the latency and throughput.

The typical lifecycle of a task within the application is simple:

1. The tasks are allocated and filled with data in the main thread.
2. They are passed to the scheduler via `do_tasks` function exposed via the scheduler object.
3. Once processing is complete, the memory is explicitly freed with the `clean_tasks` function.

To automate the data generation process for testing every single workload supported by the scheduler, the `init_tasks` function was used to create arrays of identical matrices, while `init_hetero_tasks` was employed to create arrays of heterogeneous

matrices. These datasets constitute the input used for all the tests performed on the scheduler.

```
inline task* init_tasks(size_t n_task, int m, int n, int k, Type t) {
    task* array_task = new task[n_task];

    for (int i = 0; i < n_task; i++) {
        array_task[i].M = m;
        array_task[i].N = n;
        array_task[i].K = k;
        array_task[i].type = t;

        if (t == Type::FLOAT) {
            array_task[i].A = new float[m * k];
            array_task[i].B = new float[k * n];
            array_task[i].C = new float[m * n];

            init_32bit(array_task[i].A,array_task[i].B,array_task[i].C, m,
    n, k);

        } else if (t == Type::HALF) {
            /* .. same as 32 bit .. */
        }
    }
    return array_task;
}
```

Listing 4.32: Init Tasks Function

The implementation is straightforward, the function `init_task` (see Listing 4.32) simply take the number of tasks to create, the matrix dimensions and the desired type of the matrices, it allocate the array of tasks, then the system populate the matrices with random numbers thanks to the `init_32bit` function:

```
inline void init_32bit(void* A, void* B, void* C, int M, int N, int K) {
    float* pA = (float*)A;
    float* pB = (float*)B;
    float* pC = (float*)C;

    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<float> dis(-1.0f, 1.0f);

    for (int i = 0; i < M * K; ++i)
        pA[i] = dis(gen);

    for (int i = 0; i < K * N; ++i)
        pB[i] = dis(gen);

    for (int i = 0; i < M * N; ++i)
        pC[i] = 0.0f;
}
```

Listing 4.33: Init 32 bit Function

The `init_32bit` (see Listing 4.33) function casts the generic void pointers back to float* and uses the C++ standard random number generator to fill the input matrices A and B. The output matrix C is initialized to zero for the results.

The 16 bit initialization function is omitted because it follows the same pattern but includes an additional step to convert the generated random floats into half precision bit, same for the function `init_hetero_tasks`, it simply iterate on random number to create the a stream of random dimension matrices.

4.3.5 SharedBuffer

The SharedBuffer is the tool used for communication between the threads, since the Coordinator and the Workers run independently, they need a safe place to exchange data.

```
class SharedBuffer {
    private:
        unique_ptr<task*[]> data;
        size_t size = 0;

        alignas(64) std::atomic<size_t> write_i{0};
        alignas(64) std::atomic<size_t> read_i{0};

    public:
        /* constructor */
        SharedBuffer(size_t s) : size(s), data(new task*[s]) {}

        ~SharedBuffer() {}

        /* put one task pointer inside, sleep if full*/
        void put(task* ele) {

#ifdef SLEEP
            while (((write_i + 1) % size) == read_i) {
                usleep(PUT_SLEEP);
            }
#else
            while (((write_i + 1) % size) == read_i) {}
#endif
            data[write_i] = ele;
            write_i = (write_i + 1) % size;
        }

        /* get one task pointer sleep if empty, return to the caller after
         * 5 attempt*/
        task* get() {
            int c = 0;

#ifdef SLEEP
            while (write_i == read_i){
                c++;
                usleep(GET_SLEEP);
                if (c >= 5) {return nullptr;}
            }
#else
            while (write_i == read_i){
                c++;
                if (c >= 50) {return nullptr;}
            }
#endif
            task* ele = data[read_i];
            read_i = (read_i + 1) % size;

            return ele;
        }
};
```

Listing 4.34: Shared Buffer Implementation

This class (see Listing 4.34) was implemented specifically for this scheduler to be as simple and fast as possible, avoiding the complex locking mechanisms usually found in standard libraries.

The logic is based on a circular buffer managed by two counters: `write_i` (where to put new data) and `read_i` (where to take data) and by using atomic variables, the threads can update these counters without interfering with each other.

The put function is used by the Coordinator, it checks if there is space, if the buffer is full, it waits. The get function is used by the Worker to pick up a task, if the buffer is empty, it waits for new data.

The task array is encapsulated within a `std::unique_ptr`, this ensures that the memory is automatically released when the associated SharedBuffer object is destroyed. However, the underlying structure remains a simple raw array.

Is important to note that the `alignas(64)` directive is applied to the index variables to prevent false sharing. This is a standard optimization technique, by forcing the read and write counters to reside on separate CPU cache lines (thanks to the padding that the compiler insert), it ensures that the Coordinator and Worker threads can update their respective counters independently without triggering unnecessary synchronization traffic between the processor cores, speeding up the process.

A key detail is in the get function, that it doesn't wait forever, as shown in the code, if the buffer remains empty for a certain number of attempts (5 with sleep, 50 without), the function returns nullptr. This is done for a specific reason, the worker threads run inside a while loop controlled by the scheduler. If the get function blocked indefinitely waiting for a task, the thread would get stuck there and could never check if the program is trying to shut down. By returning control to the caller periodically, the worker can check if it should stop running or keep waiting for new work.

By default, this is a busy waiting: the thread keeps checking the variable continuously, this is very fast but consumes all of the CPU core. To fix this, the `SLEEP` macro can be enabled. The thread takes a small sleep if it has to wait, freeing up the CPU. This small feature was implemented primarily for future optimizations for maximizing resource efficiency. However, since it inevitably introduces a slight overhead (increased latency), it remains disabled and was not utilized during the experimental benchmarks presented in this thesis.

4.3.6 Performance Map

The Performance Map is a specialized data structure utilized by the Coordinator thread. Its primary purpose is to act as a predictive model, providing the expected execution time for a specific task on a specific accelerator.

As explained in the previous section the raw data is acquired through automated benchmark routines (see Sec. 4.3.3). However, reading from CSV files isn't an option latency wise, therefore, the PerformanceMap class loads this data into an optimized in-memory structure at startup.

The class definition and its global variables are presented in Listing 4.35

```
class PerformanceMap {

    struct map_struct {
        unordered_map<unsigned long long, tuple<double, double>> grid_map;

        long long last_M = -1;
        long long last_N = -1;
        long long last_K = -1;
        double last_result = -1.0;
        double max_result = -1.0;
    };

    map_struct map_f;
    map_struct map_h;

    unordered_set<unsigned long long> jit_cache_f;
    unordered_set<unsigned long long> jit_cache_h;

    /* which accelerator am i */
    BT bt;

    long long STEP_SIZE;

public:

    PerformanceMap(int step_size, BT bt,
                   string filename_f,
                   string filename_h)
                   : STEP_SIZE(step_size), bt(bt) {
        init_maps(filename_f, filename_h);
    }

    double query(long long M, long long N, long long K, Type type, bool
    add_jit);
```

Listing 4.35: PerformanceMap Class

The core of the structure is the `map_struct`, which contains an `unordered_map`. This map uses an unsigned long long as a key (explained shortly) and stores a tuple containing the execution time and the JIT compilation time. To avoid redundant lookups for homogeneous workloads (where tasks are identical), the structure also

includes a caching mechanism, `last_M`, `last_N`, and `last_result` that returns the previous result immediately if the dimensions match.

Two separate instances of this struct are maintained: `map_f` for 32 bit floating point tasks and `map_h` for 16 bit half precision tasks, additionally, the `jit_cache` sets are used to track which matrix dimensions have already triggered a JIT compilation, allowing the system to predict when the "cold start" penalty must be paid. The bt variable identifies which accelerator this map belongs to, enabling specific heuristics for each hardware type.

The public interface is minimal: besides the constructor that loads the data from files, only the query function is exposed to the Coordinator.

To index the 3D matrix dimensions efficiently within a hash map, a bit-packing strategy was chosen over using complex objects or tuples as keys. This approach minimizes hashing overhead and ensures fast lookups.

```
/* return the successor of val in our step_size */
long long round_up(long long val) {
    if (val == 0) return STEP_SIZE;
    return ((val + STEP_SIZE - 1) / STEP_SIZE) * STEP_SIZE;
}

/* return the key of val in our step_size */
unsigned long long get_key(long long M, long long N, long long K) {
    long long rM = round_up(M);
    long long rN = round_up(N);
    long long rK = round_up(K);

    /* 64 bit  |4bit nil|20bit M|20bit N|20bit K| */
    return ((unsigned long long)rM << 40) | ((unsigned long long)rN << 20)
    | (unsigned long long)rK;
}
```

Listing 4.36: Get key Implementation

The `get_key` (see Listing 4.36) function first rounds up the dimensions to the nearest `STEP_SIZE` (the interval used during benchmarking) to ensure that arbitrary task sizes map to the nearest available data point. Then, it packs the three task dimensions into a single 64 bit integer by shifting bits: M the high 20 bits, N the middle, and K the low bits. This creates a unique integer identifier for any matrix configuration.

The query function is the main part of this component, it takes the matrix dimensions, the data type, and a boolean flag `add_jit`, part of the code is shown in Listing 4.37:

```
/* return the time, add_jit true = populate the jit cache */
double query(long long M, long long N,
             long long K, Type type, bool add_jit) {

    /* ... setup pointers to the correct map and cache ... */

    unsigned long long key = get_key(M, N, K);

    /* we cache the last results */
    if (M == data->last_M && N == data->last_N && K == data->last_K)
        {return data->last_K;}

    data->last_M = M; data->last_N = N; data->last_K = K;

    /* return if we find the key */
    if (data->grid_map.find(key) != data->grid_map.end()) {
        time_ms = get<0>(data->grid_map[key]);
        jit_ms = get<1>(data->grid_map[key]);
        /* add jit_ms only if openvino never see M N K*/
        if (bt == BT::OPENVINO) {
            /* mitigate overhead when other accelerator are running */
            time_ms += time_ms * 0.1;
            if (jit_cache->find(key) == jit_cache->end()){
                if (add_jit)
                    jit_cache->insert(key);
                time_ms += jit_ms * 1.3;
            }
        }

        /* add jit_ms only if sycl never see N */
        if (bt == BT::SYCL) {
            /* mitigate overhead when other accelerator are running */
            time_ms += time_ms * 0.1;
            unsigned long long sycl_key = (unsigned long long) N;
            if (jit_cache->find(sycl_key) == jit_cache->end()){
                if (add_jit) jit_cache->insert(sycl_key);
                time_ms += JIT_MS_SYCL;
            }
        }

        /* mitigate openblas error in bencharks  */
        if (bt == BT::OPENBLAS) time_ms = time_ms * 0.5;
        data->last_K = time_ms;
        return time_ms;
    }

    /* if we don't have data on this matrix, just return max time * 2 */
    time_ms = data->max_result * 2;
    /* add jit expences */
    if (bt == BT::SYCL || bt == BT::OPENVINO) time_ms += JIT_MS_SYCL;
    /* cache the result */
    data->last_K = time_ms;
    return time_ms;
}
```

Listing 4.37: Query Implementation

The `add_jit` boolean serve a specific task, the Coordinator often needs to query the cost of a task purely for planning purposes, without actually scheduling it. In

these cases, `add_jit` is set to false, preventing the map from marking the matrix as "seen" in the JIT cache, ensuring that the compilation cost is correctly applied only when the task is actually dispatched.

The function `query` implements specific heuristics derived and observed from extensive black box analysis of the drivers, as their internal implementations are not public:

- **CUDA**: For the nvidia backend, no JIT logic is applied, the driver handles kernel caching transparently and efficiently, so the raw execution time from the benchmark is returned directly.
- **OpenBLAS**: Similarly, the CPU backend has no compilation overhead, however, we observed that during the benchmark routines, the processor often failed to maintain the maximum Boost frequency consistently or suffered from thermal throttling, resulting in benchmark times that were slightly slower than real world execution. To correct this, a scaling factor is applied to the estimated time to align it with the actual performance observed during runtime.
- **OpenVINO**: The NPU driver behavior was straightforward, the JIT compilation times measured during benchmarking matched the runtime behavior perfectly. Therefore if a matrix key is not in the `jit_cache`, the stored JIT time is added to the result. Additionally, we observed that when the NPU runs in parallel with other accelerators, it suffers from a slight performance degradation due to system bus contention or power sharing. To mitigate this, a small percentage of overhead is added to the base execution time.
- **SYCL**: This was the most complex driver to analyze. Our tests revealed that the driver compiles an optimized kernel for a wide range of matrix sizes, but it triggers a recompilation whenever the N dimension changes. Furthermore, the compilation cost was found to be constant, around 100-120 ms, regardless of the matrix size. Consequently, the benchmarked JIT times were discarded in favor of adding a fixed constant `JIT_MS_SYCL`) whenever a new N is encountered. Similar to the NPU, the Intel GPU also exhibits slight overhead during parallel execution, which is compensated for by adding a small factor to the predicted time.

All of these tweaks exhibit, as it will be shown in the next section, a good behavior in predicting the time of completition of a task, furthermore the code used for the black box analysis is intirely in the tests.hpp file, found in Appendix **??**.

4.3.7 Profiler

To evaluate the performance and the efficiency of the scheduling logics, the `Profiler` tool was developed integrated directly into the scheduler's codebase. It acts as an observer, recording precise timestamps and sensor data during the execution of workloads without interfering with the logic.

The class provides a simple interface to start and stop counters for various metrics. Its primary responsibilities are:

- Coordinator Latency, tracking the exact time spent in specific phases Fetching, Logic, Dispatching using high-resolution clocks.
- Worker Statistics, monitoring each worker thread to calculate occupancy.
- Power Monitoring, measuring the total energy consumed by the hardware during the workspan.

See Listing 4.38 for a simplified view of the class structure and its key components:

```
struct EnergyConsumption {};  /* Consumption stats */
struct Metric {};             /* Coordinator stats */
struct WokerMetric {};        /* Workers stats */

class Profiler {
public:

    /* power monitor helpers */
    void start_power_monitor() {
        #ifdef ENABLE_INTEL_POWER_PROFILE
        /* read RAPL files */
        #endif

        #ifdef ENABLE_CUDA
        /* call NVML library */
        #endif
    }
    void stop_power_monitor();

    /* scheduler logic monitor helpers */
    void start_fetch();
    void start_logic();
    void start_dispatch();
    void stop_fetch();
    void stop_logic();
    void stop_dispatch();
    void init_last_work();
    void start_worker();
    void end_worker();
    void record_sample();
    void print_stats();
    void save_to_csv();
};
```

Listing 4.38: Profiler Class Structure

For the NVIDIA GPU, collecting data was straightforward thanks to the NVML library, the function `nvmlDeviceGetTotalEnergyConsumption` provides a monotonic counter of the total energy consumed by the board in millijoules since the driver started. By reading this counter before and after the workload, the exact consumption is derived.

For the Intel hardware the system relies on RAPL. On Linux systems these hardware counters are exposed as standard text files in `/sys/class/powercap/`, the Profiler reads the energy values in microjoules directly from these files. While RAPL allows for fine grained domains (like DRAM or specific cores), monitoring the entire CPU Package was deemed sufficient for our goals, as we are interested in the total system efficiency gain, furthermore the GPU and NPU are also included in the CPU package.

Using Joules as the primary metric provides a way to compare efficiency, as it inherently accounts for both the power draw and the duration of the task.

Finally, all collected metrics are aggregated and appended to a results.csv file. This automated data collection was crucial for generating the results presented in the next chapter.

4.3.8 Coordinator

The heart of the Scheduler is the Coordinator thread. It is the decision making engine that determines how tasks are distributed among the available workers. As previously mentioned, the coordinator supports multiple task dispatch logics, ranging from simple baselines to more complex algorithms.

Just like the worker threads, the Coordinator operates within a continuous while loop controlled by the `threads_keep_running` atomic flag. Before the loop, a switch statement selects the active logic based on the configuration provided during the scheduler's object instantiation.

Baseline: CUDA only and Round Robin

To validate the internal architecture and establish a performance baseline, two simple strategies were implemented first.

`CUDA_ONLY` is the most basic logic simply bypasses the heterogeneity of the system and dispatch all the tasks to the fastest accelerator. This serves as the primary baseline: any strategy must outperform this single device execution at least to be considered effective.

```
case Logic::CUDA_ONLY:

PROF(profiler.start_power_monitor());

if (buffers[BT::CUDA] == nullptr) {
    cerr << "[SCHEDULER] Error, cuda logic but no cuda card";
    exit(EXIT_FAILURE);
}

while (threads_keep_running) {
    single_task = buffer->get();
    if (!single_task) {continue;}
    buffers[BT::CUDA]->put(single_task);
}

PROF(profiler.stop_power_monitor());

break;
```

Listing 4.39: CUDA Only Logic

The code is straightforward (see Listing 4.39), it fetches a task from the input buffer and immediately pushes it into the CUDA worker's buffer. The PROF macros are used to trigger the start and stop of the monitoring tools within the Profiler, and thanks to this macro, can be turned off for minimizing the overhead simply by not defining `ENABLE_PROFILING`.

To verify the system's ability to manage multiple accelerators concurrently, a Round

Robin logic was implemented.

```
case Logic::ROUND_ROBIN:

while (threads_keep_running) {
    single_task = buffer->get();
    if (!single_task) {continue;}

    buffers[bts[i]]->put(single_task);

    i = (i+1) % bts_len;
}
break;
```

Listing 4.40: Round Robin Logic

This logic (see Listing 4.40) distributes tasks equally among all available workers in a cyclic manner, while effective for validating, this strategy is definitely not performance oriented.

4.3.9 Static Partitioning

This strategy represents the core logic for handling homogeneous workloads (streams of identical matrices) in a batch style manner. Ideally, to minimize the makespan, the workload should be distributed proportionally to the computing power of each accelerator.

The algorithm operates in batches defined by `BATCH_SIZE`. The execution flow is divided into three distinct phases: Fetch, Logic, and Dispatch.

First, the coordinator retrieves a batch of tasks from the input buffer. Then, it queries the PerformanceMap to estimate the execution time for a single task on each accelerator.

```
case Logic::STATIC_PARTITIONING:

PROF(profiler.start_power_monitor());

while (threads_keep_running) {
    /* fetch */
    PROF(profiler.start_fetch());
    while (c < BATCH_SIZE) {
        tasks[c] = buffer->get();
        if (!tasks[c]) {break;}
        c++;
    }
    PROF(profiler.stop_fetch());

    if (c == 0) {continue;}

    /* calculate speeds */
    PROF(profiler.start_logic());
```

```
19      total_speed = 0.0;
20      for (int j=0; j<bts_len; j++){
21          double ms = bts_map[bts[j]]->query(tasks[0]->M, tasks[0]->N,
22                                             tasks[0]->K, tasks[0]->type,
    true);
23
24          /* speed = 1 / time */
25          acc_speed[j] = 1.0 / (ms + 1e-9);
26          total_speed += acc_speed[j];
27      }
28
29      /* .... */
```

Listing 4.41: Static Partitioning: Fetch and speed

The speed of each accelerator j is calculated as the inverse of its execution time:

$$v_j = 1/t_j$$

and it will be used for the task ratios for each accelerator.

As a standard practice, a small epsilon is added to avoid division by zero, in our logic its just 1 nanosecond, so it doesn't interfere with the times, but is useful in case of data errors from the query.

Once the total system speed (V) is known (simply the sums of each accelerator speed), the ideal share of tasks for each accelerator (S_j) is calculated as:

$$S_j = \frac{v_j}{V} \cdot N_{task}$$

Since tasks cannot be split, this floating-point share must be converted into an integer.

```
1  /* .... */
2
3  /* exact task per acc (double) */
4  double shares[bts_len];
5  int total_assigned = 0;
6
7  for (int j=0; j<bts_len; j++) {
8      double ratio = acc_speed[j] / total_speed;
9      shares[j] = ratio * (double)c;
10     n_acc_task[j] = (int)shares[j]; /* integer part */
11     total_assigned += n_acc_task[j];
12 }
13
14 int missing_c = c - total_assigned;
15
16 /* assign the missing task to the accelerator
17  * mitigate slowness between accelerators */
18 while (missing_c > 0) {
19     int best_j = -1;
20     double max_decimal = -1.0;
21
```

```
22      /* find the accelerator with the highest decimal part */
23      for (int j=0; j<bts_len; j++) {
24          double decimal_part = shares[j] - (double)n_acc_task[j];
25          if (decimal_part > max_decimal) {
26              max_decimal = decimal_part;
27              best_j = j;
28          }
29      }
30
31      /* assign one more task to the winner */
32      if (best_j != -1) {
33          n_acc_task[best_j]++;
34          /* set to -1 so it won't be picked again */
35          shares[best_j] = -1.0;
36          missing_c--;
37      } else {
38          /* fallback */
39          n_acc_task[0]++;
40          missing_c--;
41      }
42  }
43
44  prof(profiler.stop_logic());
45
46  /* .... */
```

Listing 4.42: Static Partitioning: Distribution Logic

The integer conversion inevitably leads to a truncation error (e.g., 3.7 tasks becomes 3), leaving some tasks unassigned, inside `missing_c`. To solve this problem and ensure complete fairness between tasks, the scheduler implements the **Largest Remainder Method** for the decimal part of the array `shares`.

Let's consider a simple example, a batch of 10 tasks distributed between two accelerators A and B, where the logic assigns: 4.7 tasks to A and 5.3 tasks to B. If we simply truncate integers A gets 4 tasks and B gets 5 tasks, so the total assigned will be 9 and we are missing 1 task.

To decide who receives this unassigned task, the scheduler looks at the decimal remainders. A lost 0.7 and B lost 0.3. Since $0.7 > 0.3$, the accelerator A has the "highest claim" to the extra task. Therefore, the final distribution becomes 5 for A and 5 for B, correctly summing to 10. This method ensures that the final integer distribution remains as close as possible to the ideal proportion calculated before.

Finally, the calculated number of tasks in `n_acc_task[j]` is dispatched to the corresponding worker buffers.

```
1       /* .... */
2       PROF(profiler.start_dispatch());
3
4       /* send task to the accelerators */
5       t = 0;
6       for (int j=0; j<bts_len; j++) {
7           for (int w=0; w<n_acc_task[j]; w++){
```

```
            if (t >= c) break;

            buffers[bts[j]]->put(tasks[t]);
            t++;
        }
    }

    c = 0;
    PROF(profiler.stop_dispatch());
    PROF(profiler.record_sample());
}

PROF(profiler.stop_power_monitor());
break;
```

Listing 4.43: Static Partitioning: Dispatch

4.3.10 Large Matrix Split

This static strategy addresses a specific case, handling a single matrix multiplication that is too large to fit into the memory of any individual accelerator. Instead of treating the matrix as an indivisible unit, this logic implements a Split-Compile-Merge approach, splitting the large matrix horizontally in smaller chunks and sending smaller tasks to each accelerator, enabling the sistem to process this workload in parallel, then the result are written directly into the corresponding section.

To determine the optimal number of rows for each device, the scheduler employs an **Iterative Refinement algorithm**. This logic is also splitted in 3 phase, the initial estimation, the iterative refinement and then the dispatchign.

```
case Logic::LARGE_MATRIX_SPLIT:
PROF(profiler.start_power_monitor());

while (threads_keep_running) {
    PROF(profiler.start_fetch());

    single_task = buffer->get();
    if (!single_task) {continue;}

    PROF(profiler.stop_fetch());
    PROF(profiler.start_logic());

    double curr_speeds[bts_len];
    int rows[bts_len];
    int rest = single_task->M % bts_len;
    int base = single_task->M / bts_len;

    /* equal row initially */
    for (int j=0; j<bts_len; j++)
        rows[j] = base + (j < rest ? 1 : 0);
    /* .... */
```

Listing 4.44: Split Logic: Initialization

The process begins by retrieving the large task and distributing the rows as evenly as possible among all available accelerators. This serves as a starting point for the refinement loop.

The core of the logic is a feedback loop that runs for a fixed number of iterations (decided with `SPLIT_MATRIX_ITERATION`). In each iteration, the scheduler queries the PerformanceMap to estimate how fast each accelerator can process its currently assigned chunk.

```
/* .... */
/* loop for adjusting the ratios */
for (int i = 0; i < SPLIT_MATRIX_ITERATION; i++) {
    double total_speed = 0.0;

    /* current speed for 1 row */
    for (int j=0; j<bts_len; j++) {
        int r = (rows[j] > 0) ? rows[j] : 1;

        double ms = 0.0;

        ms = bts_map[bts[j]]->query(r,
                    single_task->N, single_task->K, single_task->type,
    false);

        /* update time if r exceed max size */
        if (r > MAX_SIZE){
            double max_ms = bts_map[bts[j]]->query(MAX_SIZE,
                single_task->N, single_task->K, single_task->type, false);
            ms = max_ms * ((double)r / (double)MAX_SIZE);
        }

        /* single row speed, number of rows / ms */
        double speed = (double)r / (ms + 1e-9);
        curr_speeds[j] = speed;
        total_speed += speed;
    }

    /* adjust the row with the ratios */
    int rows_distributed = 0;
    for (int j=0; j<bts_len; j++) {
        double ratio = curr_speeds[j] / total_speed;

        /* new rows from the ratio */
        int target_rows = (int)(ratio * single_task->M);
        rows[j] = target_rows;
        rows_distributed += rows[j];
    }
}
PROF(profiler.stop_logic());
/* .... */
```

Listing 4.45: Split Logic: Refinement Loop

The metric used is rows per millisecond (rows / ms):

- The scheduler queries the map for the time required to compute the currently

assigned number of rows.

- If the number of rows exceeds the maximum size present in the performanceMap, the time is estimated via $T_{new} = T_{max} \cdot \frac{row}{maxrow}$
- The throughput for each device is calculated, and the total rows M are redistributed proportionally to these new speeds.

This loop converge towards a balanced distribution where all accelerators finish at approximately the same time, and once the optimal row counts inside `rows[j]` are calculated, the actual tasks must be created and dispatched. This step does not copy any data, instead, it uses pointer arithmetic to create the smaller tasks.

```
/* .... */
    size_t elem_size = (single_task->type == Type::FLOAT) ? 4 : 2;
    size_t offset_rows_acc = 0;

    /* dispatch tasks */
    for (int j=0; j<bts_len; j++) {
        int h = rows[j];
        if (h <= 0) continue;

        /* divide rows into tasks if exceed MAX_SIZE */
        while (h > 0) {
            int h_limit = h;

            if (h_limit > MAX_SIZE) h_limit = MAX_SIZE;

            task* sub_task = new ::task;
            *sub_task = *single_task;

            /* save the pointer for later freeup */
            sub_task_array.push_back(sub_task);

            /* A row */
            sub_task->M = h_limit;

            /* A offset */
            size_t offset_A = offset_rows_acc * single_task->K * elem_size;
            sub_task->A = (char*)single_task->A + offset_A;

            /* B offset still all in memory */
            sub_task->B = single_task->B;

            /* C offset */
            size_t offset_C = offset_rows_acc * single_task->N * elem_size;
            sub_task->C = (char*)single_task->C + offset_C;

            /* add task to the pending */
            pending_tasks++;

            /* submit to the acc */
            buffers[bts[j]]->put(sub_task);

            /* for each accelerator we add more offset */
            offset_rows_acc += h_limit;

```

```
45                 h -= h_limit;
46             }
47         }
48
49         pending_tasks--;
50     }
```

Listing 4.46: Split Logic: Sub Task dispatch

For each accelerator, the logic (in Listing 4.46) creates a new task structure:

1. **A**: The pointer is shifted forward to skip the rows processed by previous accelerators.
2. **B**: The pointer remains unchanged, as B is needed in its entirety for every row of A.
3. **C**: The pointer is shifted similarly to A, ensuring that the accelerator writes its result into the correct slice of the output buffer.

Furthermore, if the assigned chunk of rows is still too large for the device memory, larger than `MAX_SIZE`, the chunk is further subdivided into smaller, sequential tasks for that specific worker.

4.3.11 Static Hetero Partitioning

This strategy is an evolution of the Static Partitioning logic, designed for heterogeneous workloads where tasks differ in size and computational cost (stream of matrices with random dimensions).

In this scenario, calculating a simple ratio based on the first task (as done in the previous strategy) would be incorrect, because the speed of an accelerator is not constant but depends on the problem size. Therefore, instead of assigning tasks based on a global ratio, this algorithm adopts a **greedy approach**: it simulates the execution timeline and assigns each task to the accelerator that can complete it soonest, considering its current accumulated load.

```
/* ... fetch logic same as before ... */

/* iterator for each bt */
int i_bt[bts_len];

for (int i=0; i<bts_len; i++)
    i_bt[i] = 0;

task* dispatch_tasks[bts_len][BATCH_SIZE_HETERO];

/* for every task */
for (int i = 0; i < c; i++) {
    int best_bt = -1;
    double min_ms = 1e15;

    /* for every bt */
    for (int j = 0; j < bts_len; j++) {
        double query_ms = bts_map[bts[j]]->query(tasks_hetero[i]->M,
                tasks_hetero[i]->N, tasks_hetero[i]->K,
                tasks_hetero[i]->type, false);

        double finish_ms = bt_ms[j] + query_ms;

        /* we assign the task to the least "filled" */
        if (finish_ms < min_ms) {
            min_ms = finish_ms;
            best_bt = j;
        }
    }

    /* openvino and sycl jit updates */
    if (bts[best_bt] == BT::OPENVINO || bts[best_bt] == BT::SYCL)
        bts_map[bts[best_bt]]->query(tasks_hetero[i]->M,
            tasks_hetero[i]->N, tasks_hetero[i]->K,
            tasks_hetero[i]->type, true);

    dispatch_tasks[best_bt][i_bt[best_bt]] = tasks_hetero[i];
    i_bt[best_bt]++;
    bt_ms[best_bt] = min_ms;
}

/* .... */
```

Listing 4.47: Static Hetero Logic

The core of the logic on the Listing. 4.47 relies on the `bt_ms` array, wich tracks the load assigned to each accelerator so far (in the current batch). The load stand for the accumulated ms expected for that particular accellerator.

The logic iterates through the batch of tasks one by one. For each task, it queries the PerformanceMap to predict how long it will take on every available accelerator, then it compute the finish time expected with the load plus the task expences, than it update the best `bt`. Finally the task then is assigned to the best accelerator that has the smallest finish time, and the current load of that accellerator is updated.

Additionally, once a decision is made, the query function is called again with `add_jit = true` for the chosen accelerator, ensures that the JIT cache is updated.

After the simulation loop processes the entire batch, the tasks are pushed to the worker buffers same as before.

4.3.12 Dynamic

The Dynamic strategy was the final logic implemented in the scheduler. Unlike the static approaches which rely on performance map and pre calculated distributions, this strategy implements a simple first come first served method. Since the scheduler's architecture was originally designed around batch processing and static partitioning, this logic was adapted to fit the existing structure by introducing a polling mechanism to monitor worker status (already seen in the worker section, see Sec. 4.3.2).

The Coordinator effectively polls the state of the worker threads to find one that is currently idle.

```
case Logic::DYNAMIC:
i = 0;
while (threads_keep_running) {

    if (dispached) {
        single_task = buffer->get();
        if (!single_task) {continue;}
        dispached = false;
    }

    if (*busy[bts[i]] == false) {
        *busy[bts[i]] = true;
        dispached = true;
        buffers[bts[i]]->put(single_task);
    }

    i++;
    i = i % bts_len;
}
```

Listing 4.48: Static Hetero Logic

The Coordinator fetches a new task from the input buffer only if the dispatched flag is true, ensures that the previous task was dispatched to some idle accelerator. The loop iterates through the list of accelerators `bts` and checks the busy atomic flag of the current worker. If `*busy[bts[i]]` is false (worker is idle), the task is immediately pushed to that worker's buffer. The busy flag is then set to true, and dispatched is set to true to trigger the fetch of the next task. If the current accelerator is busy, the loop increments i to check the next one in the list, ensuring that all workers are polled fairly in a round robin manner.

This approach minimizes the scheduler's complexity but introduces a busy wait cycle on the Coordinator as it constantly checks for free workers.

As shown later on in the results chapter, this method performs efficiently in homogeneous workloads, However, due to its greedy nature and lack of knowledge about task complexity or hardware capabilities, it tends to underperform in highly heterogeneous environments, where it fails to optimize load balancing effectively.

Chapter 5

Results

This chapter, discusses the performance of the AMM Scheduler. The objective of the discussed tests is the validation of the effectiveness of the proposed scheduling logics (Static, Static Hetero, and Dynamic) and to analyze how the workload is distributed among the available accelerators in a real-world heterogeneous environment.

All the experiments were conducted as already described in the Tool chapter (see Chap. 2), on a modern high performance laptop, specifically the Lenovo Yoga Pro 9.

The specific hardware and software used are listed below:

- CPU the Intel Core Ultra 9 185H.
- Intel Arc Graphics as the integrated GPU.
- Intel AI Boost NPU.
- NVIDIA GeForce RTX 4060 Laptop (8GB VRAM).
- RAM 32 GB DDR5.
- OS Kubuntu 25.10 with the linux kernel 6.17.

The results presented in the following sections were obtained directly from the execution of the code (see Appendix A.1.2, `main.cpp`), utilizing the instrumentation described in the Profiler implementation (see Sec. 4.3.7).

Upon completion of a batch of tasks, the scheduler automatically exports all relevant metrics (including execution time, latency, energy consumption, and worker statistics) into structured .csv files. These files constitute the source dataset used to generate the graphs and observe the results discussed in this chapter.

To illustrate the data collection process, Listing 5.1 reports an example of the raw terminal output generated by the scheduler after executing a batch of 4096×4096 matrix multiplication tasks.

```
Matrix: 4096x4096x4096 | Tasks: 110
Total Time: 8386.45 ms
Latency (ms): Avg 3854.22 | Min 60.89 | Max 8386.44

Power consumption:
Duration: 8.39 s
cpu package:   484.22 J   (57.74 W avg)
cpu core:      311.32 J   (37.12 W avg)
cuda gpu:     1073.96 J  (128.06 W avg)
total:        1558.18 J

Scheduler: 110 batches
Fetch      | avg: 0.30   us | min: 0.02   us | max: 1.95   us
Logic      | avg: 0.41   us | min: 0.09   us | max: 1.39   us
Dispatch   | avg: 0.23   us | min: 0.03   us | max: 0.57   us
Total avg overhead: 0.94 us  Total overhead: 103.09 us

Workers:
[CUDA]
tasks: 26
occupancy:          98.98 %
total idle time: 70.47 ms,  Avg idle time:   2710.28 us
total work time: 6811.38 ms,  Avg work time:   261976.09 us
total time:         6881.85 ms
[SYCL]
tasks: 55
occupancy:          99.66 %
total idle time: 23.17 ms,  Avg idle time:   421.24 us
total work time: 6755.88 ms,  Avg work time:   122834.20 us
total time:         6779.05 ms
[OPENVINO]
tasks: 24
occupancy:          99.73 %
total idle time: 18.59 ms,  Avg idle time:   774.47 us
total work time: 6836.67 ms,  Avg work time:   284861.12 us
total time:         6855.25 ms
[OPENBLAS]
tasks: 5
occupancy:          99.83 %
total idle time: 14.04 ms,  Avg idle time:   2808.41 us
total work time: 8382.58 ms,  Avg work time:   1676516.11 us
total time:         8396.62 ms
```

Listing 5.1: Example of Scheduler Execution Output

5.1 Homogeneous Workloads

The first set of tests are on homogeneous workloads, where the scheduler process batch of identical matrix multiplication tasks, we analyzed three different matrix sizes: 1024x1024, 2048x2048, and 4096x4096, all the data used are with 32 bit precision.

In this case, we compare the Static Partitioning logic against the Dynamic logic and for the baseline the CUDA only logic will be used.

5.1.1 Matrix: 1024x1024

In this case, we are dealing with small matrices, where the biggest overhead is certainly the latency of moving data between memories. The results of the logic execution are presented below:

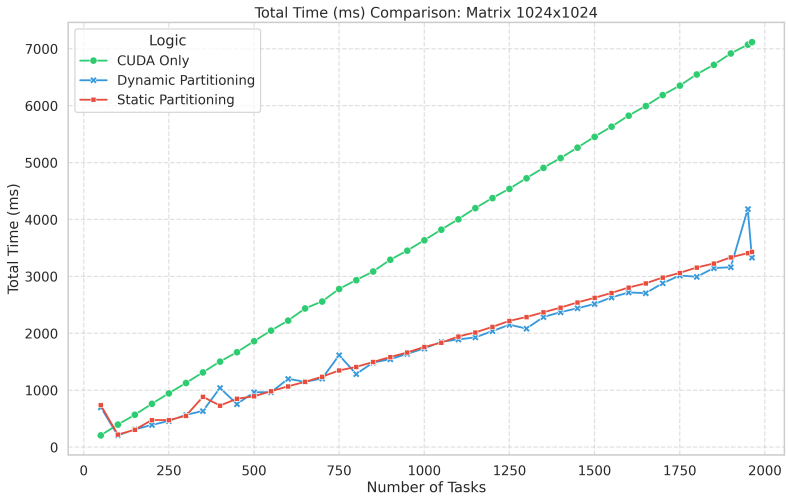

Figure 5.1: Time Comparison 1024

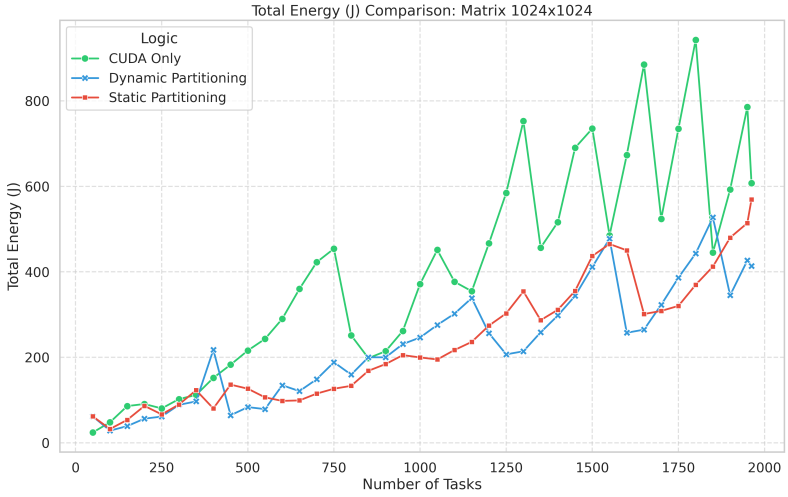

Figure 5.2: Energy Comparison 1024

As we can see from Figure 5.1, with these small tasks, the static logic and the dynamic logic perform almost in the same way, both clearly outperform the single CUDA card by using multiple accelerators in parallel. An average speedup of 1.9% was observed for the static logic, and 2% for the dynamic logic, compared to the CUDA logic. The graph in Figure 5.2 is very interesting, where we can clearly see how the two strategy manage to stay below the total consumption of the CUDA

logic (for energy metrics, see Sec. 4.3.7). This is a very significant result because it shows how these different accelerators, even if they are not as fast as the CUDA card, can still be efficient; and if used in parallel, they lead to an increase in the total system efficiency. The metrics shows an average of 231.8 J for the dynamic logic and 236.2 J for the static, both outperforms the 405.5 J for the CUDA only. In this case, the metric efficiency was calculated as Tasks per Joule, and we obtained:

- Cuda only: 2.62 Tasks/J
- Dynamic: 4.46 Tasks/J
- Static: 4.37 Tasks/J

Furthermore, to investigate the performance of the two logics, graphs were made to see how the tasks are distributed among the various accelerators, and how much time the accelerators actually work during the whole makespan.

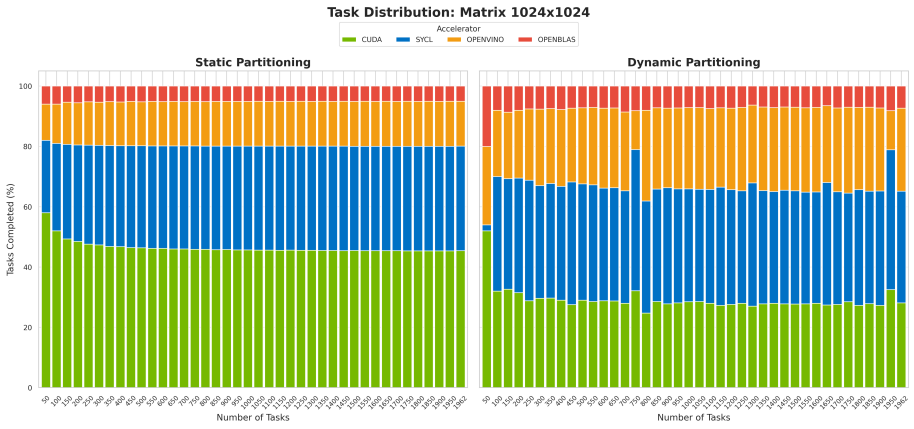

Figure 5.3: Task Distribution 1024

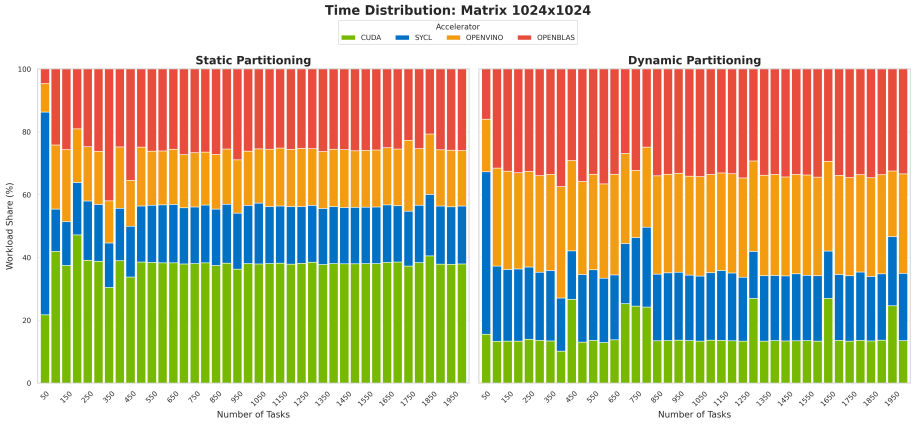

Figure 5.4: Time Distribution 1024

In Fig. 5.3 and Fig. 5.4 we can see how the two logics fundamentally differ in their operation. The greedy nature of the dynamic logic tends to make the accelerators work more continuously in this case of small tasks, managing to "spread" the work better among the accelerators. As we can note from Figure 5.3, it is clear that the static logic underestimated the performance of the non CUDA accelerators, and this led to a higher number of tasks on NVIDIA card, meanwhile, the dynamic logic managed, thanks to its greedy nature, to make the other accelerators work much more evenly, and this indeed led to a bit more faster times. In fact, as we can see from the graph in Figure 5.4, the dynamic logic definitly make more use of the other accelerators over the CUDA card. This is due to the fact that they are slightly faster in this smaller tasks, having much less memory overhead.

5.1.2 Matrix: 2048x2048

In this scenario, the workload increases significantly. The matrix size is large enough that the computational cost starts to increase over the data transfer overhead. The results of the logic execution are presented below:

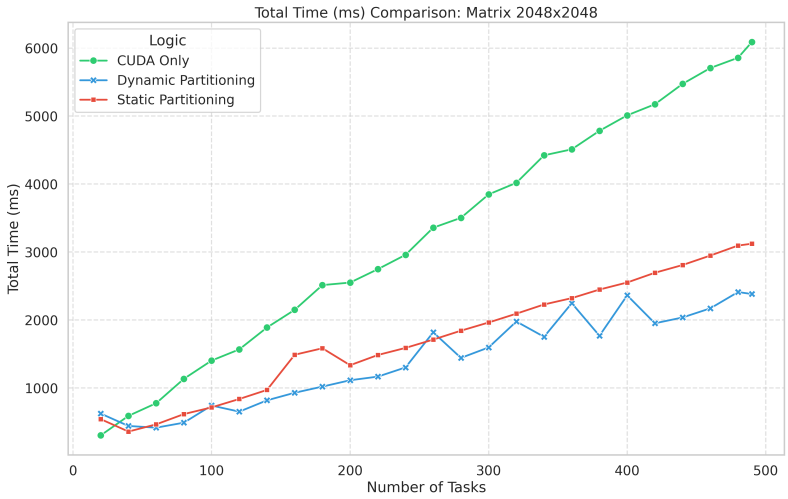

Figure 5.5: Time Comparison 2048

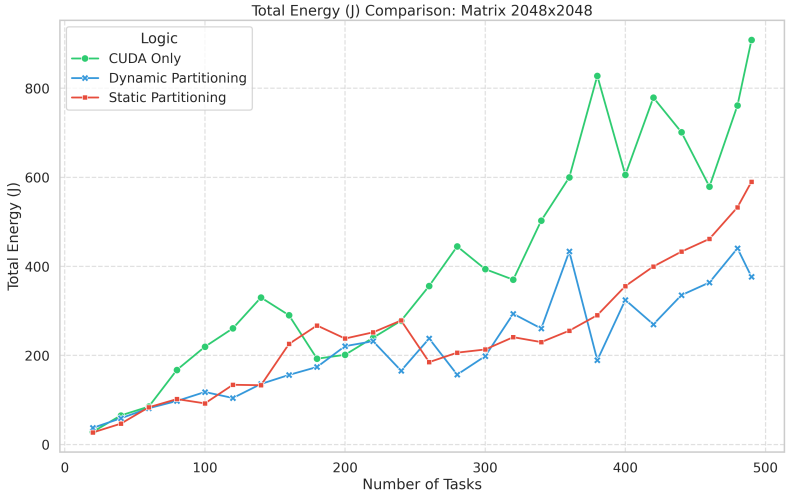

Figure 5.6: Energy Comparison Matrix 2048

As shown in Figure 5.5, both strategies continue to outperform the single CUDA baseline, validating the benefit of parallel execution even in this kind of workload. However, a clear different appears between the two logics. The dynamic strategy achieves an average speedup of 2.21x (peaking at 2.71x), while the static strategy lags slightly behind with an average speedup of 1.82x. This indicates that the dynamic logic is 20% faster on average for this specific workload.

Regarding energy efficiency (Figure 5.6) the heterogeneous execution significantly reduces the total energy consumption compared to the CUDA only approach, thanks to the reduction in total execution time. The dynamic strategy proves to be the most efficient, consuming only 218 J on average, compared to 407 J for CUDA Only. The static logic consume on average 251.01 J.

The efficiency metric show this gain:

- **CUDA Only**: 0.66 Tasks/J
- **Dynamic**: 1.14 Tasks/J
- **Static**: 1.00 Tasks/J

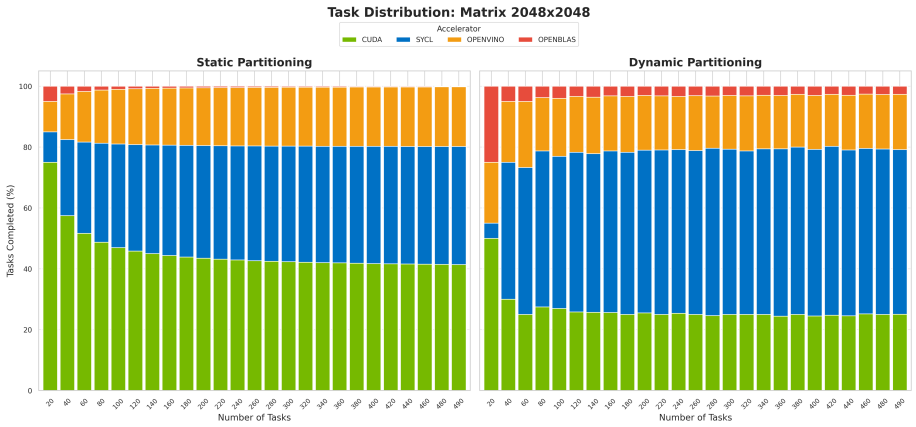

Figure 5.7: Task Distribution: Matrix 2048x2048

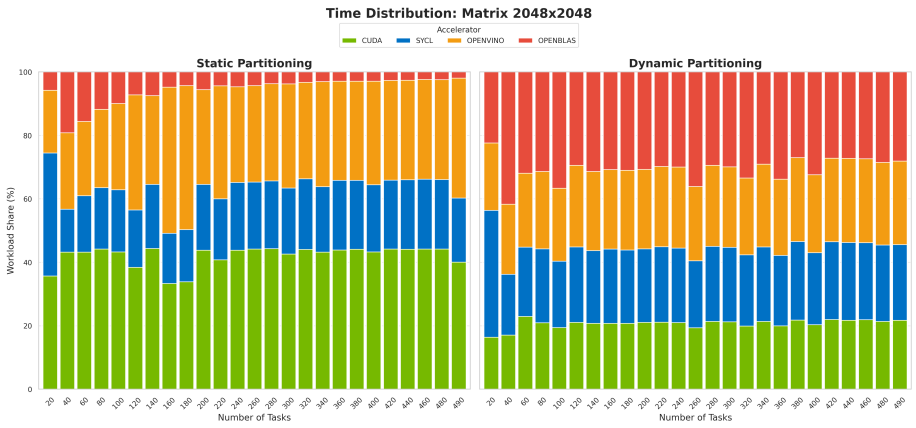

Figure 5.8: Time Distribution: Matrix 2048x2048

Figure 5.7 reveals again the cause of the performance difference. The static strategy assigns a large portion of tasks to the CUDA GPU and a smaller share to the other accelerators. While safe, this distribution appears to underutilize the other accelerators.

However, the dynamic strategy as already said, adapts in real time, so as seen in the Figure 5.8, the dynamic logic keeps the CPU, the intel GPU and NPU work more evenly.

5.1.3 Matrix: 4096x4096

This scenario expose a heavy workload on the scheduler, the computational cost now should dominate the ovehread of data transfer. The results of the logic execution are presented in Fig. 5.9 and Fig. 5.10:

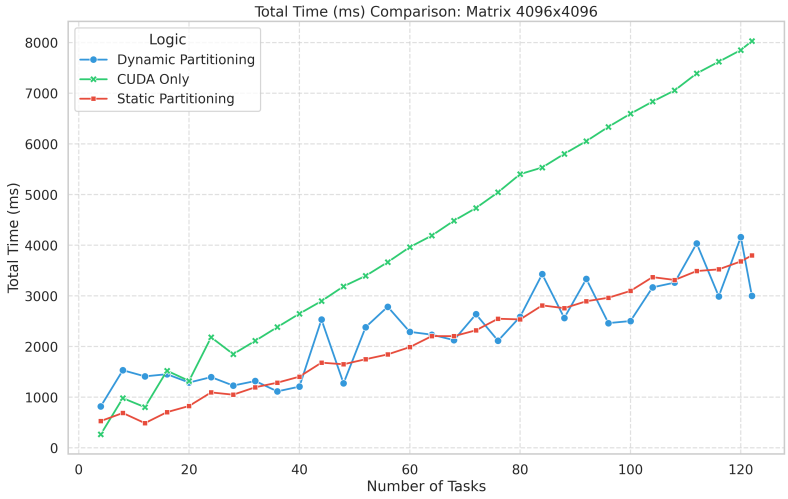

Figure 5.9: Time Comparison 4096

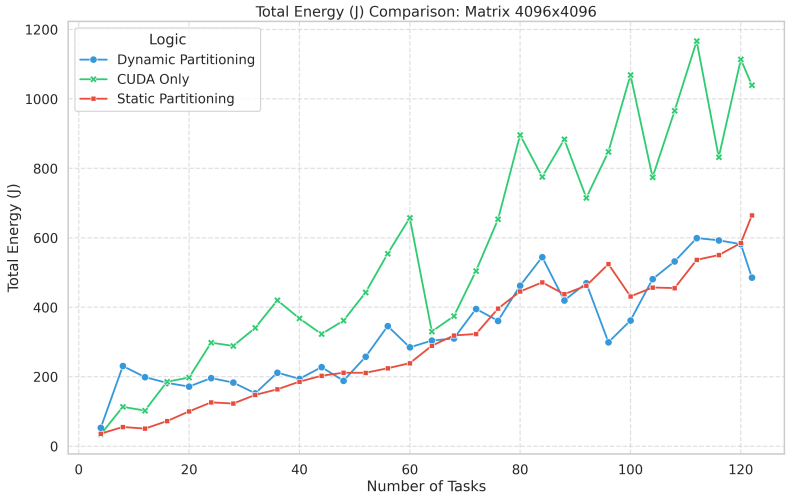

Figure 5.10: Energy Comparison 4096

As shown in Figure 5.9 both strategies still significantly outperform the single CUDA baseline. However, the static strategy show its superiority in this specific scenario,

achieving an average speedup of 1.91x (peaking max at 2.16x) and the dynamic strategy follows with an average speedup of 1.78x (peaking at 2.67x). Ultimately, the static logic proves to be 10% faster on average compared to the Dynamic one.

Regarding efficiency (Figure 5.10), the results still shows that despite activating all available accelerators, the parallel execution reduces the total energy consumed, the CUDA-only execution consumes on average 568.70 J, while the heterogeneous strategies drop this to roughly 330-300 J.

- **CUDA Only**: 0.11 Tasks/J
- **Dynamic**: 0.18 Tasks/J
- **Static**: 0.21 Tasks/J

This represents a 40% efficiency gain for the static strategy over the baseline: using the CPU and NPU alongside the GPU for heavy tasks actually saves energy because the entire system finishes the job much faster.

To understand why the Static logic outperforms the Dynamic one here, we analyze the task distribution:

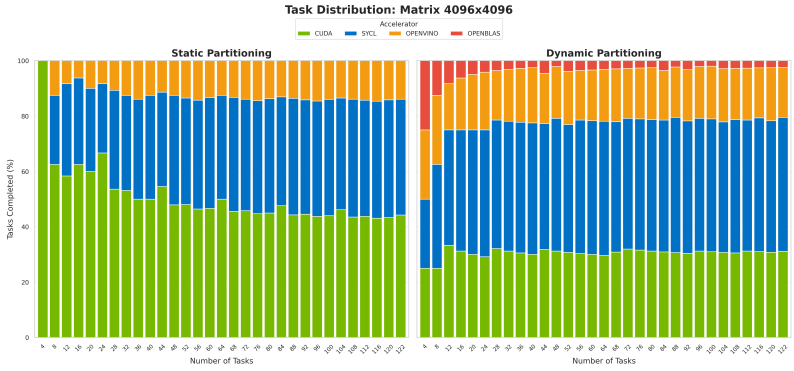

Figure 5.11: Task Distribution 4096

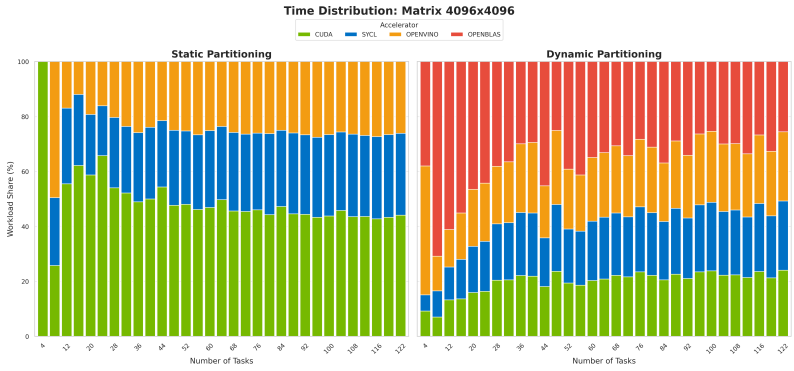

Figure 5.12: Time Distribution 4096

Figure 5.11 shows that the static strategy assigns the vast majority of tasks to the powerful CUDA GPU and SYCL, minimizing the load on slower devices, however, the dynamic logic, due to its greedy nature, assigns a slightly larger portion of tasks to the CPU and OpenVINO (as seen in Figure 5.12). For very large matrices, the time difference between the GPU, NPU and mostly CPU is massive, so if the dynamic logic assigns the last few tasks to a slow device while the GPU finishes early, the entire makespan is delayed waiting for that slow device to complete. The static strategy with all the logics and heuristcs avoids this by predicting the velocity of the accelerators and restricting the slow devices to a more smaller amount of work, achieving the highest speedup. This clearly shows how the static approach have his benefit in a lot of workload in this heterogeneous scenario. It is worth noting that the dynamic strategy can occasionally outperform the static logic during specific "lucky runs". If the greedy assignment happens to align perfectly by chance, it can squeeze slightly higher utilization out of the accelerators. However, as the overall results shows, the latency issues outweigh these occasional marginal gains in average.

5.1.4 Batch Comparison

We now present the following graphs to illustrate the impact of batch size on the execution times of the static logic:

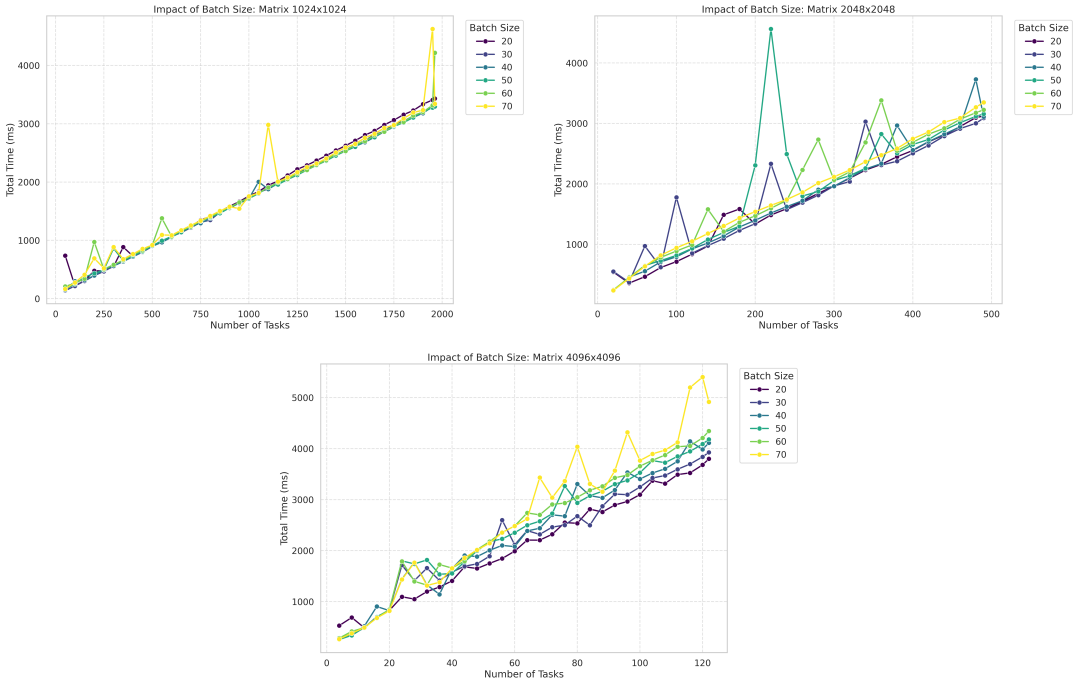

Figure 5.13: Batch Size Analysis

As is evident from the graphs in Fig 5.13, a relatively small batch size specifically ranging from 20 to 30 proved to be the most suitable for the selected algorithm and the dataset utilized across each workload. Conversely, it is evident that larger batch sizes frequently result in inaccurate predictions, while inevitably introducing additional overhead to the coordinator adding more time beetween the dispatch of the tasks to the workers.

5.2 Heterogeneous Workloads

5.2.1 Static Heterogeneous

This test involves streams of matrices with random dimensions (from 512 to 4096). This represents the most realistic scenario for a general-purpose system, where tasks of varying complexity arrive at the scheduler.

The results of the execution are presented in Fig. 5.14 and Fig. 5.15:

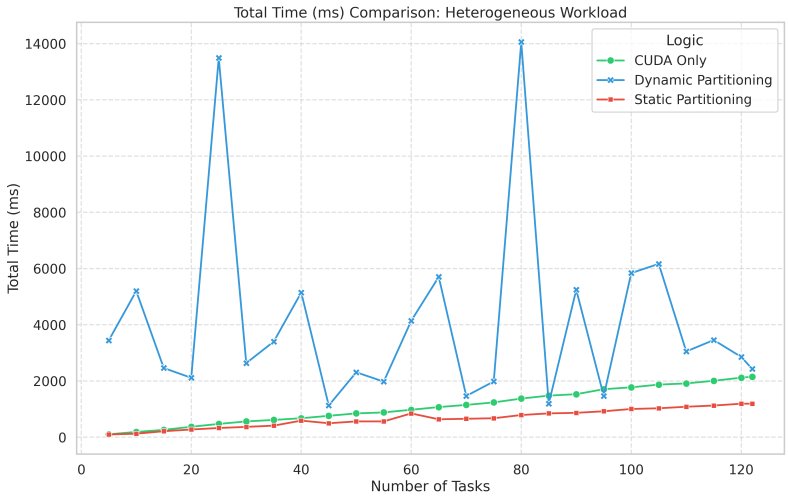

Figure 5.14: Time Comparison: Heterogeneous Stream

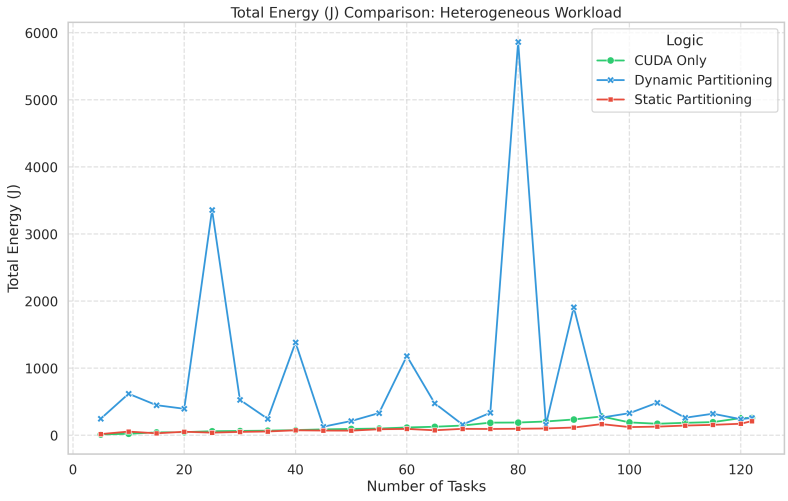

Figure 5.15: Energy Comparison: Heterogeneous Stream

As shown in Fig. 5.14, the difference between the strategies in this case is drastic. The static hetero strategy as the red line proves to be extremely robust, consistently outperforming the single CUDA baseline as the green line. It achieves an average

speedup of 1.59x (peaking at 1.85x), demonstrating how this logic achieves a consistent speedup increase while orchestrating the diverse accelerators to handle the heterogeneous workload.

In contrast, the dynamic strategy as the blue line fails to maintain consistent execution times this type of workload. With an average speedup of only 0.42x (75% slower than the static), it is significantly slower than using the discrete GPU alone. This failure is due to its greedy nature: without looking ahead, it treats a 512×512 matrix and a 4096×4096 matrix as identical tasks, and consistently fails to distribute them efficiently to achieve maximum performance.

The energy analysis in Fig. 5.15 mirrors the performance gap already observed:

- **Static Strategy**: Consumes on average 94 J, achieving a -30% energy saving compared to the average CUDA baseline (136 J).
- **Dynamic Strategy**: Due to the extremely long execution times, the energy consumption skyrockets to an average of 805 J, with a +490% increase over the baseline.

To better understand these results, we analyze the distribution graphs in Fig. 5.16 and Fig. 5.17:

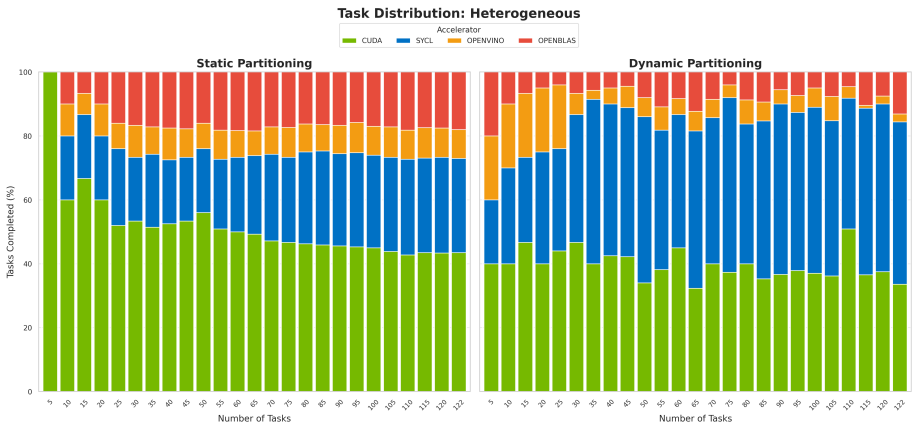

Figure 5.16: Task Distribution: Heterogeneous Stream

Regarding the task distribution (see Fig. 5.16), the graph doesn't describe the full scenario since every task has a different weight. However, for the static strategy, we can observe that the number of tasks assigned to each accelerator remains proportional as the total number of tasks increases. This confirms that the strategy is consistent in its decision making logic.

In contrast, the dynamic strategy shows a completely chaotic behavior. Since tasks are assigned to whichever device is free, the distribution varies unpredictably. Unlike the homogeneous tests where the dynamic distribution was relatively stable with

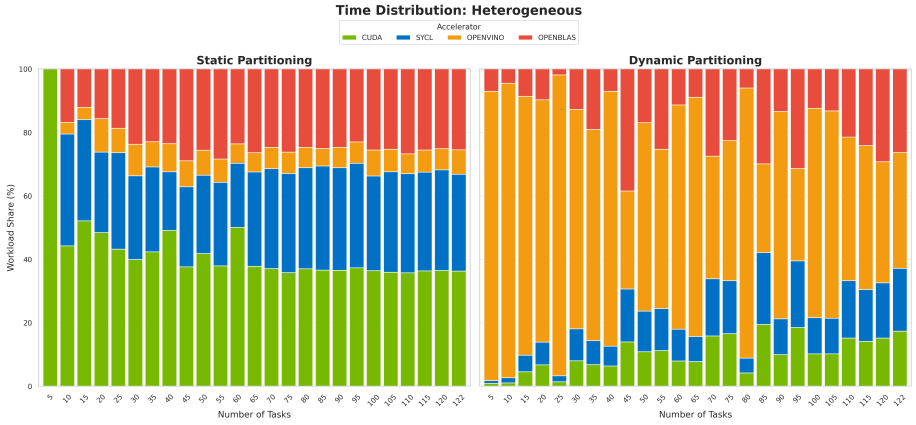

Figure 5.17: Time Distribution: Heterogeneous Stream

smaller tasks, here the randomness of task sizes combined with greedy scheduling leads to a random assignment that changes with every run.

As seen in the Time Distribution (Figure 5.17), the CPU and OpenVINO bars are massive. This indicates that the scheduler blindly assigned computationally heavy tasks (e.g., a 4096×4096 matrix) to slow devices simply because they were "free"" at that moment. This creates a massive bottleneck where the powerful GPU finishes its work quickly and sits idle for seconds, waiting for the CPU to finish a single large matrix.

This result conclusively proves that for heterogeneous streams of tasks, smart scheduling with performance prediction is mandatory in this kind of workloads. A simple greedy approach is insufficient and leads to severe performance degradation.

5.2.2 Batch Comparison

We now present the following graph in Fig. 5.18 to illustrate the impact of batch size on the execution times of the static logic under heterogeneous workload conditions:

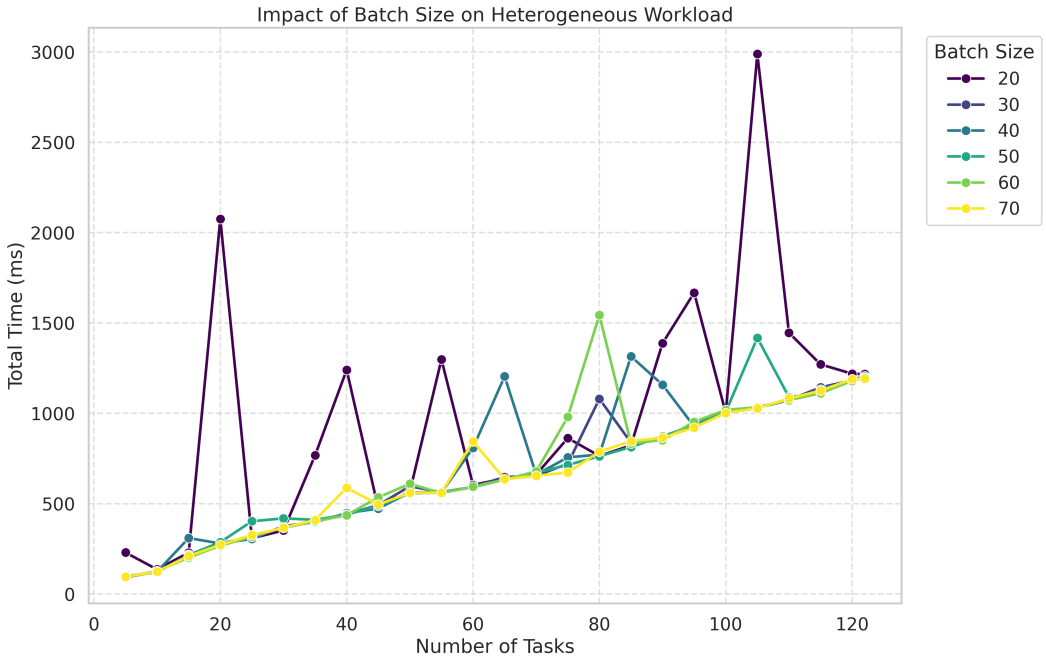

Figure 5.18: Heterogeneous Batch Size

As is evident from the graph in Fig 5.18, a relatively large batch size, specifically ranging from 60 to 70, proved to be the most suitable for the static hetero algorithm. Conversely, it is evident that smaller batch sizes (e.g. 20) frequently result in highly unstable execution times, introducing load imbalance due to the high variance of task duration. Larger batches effectively smooth out these variations, allowing the static ratios calculated within the algorithm to converge towards a more efficient distribution of the task, effectively respecting the planned times.

5.2.3 Large matrix split

This scenario involves decomposing a single very large matrix multiplication (where M grows up to 230,000 rows) into smaller sub-tasks (chunks) to fit within device memory limits and distribute the computation.

To provide a fair and meaningful baseline, a simplified version of the splitting logic was implemented. This strategy performs the same chunking operation required to handle the large matrix dimensions but dispatches all chunks exclusively to the NVIDIA GPU (the code can be seen in Appendix A.1.2, `scheduler.hpp`).

Furthermore, for this specific test case, the OpenVINO backend was excluded. Pre-

liminary tests showed that OpenVINO's performance degrades when handling extremely "rectangular" matrices (very high M, fixed N,K), making it an unsuitable for this specific splitting workload. Our strategy therefore in this test leverages CUDA, SYCL, and OpenBLAS.

The following graphs (Fig. 5.19) illustrate the comparison between the Hybrid Split strategy and the CUDA Only baseline:

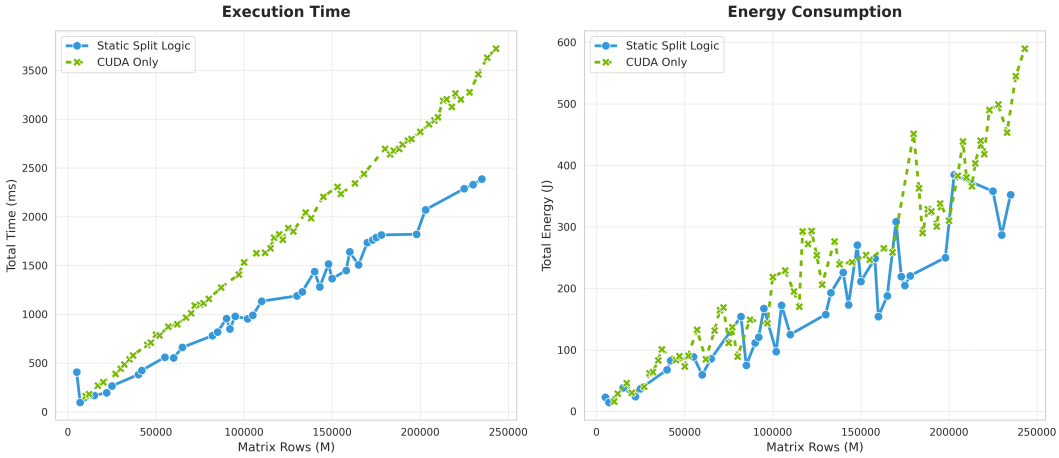

Figure 5.19: Time and Task Analysis

The execution time comparison on Fig. 5.19 on the left clearly demonstrates the advantage of the Hybrid split approach. The split logic consistently remains below the green line (CUDA Only), indicating a lower total time to completion across the entire range of matrix sizes. The hybrid strategy achieves an 1.49x average speedup compared to using the GPU alone. In specific configurations the speedup reaches a peak of 2.46x, demonstrating that the combined throughput of the CPU and iGPU can more than double the system's performance for certain problem sizes.

Consumptions wise, the Hybrid strategy also demonstrated superior energy efficiency in some cases (see Fig. 5.19 on the right): on average, the large static split approach reduced energy consumption by 21% compared to the CUDA baseline. Furthermore over the entire benchmark, the hybrid strategy consumed 38 J less on average than the 52 J required by the CUDA only execution. This result highlights that finishing the task faster, even by powering up the CPU and iGPU, is often more energy efficient than keeping the discrete GPU active for a longer duration.

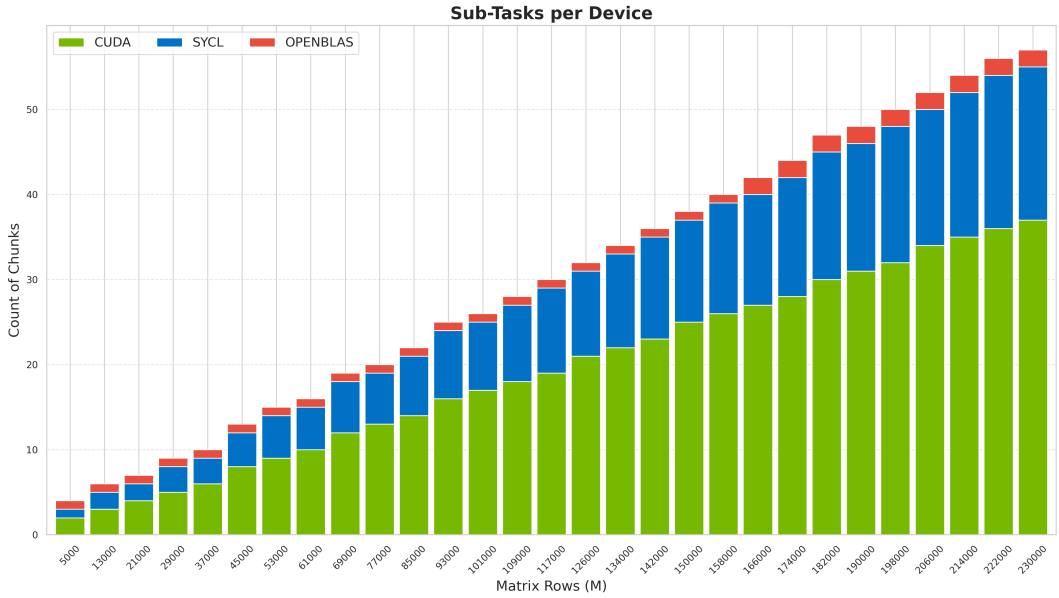

Figure 5.20: Chunk Distribution Analysis

The load balancing graph on Fig. 5.20 provides insight into how the work was distributed. As expected, the powerful CUDA GPU handles the majority of the chunks (approximately 50% or 60% of the workload). The remaining of the work is successfully offloaded to the iGPU and CPU backends. This is the direct cause of the observed speedup: by keeping all accelerators active, the system completes the massive calculation faster than the single GPU could.

5.3 Real-World Application: Video Filtering

To demonstrate the flexibility of the AMM Scheduler beyond synthetic benchmarks, a real-world video processing application was implemented. This test case involves applying an edge detection filter (convolution) to a video stream in real-time.

The application, whose source code can be found in Appendix A.1.5, uses OpenCV to decode a video file frame by frame. Each frame is converted into a matrix format suitable for the scheduler: the image is flattened, and the convolution kernel is treated as the second matrix in a multiplication operation (using the *im2col* technique). This effectively transforms a standard image processing task into a stream of matrix multiplication tasks.

For this specific test, the OpenVINO backend was disabled. Since the *im2col* transformation results in matrices that are essentially long vectors (very high N, small K), the specialized architecture of the NPU does not provide a benefit compared to the CPU or GPU. Therefore, the task distribution focuses on CUDA, SYCL, and OpenBLAS.

5.3.1 Performance Comparison

To evaluate the scheduler under a workload that is as realistic as possible, a video processing application was implemented. The specific task consisted of applying an edge-detection filter to a standard .mp4 video file.

To manage the video format and frame extraction, the OpenCV library was utilized. Typically, applying a filter to an image is performed using convolution operations. However, since the AMM Scheduler is specialized exclusively for Matrix Multiplication, the standard im2col (image-to-column) technique was employed. This technique reshapes the image data and the filter kernel, transforming the convolution problem into a matrix multiplication task that our system can execute.

Once the frames were extracted and converted, they were simply dispatched to the scheduler following the standard procedure already described in the Implementation Chapter (see Sec. 4.3). The scheduler then handled the distribution of these tasks across the available hardware automatically.

To demonstrate the results, we present the raw execution logs generated directly by the scheduler. We compared a baseline execution using only the Discrete GPU (Cuda only logic) against the Dynamic Logic of the scheduler.

In Fig. 5.2 is the output log for the CUDA Only execution:

```
Matrix: 1x917604x9 | Tasks: 241
Total Time: 2896.41 ms
Latency (ms): Avg 1452.57 | Min 29.15 | Max 2896.41

Power consumption:
Duration: 2.89 s
cpu package:     89.46 J  (30.89 W avg)
cpu core:        59.04 J  (20.38 W avg)
cuda gpu:       231.69 J  (79.99 W avg)
total:          321.16 J
```

Listing 5.2: Video Filter Output: CUDA Only

And in Fig. 5.3 is the output using the dynamic Logic:

```
Matrix: 1x917604x9 | Tasks: 241
Total Time: 592.46 ms
Latency (ms): Avg 306.42 | Min 12.55 | Max 592.46

Power consumption:
Duration: 0.59 s
cpu package:     23.17 J  (39.11 W avg)
cpu core:        17.12 J  (28.90 W avg)
cuda gpu:       101.43 J (171.20 W avg)
total:          124.60 J

Scheduler: 241 batches
Fetch      | avg: 0.20   us
```

```
Logic      | avg: 6.89   us
Dispatch   | avg: 0.19   us
Total overhead: 1755.25 us

Workers:
[CUDA]
tasks: 43
occupancy:          87.45 %
total work time: 522.00 ms,  Avg work time:   12139.50 us
[SYCL]
tasks: 40
occupancy:          96.48 %
total work time: 561.80 ms,  Avg work time:   14045.01 us
[OPENBLAS]
tasks: 158
occupancy:          87.59 %
total work time: 511.55 ms,  Avg work time:   3237.66 us
```

Listing 5.3: Video Filter Output: Dynamic

The results show a massive performance improvement with the scheduler. The total execution time dropped from 2896 ms to 592 ms, representing a speedup of roughly 4.8x. Consequently, the total energy consumed for the task was reduced from 321 J to 124 J.

This speedup is due to the specific size of the matrices produced by the im2col transformation. As shown in the logs (1×917604×9), these matrices are essentially long vectors. The CPU and the Integrated Graphics are extremely efficient, as they perform the calculation directly in system memory without the high latency overhead required to move data to the discrete GPU and launch kernels.

The dynamic logic naturally exploited this characteristic: by assigning tasks to whichever device was free, it allowed the CPU to process the majority of the frames (158 tasks) simply because it was completing them faster (3.2 ms per task) than the discrete GPU (12 ms per task).

Finally, to qualitatively validate the correctness of the computation, we present a visual comparison between the input video stream and the output generated by the scheduler. Figure 5.23 displays a single frame extracted from the test video. On the left, the original frame is shown. On the right, the result of the Edge Detection filter is visible showing how the scheduler successfully applied the kernel using the *im2col* matrix multiplication technique.

Figure 5.21: Original Input Frame

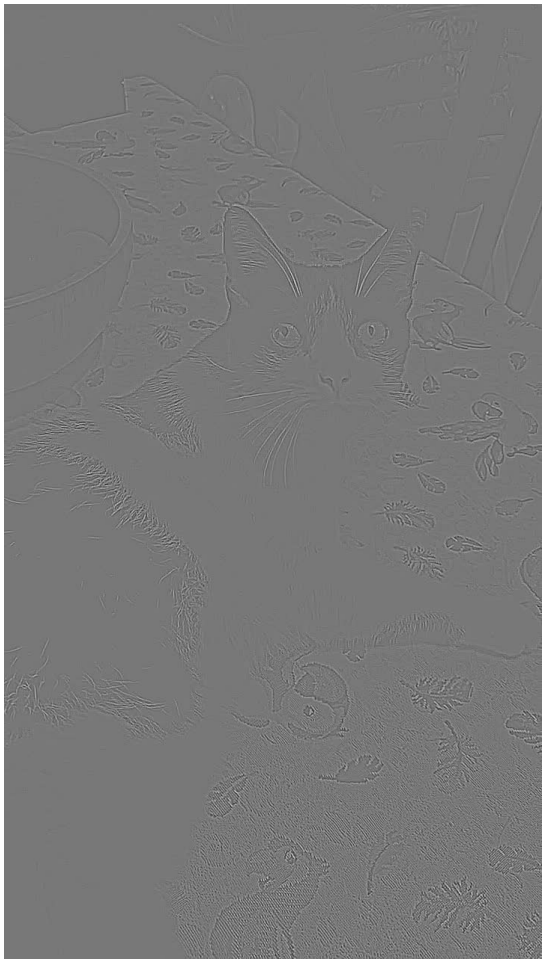

Figure 5.22: Edge Detection Output

Figure 5.23: Visual comparison of the video filter application.

Chapter 6

Conclusions

The primary objective of this thesis was to design and implement a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures. The goal was to overcome the limitations of traditional software, which typically relies on a single processor, by simultaneously exploiting the entire heterogeneous topology of a computer: the CPU, the Integrated GPU, the Discrete GPU, and the Neural Processing Unit.

To achieve this, an essential preliminary phase focused on integrating and studying different compute backends (CUDA, SYCL, OpenVINO, and OpenBLAS) to create a uniform abstraction layer over the hardware. Following this, the AMM Scheduler was developed, featuring distinct scheduling logics designed to orchestrate these asymmetric resources in real time.

The experimental results, conducted on a complex hybrid architecture, validated the effectiveness of the proposed solution. The system successfully demonstrated that coordinating multiple accelerators can lead to performance gains and increased energy efficiency.

Specifically, the comparison between the scheduling logics highlighted distinct behaviors depending on the workload:

- **Homogeneous Workloads**: For streams of identical size (1024x1024 and 2048x2048), the dynamic strategy proved to be effective. Its greedy nature allowed it to minimize idle time by keeping all accelerators busy, achieving speedups of up to 2.20 compared to the single CUDA card. However, as the matrix size increased to heavy workloads, the static strategy demonstrated superior performance (1.91x speedup vs 1.78x for dynamic). The predictive nature of the static logic prevented, in most cases, a slow device that delays the entire batch by taking on a final massive task.
- **Heterogeneous Workloads**: In scenarios with mixed matrix sizes (streams

from 512x512 to 4096x4096), the necessity of a smart, predictive scheduler became evident. The static hetero strategy successfully mapped task complexity to hardware capability, achieving 1.60x speedup. In contrast, the dynamic logic failed to handle the variance, resulting in severe performance degradation (0.42x speedup) due to inefficient task assignment, proving that a simple greedy approach is insufficient for diverse workloads.

- **Large Matrix Splitting**: The split strategy demonstrated that even for a single massive operation, decomposing the problem and offloading chunks to the CPU and iGPU can yield a 1.49x speedup and reduce total energy consumption by 21%.

In conclusion, all the initial project objectives were met. The AMM Scheduler proved that an approach to hardware that embraces the diversity of available accelerators rather than ignoring it, is a viable path to maximizing both throughput and energy efficiency in modern computing systems.

6.1 Future Work

While the current implementation offers the initial foundation for heterogeneous scheduling, several avenues for future research and optimization remain open to enhance this project capabilities:

- Multi Node: The evolution of this project could be to extend the scheduler's scope beyond a single machine. Future work could focus on adapting the coordinator to distribute workloads across a server cluster, treating remote nodes as additional accelerators. This would involve implementing network aware logic to balance computation with network latency, and implement hardware aware logic if the nodes have all different accelerators.

- Expansion of Supported Primitives and Logics: Currently the scheduler is specialized in Matrix Multiplication. Future developments could aim to extend the library of supported operations, adding other primitives such as Convolutions, FFTs, and so on. Moreover the current logic handles streams of independent tasks. A significant step would be the implementation of a dependency aware scheduler capable of managing Directed Acyclic Graphs (DAGs). This would allow the system to orchestrate entire computational pipelines (e.g. full Neural Network inference).

6.2 Personal Balance

This thesis represented my first encounter with a software project of this magnitude and complexity. It has been a long journey, characterized by numerous challenges. One of the most significant aspects was the responsibility of making critical architectural decisions early on, choices that would dictate the entire development of the project. The uncertainty of whether these foundational decisions would prove effective could only be verified upon completion. However, this pressure ultimately served as a driving force. It compelled me to approach all the challenges with a pragmatic mindset, and in the end the results validated the effort.

Above all, as a hardware enthusiast, working on this project was very exciting. I had the chance to interact directly with many different accelerators and orchestrate them, exploring and exploiting all of their libraries. This experience taught me a lot about their architectures and gave me a strong foundation in High Performance Computing.

I would like to express my sincere gratitude to my supervisor for the trust placed in me for this thesis. I deeply appreciate the freedom granted to explore this project, accompanied by his constant guidance and ability to point me in the right direction whenever necessary. Confirming what I already knew, and as this experience has fully confirmed, it has been a true pleasure and privilege to work on this Master's thesis with him.

In conclusion, I remain profoundly satisfied with this work. It has fostered my growth both personally and as a software engineer, fueling my ambition to contribute to a better (and faster!) world.

Appendix A

Appendix: Source Code

This appendix contains the source code developed for the AMM Scheduler and the preliminary benchmarks used to analyze the hardware accelerators. The code is organized by project structure.

A.1 AMM Scheduler

The core project of this thesis. It includes the scheduler logic, the task definitions, and the specific backend implementations for CUDA, SYCL, and OpenVINO.

A.1.1 Build and Scripts

```
cmake_minimum_required(VERSION 3.15)
project(amm_scheduler)

include(ExternalProject)

set(LIB_OUTPUT_DIR ${CMAKE_SOURCE_DIR}/bin)
file(MAKE_DIRECTORY ${LIB_OUTPUT_DIR})

# CUDA SYCL OPENVINO
option(USE_CUDA "CUDA ON" OFF)
option(USE_SYCL "SYCL ON" OFF)
option(USE_OPENVINO "OpenVINO ON" OFF)
option(USE_OPENBLAS "OpenBLAS ON" OFF)
option(USE_OPENCV "OpenCV ON" OFF)

set(SCHEDULER_DEPS "") # depends variable

set(OPT_FLAGS "-O2 -DNDEBUG")

# CUDA
if(USE_CUDA)
    message(STATUS "CUDA Backend ON")
    ExternalProject_Add(build_cuda
```

```
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/cuda
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            -DCMAKE_CUDA_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )

    list(APPEND SCHEDULER_DEPS build_cuda)
else()
    message(STATUS "CUDA Backend OFF")
endif()

# SYCL
if(USE_SYCL)
    message(STATUS "SYCL Backend: ON")
    ExternalProject_Add(build_sycl
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/sycl
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_COMPILER=icpx
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )
    list(APPEND SCHEDULER_DEPS build_sycl)
else()
    message(STATUS "SYCL Backend: OFF")
endif()

# OPENVINO
if(USE_OPENVINO)
    message(STATUS "OPENVINO Backend: ON")
    ExternalProject_Add(build_openvino
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/openvino
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )
    list(APPEND SCHEDULER_DEPS build_openvino)
else()
    message(STATUS "OPENVINO Backend: OFF")
endif()

# scheduler
ExternalProject_Add(build_scheduler
    SOURCE_DIR ${CMAKE_SOURCE_DIR}/scheduler
    CMAKE_ARGS
        -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
        -DCMAKE_BUILD_TYPE=Release
        -DLIBS_DIR=${LIB_OUTPUT_DIR}
        -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}

        -DCUDA_SRC_DIR=${CMAKE_SOURCE_DIR}/cuda
        -DSYCL_SRC_DIR=${CMAKE_SOURCE_DIR}/sycl
```

```
        -DOPEN_SRC_DIR=${CMAKE_SOURCE_DIR}/openvino

        -DENABLE_CUDA=${USE_CUDA} # USE_CUDA
        -DENABLE_SYCL=${USE_SYCL} # USE_SYCL
        -DENABLE_OPENVINO=${USE_OPENVINO} # USE_OPENVINO
        -DENABLE_OPENBLAS=${USE_OPENBLAS} # USE_OPENBLAS
        -DENABLE_OPENCV=${USE_OPENCV} # USE_OPENCV

        -DCMAKE_EXPORT_COMPILE_COMMANDS=ON # for clangd
    DEPENDS ${SCHEDULER_DEPS}
    #BUILD_ALWAYS 1
)

#add_custom_target(UpdateLSP
#    ALL
#    COMMAND ${CMAKE_COMMAND} -E echo "update LSP..."
#    COMMAND python3 ${CMAKE_SOURCE_DIR}/merge_clangd.py
#    COMMENT "merge compile_commands.json for clangd"
#    DEPENDS build_scheduler
#)
```

Listing A.1: Root (CMakeLists.txt)

```
source /opt/intel/oneapi/2025.3/oneapi-vars.sh

sudo chmod a+r /sys/class/powercap/intel-rapl:0/energy_uj
sudo chmod a+r /sys/class/powercap/intel-rapl:0/intel-rapl:0:0/energy_uj
sudo chmod a+r /sys/class/powercap/intel-rapl:0/intel-rapl:0:1/energy_uj
```

Listing A.2: Setup Instructions (setup.txt)

```
mkdir bin
mkdir build
rm -rf build/*
rm bin/scheduler
rm -rf bin/lib/*
cd build/

export CMAKE_BUILD_PARALLEL_LEVEL=$(nproc)
cmake -DUSE_CUDA=ON -DUSE_SYCL=ON -DUSE_OPENVINO=ON -DUSE_OPENBLAS=ON -
    DUSE_OPENCV=ON ..
make -j$(nproc)

cd ..
#python3 merge_clangd.py
./bin/scheduler
```

Listing A.3: Build Script (make_run.sh)

```
import json
import os
import glob

OUTPUT_FILE = "compile_commands.json"
```

```

def merge_json_files():
    search_path = os.path.join(os.getcwd(), "build", "**", "
    compile_commands.json")
    files = glob.glob(search_path, recursive=True)

    combined_commands = []

    print(f"[LSP] {len(files)} command file")

    for fpath in files:
        if os.path.abspath(fpath) == os.path.abspath(OUTPUT_FILE):
            continue

        try:
            with open(fpath, 'r') as f:
                data = json.load(f)
                if isinstance(data, list):
                    combined_commands.extend(data)
        except Exception as e:
            print(f"Error {fpath}: {e}")

    with open(OUTPUT_FILE, 'w') as f:
        json.dump(combined_commands, f, indent=4)

    print(f"[LSP] Create {OUTPUT_FILE} with {len(combined_commands)}
    commands ")

if __name__ == "__main__":
    merge_json_files()
```

Listing A.4: Clangd Utility (merge_clangd.py)

A.1.2 Scheduler

This section contains the main orchestration logic, including the `AMScheduler` class, the profiler, and the task definitions.

```
#include <cstddef>
#include <iostream>

#include "scheduler.hpp"
#include "tasks.hpp"
#include "config.hpp"
#include "tests.hpp"
#include "opencv_test.hpp"

using namespace std;

/************************ Test functions ****************************/

void test_hetero_logic() {
    size_t n_matrix = 80;

    task* task_array = init_hetero_tasks(n_matrix, Type::FLOAT);
    cout << "Task create \n";
    Logic l = Logic::CUDA_ONLY;
```

```
20      for (int i=0; i<3; i++) {
21
22          if (i == 1) {l = Logic::STATIC_HETERO_PARTITIONING; cout << "\
    nStatic partitioning \n";}
23          if (i == 2) {l = Logic::DYNAMIC; cout << "\nDynamic \n";}
24
25          AMScheduler scheduler = AMScheduler(l);
26
27          scheduler.do_tasks(task_array, n_matrix);
28          scheduler.wait();
29
30          /* test_compare_task(task_array, num_matrix); */
31          scheduler.print_stats(task_array,n_matrix);
32
33      }
34      clean_tasks(task_array, n_matrix);
35  }
36
37  void test_dynamic() {
38      size_t n_matrix = 500;
39      int M = M_;
40      int N = N_;
41      int K = K_;
42      Logic l = Logic::DYNAMIC;
43      Type type = Type::FLOAT;
44
45      task* task_array = init_tasks(n_matrix, M, N, K, type);
46      AMScheduler scheduler = AMScheduler(l);
47      scheduler.do_tasks(task_array, n_matrix);
48      scheduler.wait();
49      scheduler.print_stats(task_array,n_matrix);
50      clean_tasks(task_array, n_matrix);
51  }
52
53  void test_scheduler_logics() {
54      size_t n_matrix = N_MATRIX;
55      int M = M_;
56      int N = N_;
57      int K = K_;
58
59      Type type = Type::FLOAT;
60
61      task* task_array = init_tasks(n_matrix, M, N, K, type);
62      for (int w=0; w<1; w++) {
63          if (w == 0) {cout << "\n--------------------  32BIT:  \n";}
64          if (w == 1) {cout << "\n--------------------  16BIT:  \n"; type =
    Type::HALF;}
65
66          /* cuda test without passing from scheduler */
67          //test_cuda_streaming(M, N, K, n_matrix, type);
68
69          Logic l;
70          for (int i=0; i<3; i++) {
71              if (i == 0) {l = Logic::CUDA_ONLY; cout << "\nCuda with
    scheduler \n";}
72              if (i == 1) {l = Logic::STATIC_PARTITIONING; cout << "\nStatic
    partitioning \n";}
73              if (i == 2) {l = Logic::DYNAMIC; cout << "\nDynamic \n";}
74
75              AMScheduler scheduler = AMScheduler(l);
76              scheduler.do_tasks(task_array, n_matrix);
```

```
77              scheduler.wait();
78              /* test_compare_task(task_array, num_matrix); */
79              scheduler.print_stats(task_array,n_matrix);
80          }
81      }
82      clean_tasks(task_array, n_matrix);
83  }
84
85  void test_large_matrix_split() {
86      cout << "\nLarge Matrix Split \n";
87      Logic l = Logic::LARGE_MATRIX_SPLIT;
88      task* big_task = init_tasks(1, M_split,N_split,K_split,Type::FLOAT);
89      AMScheduler scheduler = AMScheduler(l);
90      scheduler.do_tasks(big_task, 1);
91      scheduler.wait();
92      scheduler.print_stats(nullptr,1);
93      /* test_compare_task(big_task, 1); */
94      clean_tasks(big_task, 1);
95  }
96
97  /********************** real benchmarks ****************************/
98
99  /* 23 GB */
100 const size_t RAM_LIMIT = 23ULL * 1024 * 1024 * 1024;
101
102 size_t check_mem(int n_tasks, int M, int N, int K) {
103     size_t elem = (size_t)M * K + (size_t)K * N + (size_t)M * N;
104     return n_tasks * elem * 4;
105 }
106
107 int get_max_tasks(int M, int N, int K) {
108     size_t elem = (size_t)M * K + (size_t)K * N + (size_t)M * N;
109     size_t task_size = elem * 4;
110     return (int)(RAM_LIMIT / task_size);
111 }
112
113 size_t actual_tasks_size(task* tasks, int n) {
114     size_t total_bytes = 0;
115     for (int i = 0; i < n; ++i) {
116         size_t matrix_A = (size_t)tasks[i].M * tasks[i].K;
117         size_t matrix_B = (size_t)tasks[i].K * tasks[i].N;
118         size_t matrix_C = (size_t)tasks[i].M * tasks[i].N;
119         total_bytes += (matrix_A + matrix_B + matrix_C) * 4;
120     }
121     return total_bytes;
122 }
123
124 void bench_static_partitioning() {
125     int sizes[] = {1024, 2048, 4096};
126
127     for (int s : sizes) {
128         int step;
129         if (s == 1024) step = 50;
130         else if (s == 2048) step = 20;
131         else step = 4;
132
133         int max_possible = get_max_tasks(s, s, s);
134
135         cout << "Allocating " << max_possible << " tasks for size " << s <<
        "...\n";
136         task* all_tasks = init_tasks(max_possible, s, s, s, Type::FLOAT);
```

```
137
138         for (int b = 20; b <= 70; b += 10) {
139             BATCH_SIZE = b;
140
141             int t = step;
142             bool last_run = false;
143
144             while (!last_run) {
145                 if (t >= max_possible) {
146                     t = max_possible;
147                     last_run = true;
148                 }
149
150                 csvname = "bin/csv/static_part_S" + to_string(s) + "_B" +
    to_string(b) + ".csv";
151
152                 if (check_mem(t, s, s, s) > RAM_LIMIT) {
153                     break;
154                 }
155
156                 cout << "Static Part | Size: " << s << " | Batch: " << b <<
     " | Tasks: " << t;
157                 if (last_run) cout << " (MAX RAM)";
158                 cout << endl;
159
160                 AMScheduler sched(Logic::STATIC_PARTITIONING);
161                 sched.do_tasks(all_tasks, t);
162                 sched.wait();
163                 sched.print_stats(all_tasks, t);
164
165                 t += step;
166             }
167         }
168         clean_tasks(all_tasks, max_possible);
169     }
170 }
171
172 void bench_dynamic_homo() {
173     int sizes[] = {1024, 2048, 4096};
174
175     cout << "\n=== START DYNAMIC & CUDA HOMOGENEOUS BENCHMARK ===\n";
176
177     for (int s : sizes) {
178         int step;
179         if (s == 1024) step = 50;
180         else if (s == 2048) step = 20;
181         else step = 4;
182
183         int max_possible = get_max_tasks(s, s, s);
184
185         cout << "Allocating " << max_possible << " tasks for size " << s <<
     "...\n";
186         task* all_tasks = init_tasks(max_possible, s, s, s, Type::FLOAT);
187
188         int t = step;
189         bool last_run = false;
190
191         while (!last_run) {
192             if (t >= max_possible) {
193                 t = max_possible;
194                 last_run = true;
```

```
195             }
196
197             if (check_mem(t, s, s, s) > RAM_LIMIT) {
198                 break;
199             }
200
201             csvname = "bin/csv/dynamic_homo_S" + to_string(s) + ".csv";
202             cout << "Dynamic Homo | Size: " << s << " | Tasks: " << t <<
    endl;
203
204             {
205                 AMScheduler sched_dyn(Logic::DYNAMIC);
206                 sched_dyn.do_tasks(all_tasks, t);
207                 sched_dyn.wait();
208                 sched_dyn.print_stats(all_tasks, t);
209             }
210
211             t += step;
212         }
213
214         t = step;
215         last_run = false;
216
217         while (!last_run) {
218             if (t >= max_possible) {
219                 t = max_possible;
220                 last_run = true;
221             }
222
223             if (check_mem(t, s, s, s) > RAM_LIMIT) {
224                 break;
225             }
226
227             csvname = "bin/csv/cuda_only_S" + to_string(s) + ".csv";
228             cout << "CUDA Only    | Size: " << s << " | Tasks: " << t;
229             if (last_run) cout << " (MAX RAM)";
230             cout << endl;
231
232             {
233                 AMScheduler sched_cuda(Logic::CUDA_ONLY);
234                 sched_cuda.do_tasks(all_tasks, t);
235                 sched_cuda.wait();
236                 sched_cuda.print_stats(all_tasks, t);
237             }
238
239             t += step;
240         }
241
242         clean_tasks(all_tasks, max_possible);
243     }
244 }
245
246 void bench_hetero_comparison() {
247     cout << "\n=== START HETERO COMPARISON BENCHMARK ===\n";
248
249     int max_allocatable = get_max_tasks(MAX_SIZE, MAX_SIZE, MAX_SIZE);
250
251     task* shared_tasks = init_hetero_tasks(max_allocatable, Type::FLOAT);
252
253     size_t current_mem = actual_tasks_size(shared_tasks, max_allocatable);
```

```
254     cout << "Allocated " << max_allocatable << " tasks using " << (double)
        current_mem / (1024*1024*1024) << " GB of RAM.\n";
255
256     int max_total_tasks = max_allocatable;
257     int step = 5;
258
259     for (int b = 20; b <= 70; b += 10) {
260         BATCH_SIZE_HETERO = b;
261
262         int t = step;
263         bool last_run = false;
264
265         while (!last_run) {
266             if (t >= max_total_tasks) {
267                 t = max_total_tasks;
268                 last_run = true;
269             }
270
271             csvname = "bin/csv/static_hetero_B" + to_string(b) + ".csv";
272             cout << "Static Hetero | Batch: " << b << " | Tasks: " << t;
273             if (last_run) cout << " (MAX ALLOC)";
274             cout << endl;
275
276             AMScheduler sched(Logic::STATIC_HETERO_PARTITIONING);
277             sched.do_tasks(shared_tasks, t);
278             sched.wait();
279             sched.print_stats(shared_tasks, t);
280
281             t += step;
282         }
283     }
284
285     cout << "\n--- Dynamic Logic on Hetero ---\n";
286
287     int t = step;
288     bool last_run = false;
289
290     while (!last_run) {
291         if (t >= max_total_tasks) {
292             t = max_total_tasks;
293             last_run = true;
294         }
295
296         csvname = "bin/csv/dynamic_hetero.csv";
297         cout << "Dynamic Hetero | Tasks: " << t;
298         if (last_run) cout << " (MAX ALLOC)";
299         cout << endl;
300
301         AMScheduler sched(Logic::DYNAMIC);
302         sched.do_tasks(shared_tasks, t);
303         sched.wait();
304         sched.print_stats(shared_tasks, t);
305
306         t += step;
307     }
308
309     cout << "\n--- CUDA Only Logic on Hetero ---\n";
310
311     t = step;
312     last_run = false;
313
```

```
314     while (!last_run) {
315         if (t >= max_total_tasks) {
316             t = max_total_tasks;
317             last_run = true;
318         }
319 
320         csvname = "bin/csv/cuda_only_hetero.csv";
321         cout << "CUDA Only Hetero | Tasks: " << t;
322         if (last_run) cout << " (MAX ALLOC)";
323         cout << endl;
324 
325         AMScheduler sched(Logic::CUDA_ONLY);
326         sched.do_tasks(shared_tasks, t);
327         sched.wait();
328         sched.print_stats(shared_tasks, t);
329 
330         t += step;
331     }
332 
333     clean_tasks(shared_tasks, max_total_tasks);
334 }
335 
336 void bench_large_matrix() {
337     int N = 4000;
338     int K = 4000;
339     int start_M = 5000;
340 
341 
342     for (int M = start_M; ; M += 1000) {
343         if (check_mem(1, M, N, K) > RAM_LIMIT) {
344             cout << "RAM limit reached (M=" << M << ")\n";
345             break;
346         }
347 
348         csvname = "bin/csv/large_matrix_cuda.csv";
349 
350         cout << "Large Matrix | M: " << M << " N: " << N << " K: " << K <<
    endl;
351 
352         task* t = init_tasks(1, M, N, K, Type::FLOAT);
353         AMScheduler sched(Logic::LARGE_MATRIX_SPLIT_CUDA);
354         sched.do_tasks(t, 1);
355         sched.wait();
356         sched.print_stats(t, 1);
357         clean_tasks(t, 1);
358     }
359 }
360 
361 int main() {
362     test_accellerators();
363     test_scheduler_logics();
364     test_large_matrix_split();
365     test_hetero_logic();
366     test_dynamic();
367     test_jit_times();
368 
369     /* remove openvino */
370     test_video_filter(Logic::CUDA_ONLY, false);
371     test_video_filter(Logic::DYNAMIC, true);
372     test_video_filter(Logic::STATIC_PARTITIONING, false);
373 
```

```
374     bench_static_partitioning();
375     bench_dynamic_homo();
376     bench_hetero_comparison();
377 
378     /* remove openvino */
379     bench_large_matrix();
380     return 0;
381 }
```

Listing A.5: Main (main.cpp)

```
1  #ifndef SCHEDULER_H
2  #define SCHEDULER_H
3  
4  #ifdef ENABLE_OPENVINO
5  #include "ov_wrapper.h"
6  #include <openvino/op/util/attr_types.hpp>
7  #endif
8  
9  #ifdef ENABLE_CUDA
10 #include "cuda_wrapper.h"
11 #endif
12 
13 #ifdef ENABLE_SYCL
14 #include <sycl/vector.hpp>
15 #include "sycl_wrapper.h"
16 #endif
17 
18 #ifdef ENABLE_OPENBLAS
19 #include "cpu.hpp"
20 #endif
21 
22 //#include "cuda_wrapper.h"
23 //#include "ov_wrapper.h"
24 //#include "sycl_wrapper.h"
25 //#include "cpu.hpp"
26 
27 #include "sharedbuffer.hpp"
28 #include "performancemap.hpp"
29 #include "profiler.hpp"
30 #include "tasks.hpp"
31 #include "config.hpp"
32 
33 #include <cassert>
34 #include <cstddef>
35 #include <cstdio>
36 #include <cstdlib>
37 #include <filesystem>
38 #include <iostream>
39 #include <atomic>
40 #include <stdlib.h>
41 #include <thread>
42 #include <array>
43 #include <memory>
44 #include <unistd.h>
45 #include <vector>
46 #include <fstream>
47 
```

```
48 #ifdef DEBUG
49     #define PRINT(x) std::cout << "DEBUG: " << x << std::endl
50 #else
51     #define PRINT(x)
52 #endif
53
54 #ifdef ENABLE_PROFILING
55     #define PROF(x) x
56 #else
57     #define PROF(x)
58 #endif
59
60 using namespace std;
61
62 /* Scheduler Logic Type */
63 enum Logic : size_t {
64     ROUND_ROBIN,
65     CUDA_ONLY,
66     STATIC_PARTITIONING,
67     LARGE_MATRIX_SPLIT,
68     LARGE_MATRIX_SPLIT_CUDA,
69     STATIC_HETERO_PARTITIONING,
70     DYNAMIC,
71 };
72
73 /****************************** Prototypes
    ******************************************/
74 inline void handle_task(BT backend_type, task *task);
75 inline string get_benchmark_filename(BT bt, int type);
76 inline void init_acc(BT backend_type);
77 inline void free_acc(BT backend_type);
78 inline void benchmark_acc(BT bt, Type type, string filename, int M, int N,
    int K);
79 inline void benchmark(BT bt, Type type, string filename);
80 inline string get_logic_string(Logic l);
81 inline string get_acc_string(BT backend_type);
82
83 /* for simplicity this is the buffer pointer */
84 using SharedBuffer_T = array<unique_ptr<SharedBuffer>, BT::COUNT>;
85
86 /****************************** Scheduler Class
    ******************************************/
87
88 class AMScheduler {
89     private:
90
91         /* atomic for shutting down threads */
92         atomic<bool> threads_keep_running;
93         atomic<int> pending_tasks = 0;
94         atomic<bool>* busy[BT::COUNT];
95
96         /* shared buffers beetween threads */
97         SharedBuffer_T shared_buffers;
98
99         /* array of threads, one for each accellerator */
100        array<unique_ptr<thread>, BT::COUNT> threads;
101
102        /* type of scheduler strategy to use */
103        Logic strategy;
104
105        /* backend_type vector for handling the accellerators */
```

```
106         vector<BT> bts;
107
108         /* Perfomance Map for each accellerator */
109         array<unique_ptr<PerformanceMap>, BT::COUNT> bts_map;
110
111         /* metrics profiler */
112         Profiler profiler = Profiler();
113
114         /* pointer for split_matrix logic*/
115         vector<task*> sub_task_array;
116
117 /*********************************  Worker
    ***********************************/
118
119         void worker_thread(BT bt, SharedBuffer *buffer) {
120             task* task = nullptr;
121             PROF(profiler.init_last_work(bt));
122
123             if (strategy == Logic::DYNAMIC) {
124
125                 /* Dynamic execution */
126                 while (threads_keep_running) {
127                     task = buffer->get();
128                     if (!task) {continue;}
129
130                     PROF(profiler.start_worker(bt));
131                     handle_task(bt, task);
132                     PROF(profiler.end_worker(bt));
133
134                     *busy[bt] = false;
135                     pending_tasks--;
136                 }
137
138             } else {
139
140                 /* Static execution */
141                 while (threads_keep_running) {
142                     /* blocking if SLEEP on */
143                     task = buffer->get();
144
145                     /* if there isn't a task we check if we have to
    shutdown */
146                     if (!task) {continue;}
147
148                     PROF(profiler.start_worker(bt));
149                     handle_task(bt, task);
150                     PROF(profiler.end_worker(bt));
151                     pending_tasks--;
152                 }
153
154             }
155         }
156
157         /*********************************  Coordinator
    ***********************************/
158
159         /* if there is no task, we check if we have to shutdown */
160         void coordinator_thread(SharedBuffer_T &buffers) {
161             int i=0;
162             int c=0;
163             int bts_len = bts.size(); /* number of accellerators */
```

```
164
165             double acc_speed[bts_len];
166             double total_speed=0.0;
167             int n_acc_task[BATCH_SIZE];
168             int remaining_c=0;
169             int t=0;
170
171             SharedBuffer *buffer = buffers[BT::CORDINATOR].get();
172             task* single_task = nullptr;
173             task* tasks[BATCH_SIZE];
174             task* tasks_hetero[BATCH_SIZE_HETERO];
175
176             bool dispached = true;
177
178             double bt_ms[bts_len];
179             for (int i=0; i<bts_len; i++)
180                 bt_ms[i]=0.0;
181
182             switch (strategy) {
183             case Logic::CUDA_ONLY:
184
185                 PROF(profiler.start_power_monitor());
186
187                 if (buffers[BT::CUDA] == nullptr) {
188                     cerr << "[SCHEDULER] Error, cuda logic but no cuda card
    ";
189                     exit(EXIT_FAILURE);
190                 }
191
192                 while (threads_keep_running) {
193                     single_task = buffer->get();
194                     if (!single_task) {continue;}
195                     buffers[BT::CUDA]->put(single_task);
196                 }
197
198                 PROF(profiler.stop_power_monitor());
199
200                 break;
201
202             /* basic logic, just send all the tasks to all the bt */
203             case Logic::ROUND_ROBIN:
204
205                 while (threads_keep_running) {
206                     single_task = buffer->get();
207                     if (!single_task) {continue;}
208
209                     buffers[bts[i]]->put(single_task);
210
211                     i = (i+1) % bts_len;
212                 }
213                 break;
214
215                 /* batch style partitioning */
216             case Logic::STATIC_PARTITIONING:
217
218                 PROF(profiler.start_power_monitor());
219
220                 while (threads_keep_running) {
221
222                     PROF(profiler.start_fetch());
223
```

```
224                     while (c < BATCH_SIZE) {
225                         tasks[c] = buffer->get();
226                         if (!tasks[c]) {break;}
227                         c++;
228                     }
229 
230                     PROF(profiler.stop_fetch());
231 
232                     PROF(profiler.start_logic());
233 
234                     if (c == 0) {continue;}
235 
236                     /* if tasks less number of bts, send all to the fastest
     */
237                     if (bts_len >= c) {
238                         for (int j=0; j<c; j++)
239                             buffers[bts[0]]->put(tasks[j]);
240                         c = 0;
241                         continue;
242                     }
243 
244                     /* speeds */
245                     total_speed = 0.0;
246                     for (int j=0; j<bts_len; j++){
247                         double single_matrix_ms = bts_map[bts[j]]->query(
    tasks[0]->M, tasks[0]->N, tasks[0]->K, tasks[0]->type, true);
248                         acc_speed[j] = 1.0 / (single_matrix_ms + 1e-9); /*
    1e-9 for not dividing for 0 */
249                         total_speed += acc_speed[j];
250                     }
251 
252                     /* exact task per acc (double) */
253                     double shares[bts_len];
254                     int total_assigned = 0;
255 
256                     for (int j=0; j<bts_len; j++) {
257                         double ratio = acc_speed[j] / total_speed;
258                         shares[j] = ratio * (double)c;
259                         n_acc_task[j] = (int)shares[j]; /* integer part */
260                         total_assigned += n_acc_task[j];
261                     }
262 
263                     int missing_c = c - total_assigned;
264 
265                     /* assign the missing task to the accellerator
266                      * mitigate slowness between accelleratorS */
267                     while (missing_c > 0) {
268                         int best_j = -1;
269                         double max_decimal = -1.0;
270 
271                         /* find the accellerator with the highest decimal
    part */
272                         for (int j=0; j<bts_len; j++) {
273                             double decimal_part = shares[j] - (double)
    n_acc_task[j];
274                             if (decimal_part > max_decimal) {
275                                 max_decimal = decimal_part;
276                                 best_j = j;
277                             }
278                         }
279 
```

```
280                         /* assign one more task to the winner */
281                         if (best_j != -1) {
282                             n_acc_task[best_j]++;
283                             /* set to -1 so it won't be picked again */
284                             shares[best_j] = -1.0;
285                             missing_c--;
286                         } else {
287                             /* fallback */
288                             n_acc_task[0]++;
289                             missing_c--;
290                         }
291                     }
292 
293                     PROF(profiler.stop_logic());
294 
295                     PROF(profiler.start_dispatch());
296 
297                     /* send task to the accellerators */
298                     t = 0;
299                     for (int j=0; j<bts_len; j++) {
300                         for (int w=0; w<n_acc_task[j]; w++){
301                             if (t >= c) break;
302 
303                             buffers[bts[j]]->put(tasks[t]);
304                             t++;
305                         }
306                     }
307 
308                     c = 0;
309                     PROF(profiler.stop_dispatch());
310                     PROF(profiler.record_sample());
311                 }
312 
313                 PROF(profiler.stop_power_monitor());
314                 break;
315 
316             /* split a large matrix row wise with Iterative Refinement
    Partitioning*/
317             case Logic::LARGE_MATRIX_SPLIT:
318 
319                 PROF(profiler.start_power_monitor());
320 
321                 while (threads_keep_running) {
322                     PROF(profiler.start_fetch());
323 
324                     single_task = buffer->get();
325                     if (!single_task) {continue;}
326 
327                     PROF(profiler.stop_fetch());
328                     PROF(profiler.start_logic());
329 
330                     double curr_speeds[bts_len];
331                     int rows[bts_len];
332 
333                     int rest = single_task->M % bts_len;
334                     int base = single_task->M / bts_len;
335 
336                     /* equal row initially */
337                     for (int j=0; j<bts_len; j++)
338                         rows[j] = base + (j < rest ? 1 : 0);
339 
```

```
340                     /* loop for adjusting the ratios */
341                     for (int i = 0; i < SPLIT_MATRIX_ITERATION; i++) {
342                         double total_speed = 0.0;
343 
344                         /* current speed for 1 row */
345                         for (int j=0; j<bts_len; j++) {
346                             int r = (rows[j] > 0) ? rows[j] : 1;
347 
348                             double ms = 0.0;
349 
350                             ms = bts_map[bts[j]]->query(r, single_task->N,
    single_task->K, single_task->type, false);
351 
352                             /* update time if r exceed max size */
353                             if (r > MAX_SIZE){
354                                 double max_ms = bts_map[bts[j]]->query(
    MAX_SIZE, single_task->N, single_task->K, single_task->type, false);
355                                 ms = max_ms * ((double)r / (double)MAX_SIZE
    );
356                             }
357 
358 
359                             cout << "acc " << j << " ms " <<  ms << " for M
    : " << r << " N: " << single_task->N << " K: " << single_task->K << " \
    n";
360 
361                             /* single row speed, number of rows / ms */
362                             double speed = (double)r / (ms + 1e-9);
363 
364                             curr_speeds[j] = speed;
365                             total_speed += speed;
366                         }
367 
368                         /* adjust the row with the ratios */
369                         int rows_distributed = 0;
370                         for (int j=0; j<bts_len; j++) {
371                             double ratio = curr_speeds[j] / total_speed;
372 
373                             /* new rows from the ratio */
374                             int target_rows = (int)(ratio * single_task->M)
    ;
375 
376                             rows[j] = target_rows;
377                             rows_distributed += rows[j];
378                         }
379                     }
380 
381                     PROF(profiler.stop_logic());
382                     PROF(profiler.start_dispatch());
383 
384                     size_t elem_size = (single_task->type == Type::FLOAT) ?
     4 : 2;
385                     size_t offset_rows_acc = 0;
386 
387                     /* dispatch tasks */
388                     for (int j=0; j<bts_len; j++) {
389                         int h = rows[j];
390                         cout << get_acc_string(bts[j]) << " get " << h << "
     rows\n";
391                         if (h <= 0) continue;
392 
```

```
393                         /* divide rows into tasks if exceed MAX_SIZE */
394                         while (h > 0) {
395                             int h_limit = h;
396 
397                             if (h_limit > MAX_SIZE)
398                                 h_limit = MAX_SIZE;
399 
400                             task* sub_task = new ::task;
401                             *sub_task = *single_task;
402 
403                             /* save the pointer for later freeup */
404                             sub_task_array.push_back(sub_task);
405 
406                             /* A row */
407                             sub_task->M = h_limit;
408 
409                             /* A offset */
410                             size_t offset_A = offset_rows_acc * single_task
    ->K * elem_size;
411                             sub_task->A = (char*)single_task->A + offset_A;
412 
413                             /* B offset still all in memory */
414                             sub_task->B = single_task->B;
415 
416                             /* C offset */
417                             size_t offset_C = offset_rows_acc * single_task
    ->N * elem_size;
418                             sub_task->C = (char*)single_task->C + offset_C;
419 
420                             /* add task to the pending */
421                             pending_tasks++;
422 
423                             /* submit to the acc */
424                             buffers[bts[j]]->put(sub_task);
425 
426                             /* for each accellerator we add more offset */
427                             offset_rows_acc += h_limit;
428 
429                             h -= h_limit;
430                         }
431                     }
432 
433                     /* remove task */
434                     pending_tasks--;
435                     PROF(profiler.stop_dispatch());
436                     PROF(profiler.record_sample());
437                 }
438 
439                 PROF(profiler.stop_power_monitor());
440                 break;
441 
442             case Logic::STATIC_HETERO_PARTITIONING:
443 
444                 PROF(profiler.start_power_monitor());
445 
446                 while (threads_keep_running) {
447 
448                     PROF(profiler.start_fetch());
449 
450                     while (c < BATCH_SIZE_HETERO) {
451                         tasks_hetero[c] = buffer->get();
```

```
452                         if (!tasks_hetero[c]) {break;}
453                         c++;
454                     }
455 
456                     PROF(profiler.stop_fetch());
457 
458                     PROF(profiler.start_logic());
459                     if (c == 0) {continue;}
460 
461 
462                     /* iterator for each bt */
463                     int i_bt[bts_len];
464 
465                     for (int i=0; i<bts_len; i++)
466                         i_bt[i] = 0;
467 
468                     task* dispatch_tasks[bts_len][BATCH_SIZE_HETERO];
469 
470                     /* for every task */
471                     for (int i = 0; i < c; i++) {
472                         int best_bt = -1;
473                         double min_ms = 1e15;
474 
475                         /* for every bt */
476                         for (int j = 0; j < bts_len; j++) {
477                             double query_ms = bts_map[bts[j]]->query(
    tasks_hetero[i]->M, tasks_hetero[i]->N, tasks_hetero[i]->K,
    tasks_hetero[i]->type, false);
478 
479                             double finish_ms = bt_ms[j] + query_ms;
480 
481                             /* we assign the task to the least "filled" */
482                             if (finish_ms < min_ms) {
483                                 min_ms = finish_ms;
484                                 best_bt = j;
485                             }
486                         }
487 
488                         /* openvino and sycl jit updates */
489                         if (bts[best_bt] == BT::OPENVINO || bts[best_bt] ==
     BT::SYCL)
490                             bts_map[bts[best_bt]]->query(tasks_hetero[i]->M
    , tasks_hetero[i]->N, tasks_hetero[i]->K, tasks_hetero[i]->type, true);
491 
492                         dispatch_tasks[best_bt][i_bt[best_bt]] =
    tasks_hetero[i];
493                         i_bt[best_bt]++;
494                         bt_ms[best_bt] = min_ms;
495                     }
496 
497                     PROF(profiler.stop_logic());
498 
499                     PROF(profiler.start_dispatch());
500                     for (int i = 0; i < bts_len; i++) {
501                        cout << "acc" << get_acc_string(bts[i]) << "
    contiene : " << i_bt[i] <<  " tasks con " << bt_ms[i] << " ms di carico
     \n";
502 
503                         for (int j=0; j<i_bt[i]; j++)
504                             buffers[bts[i]]->put(dispatch_tasks[i][j]);
505 
```

```
506                     }
507                     PROF(profiler.stop_dispatch());
508                     PROF(profiler.record_sample());
509                     c = 0;
510                 }
511 
512                 PROF(profiler.stop_power_monitor());
513                 break;
514 
515             case Logic::DYNAMIC:
516 
517                 PROF(profiler.start_power_monitor());
518 
519                 i = 0;
520                 while (threads_keep_running) {
521                     PROF(profiler.start_logic());
522 
523                     if (dispached) {
524                         PROF(profiler.start_fetch());
525                         single_task = buffer->get();
526                         PROF(profiler.stop_fetch());
527                         if (!single_task) {continue;}
528                         dispached = false;
529                     }
530 
531                     if (*busy[bts[i]] == false) {
532                         *busy[bts[i]] = true;
533                         dispached = true;
534                         PROF(profiler.start_dispatch());
535                         buffers[bts[i]]->put(single_task);
536                         PROF(profiler.stop_dispatch());
537                     }
538 
539                     i++;
540                     i = i % bts_len;
541 
542                     PROF(profiler.stop_logic());
543 
544                     if (dispached)
545                         PROF(profiler.record_sample());
546                 }
547 
548 
549                 PROF(profiler.stop_power_monitor());
550                 break;
551 
552             case Logic::LARGE_MATRIX_SPLIT_CUDA:
553 
554                 PROF(profiler.start_power_monitor());
555 
556                 if (buffers[BT::CUDA] == nullptr) {
557                     cout << "[SCHEDULER] Error: no CUDA backend available.\
    n";
558                     exit(EXIT_FAILURE);
559                 }
560 
561                 while (threads_keep_running) {
562                     PROF(profiler.start_fetch());
563 
564                     single_task = buffer->get();
565                     if (!single_task) {continue;}
```

```
566
567                     PROF(profiler.stop_fetch());
568
569                     PROF(profiler.start_logic());
570                     int rows_left = single_task->M;
571                     int offset_rows = 0;
572                     size_t elem_size = (single_task->type == Type::FLOAT) ?
        4 : 2;
573                     PROF(profiler.stop_logic());
574
575                     PROF(profiler.start_dispatch());
576
577                     while (rows_left > 0) {
578
579                         int chunk_h = rows_left;
580                         if (chunk_h > MAX_SIZE) chunk_h = MAX_SIZE;
581
582                         task* sub_task = new ::task;
583                         *sub_task = *single_task;
584
585                         sub_task_array.push_back(sub_task);
586
587                         sub_task->M = chunk_h;
588
589                         size_t offset_A = (size_t)offset_rows * single_task
    ->K * elem_size;
590                         size_t offset_C = (size_t)offset_rows * single_task
    ->N * elem_size;
591
592                         sub_task->A = (char*)single_task->A + offset_A;
593                         sub_task->C = (char*)single_task->C + offset_C;
594
595                         pending_tasks++;
596
597                         buffers[BT::CUDA]->put(sub_task);
598
599                         rows_left -= chunk_h;
600                         offset_rows += chunk_h;
601                     }
602
603                     pending_tasks--;
604
605                     PROF(profiler.stop_dispatch());
606                     PROF(profiler.record_sample());
607                 }
608
609                 PROF(profiler.stop_power_monitor());
610                 break;
611             default:
612                 printf("[SCHEDULER] Error in chosing the logic.\n");
613                 exit(EXIT_FAILURE);
614             }
615         }
616
617 /*********************************  Init and Stop
        **********************************/
618
619         void init_threads() {
620
621             /* from the fastest to the slowest */
622             /* init here and not in the threads */
```

```
623 #ifdef ENABLE_CUDA
624             init_acc(BT::CUDA);
625             bts.push_back(BT::CUDA);
626 #endif
627 #ifdef ENABLE_SYCL
628             init_acc(BT::SYCL);
629             bts.push_back(BT::SYCL);
630 #endif
631 #ifdef ENABLE_OPENVINO
632             init_acc(BT::OPENVINO);
633             bts.push_back(BT::OPENVINO);
634 #endif
635 #ifdef ENABLE_OPENBLAS
636             init_acc(BT::OPENBLAS);
637             bts.push_back(BT::OPENBLAS);
638 #endif
639
640             /* initialize atomics for dynamic strategy */
641             if(strategy == Logic::DYNAMIC) {
642                 #ifdef SLEEP
643                 cout << "[SCHEDULER] ERROR remove "#define SLEEP" with
    Dynamic logic\n";
644                 exit(EXIT_FAILURE);
645                 #endif
646                 for (int i=0; i<bts.size(); i++) {
647                     busy[bts[i]] = new atomic<bool>;
648                     *busy[bts[i]] = false;
649                 }
650             }
651
652             for (BT i : bts) {
653                 shared_buffers[i] = make_unique<SharedBuffer>(BUFFER_LENGHT
    );
654                 threads[i] = make_unique<thread>(&AMScheduler::
    worker_thread, this,
655                                                  i, shared_buffers[i].
    get());
656             }
657
658             if (bts.size() == 0) {PRINT("[SCHEDULER] ATTENTION, no
    accellerator\n"); exit(EXIT_FAILURE);}
659
660             /* coordinator thread */
661             shared_buffers[BT::CORDINATOR] = make_unique<SharedBuffer>(
    BUFFER_LENGHT);
662             threads[BT::CORDINATOR] = make_unique<thread>(&AMScheduler::
    coordinator_thread,this,
663                                         ref(shared_buffers)); /* ref
    because the thread function try to copy all the parameters */
664         }
665
666         /* shut down threads */
667         void stop_threads() {
668             threads_keep_running = false;
669
670             for (auto& thread : threads)
671                 if (thread) /* unique pointer are null if not initializated
     */
672                     if(thread->joinable())
673                         thread->join();
674
```

```
675             /* shut down backends */
676             for (auto i : bts)
677                 free_acc(i);
678 
679             /* cleanups sub_task array */
680             if (strategy == Logic::LARGE_MATRIX_SPLIT)
681                 for (int i=0; i<sub_task_array.size(); i++)
682                     delete sub_task_array[i];
683 
684             /* cleanups atomics */
685             if(strategy == Logic::DYNAMIC)
686                 for (int i=0; i<bts.size(); i++)
687                     delete busy[bts[i]];
688 
689             PRINT("[SCHEDULER] Threads stopped.\n");
690         }
691 
692         /*********************************  Init Benchmarks and Map
    **********************************/
693 
694         /* upload the banchmark files and generate them if not presents */
695         void init_benchmarks() {
696             for (BT i : bts) {
697 
698                 string filename_f;
699                 string filename_h;
700 
701                 filename_f = get_benchmark_filename(i, Type::FLOAT);
702                 filename_h = get_benchmark_filename(i, Type::HALF);
703 
704                 if (filesystem::exists(filename_f)) {
705                     PRINT("[SCHEDULER] File: " << filename_f <<  " opened
    .\n");
706                 } else {
707                     PRINT("[SCHEDULER] File: " << filename_f <<  " not
    present\n");
708                     benchmark(i, Type::FLOAT ,filename_f);
709                     PRINT("[SCHEDULER] Benchmark file created\n");
710                 }
711 
712                 if (filesystem::exists(filename_h)){
713                     PRINT("[SCHEDULER] File: " << filename_h <<  " opened
    .\n");
714                 } else {
715                     PRINT("[SCHEDULER] File: " << filename_h <<  " not
    present\n");
716                     benchmark(i, Type::HALF ,filename_h);
717                     PRINT("[SCHEDULER] Benchmark file created\n");
718                 }
719             }
720         }
721 
722         /* create the map for each accellerator */
723         void init_maps() {
724             for (BT i : bts) {
725                 string filename_f;
726                 string filename_h;
727                 filename_f = get_benchmark_filename(i, Type::FLOAT);
728                 filename_h = get_benchmark_filename(i, Type::HALF);
729 
730                 if (filesystem::exists(filename_f) && filesystem::exists(
```

```
    filename_h)) {
731                     bts_map[i] = make_unique<PerformanceMap>(STEP_SIZE, i,
    filename_f, filename_h);
732                 } else {
733                     cerr << "[SCHEDULER] File: " << filename_f <<  " or "
    << filename_h << " not found for map init\n";
734                     exit(EXIT_FAILURE);
735                 }
736             }
737         }
738 
739         /*********************************  Public Interface
    *********************************/
740 
741     public:
742         /* constructur init threads, benchmark files, perfomanceMaps */
743         AMScheduler(Logic logic) : threads_keep_running(true), strategy(
    logic) {
744             get_logic_string(logic);
745             init_threads();
746             PRINT("[SCHEDULER] Threads started.\n");
747             init_benchmarks();
748             PRINT("[SCHEDULER] Benchmarks done.\n");
749             init_maps();
750             PRINT("[SCHEDULER] PerformanceMap done.\n");
751         }
752 
753         /* destructur, stop the threads*/
754         ~AMScheduler() {
755             stop_threads();
756         }
757 
758         /* send the tasks to the coordinator */
759         void do_tasks(task* tasks, size_t n) {
760             pending_tasks += n;
761             for (int i=0; i<n; i++) {
762                 tasks[i].start_time = chrono::high_resolution_clock::now();
763                 shared_buffers[BT::CORDINATOR]->put(&tasks[i]);
764             }
765             PRINT("[SCHEDULER] Task dispached.\n");
766         }
767 
768         /* check if all the buffers are empty */
769         void wait() {
770             PRINT("[SCHEDULER] Waiting.. \n");
771 
772             while (pending_tasks > 0) {
773                 usleep(1000);
774             }
775 
776             PRINT("[SCHEDULER] All tasks done. \n");
777         }
778 
779         /* return the profiler for printing the stats */
780         void print_stats(task* tasks, int num_tasks) {
781             if (strategy == Logic::LARGE_MATRIX_SPLIT) {
782                 profiler.print_stats(bts, sub_task_array, get_logic_string(
    strategy));
783             } else {
784                 profiler.print_stats(bts, tasks, num_tasks,
    get_logic_string(strategy));
```

```
785             }
786         }
787 };
788
789 /*********************************  Helper Functions
    **********************************/
790
791 /* benchmark the accellerator and write in the csv*/
792 /* RECALL: NON USARE LA SOLITA TASK PER IL BENCHMARK per la cache */
793 inline void benchmark_acc(BT bt, Type type, string filename, int M, int N,
    int K) {
794     chrono::high_resolution_clock::time_point start_time;
795     chrono::high_resolution_clock::time_point end_time;
796     chrono::duration<double, std::milli> duration;
797
798     const int N_RUNS = 4;
799     const int N_TASKS = N_RUNS + 2;
800
801     double total_ms = 0.0;
802     double jit_ms = 0.0;
803     double no_jit_ms = 0.0;
804
805     /* we simulate a thread */
806     task* tasks = init_tasks(N_TASKS, M, N, K, type);
807
808     /* warmup | jit detection */
809     start_time = chrono::high_resolution_clock::now();
810     handle_task(bt, &tasks[0]);
811     end_time = chrono::high_resolution_clock::now();
812     duration = end_time - start_time;
813     jit_ms = duration.count();
814
815     start_time = chrono::high_resolution_clock::now();
816     handle_task(bt, &tasks[1]);
817     end_time = chrono::high_resolution_clock::now();
818     duration = end_time - start_time;
819     jit_ms = jit_ms - duration.count();
820
821     /* runs with */
822     for (int i = 2; i < N_TASKS; i++) {
823         start_time = chrono::high_resolution_clock::now();
824         handle_task(bt, &tasks[i]);
825         end_time = chrono::high_resolution_clock::now();
826         duration = end_time - start_time;
827         total_ms += duration.count();
828     }
829
830     clean_tasks(tasks, N_TASKS);
831
832     double avg_ms = total_ms/N_RUNS;
833
834     ofstream file(filename, ios::app);
835
836     if (file.is_open()) {
837         if (filesystem::file_size(filename) == 0)
838             file << "Accelerator,DataType,M,N,K,Avg_Time_ms,Jit_Time_ms\n";
839
840         string acc_str = get_acc_string(bt);
841         file << acc_str << "," << type << "," << M << "," << N << "," << K
    << "," << avg_ms << "," << jit_ms << "\n";
842
```

```
843         PRINT("result: " << acc_str << "," << type << "," << M << "," << N
    << "," << K << "," << avg_ms << "," << jit_ms << "\n");
844         file.close();
845     } else {
846         cerr << "[SCEDULER] ERROR Cannot open file: " << filename << "\n";
847     }
848 }
849
850 /* call benchmark_acc for each test matrix*/
851 inline void benchmark(BT bt, Type type, string filename) {
852     vector<int> dims;
853
854     for (int d = STEP_SIZE; d <= STEP_SIZE*STEP_TOTAL; d += STEP_SIZE)
855         dims.push_back(d);
856
857     for (int m : dims)
858         for (int n : dims)
859             for (int k : dims)
860                 benchmark_acc(bt, type, filename, m, n, k);
861
862     PRINT("[SCHEDULER] Benchmark ===\n");
863     PRINT("Results saved in bin/csv/" << filename << "\n");
864 }
865
866 /* task handler for the workers, choose the right backend_type */
867 inline void handle_task(BT backend_type, task *task) {
868     switch (backend_type) {
869     case BT::CUDA:
870 #ifdef ENABLE_CUDA
871         if (task->type == Type::FLOAT)
872             cuda_gemm_32bit(task->A,task->B,task->C, task->M, task->N, task
    ->K);
873         if (task->type == Type::HALF)
874             cuda_gemm_16bit(task->A,task->B,task->C, task->M, task->N, task
    ->K);
875 #endif
876         break;
877
878     case BT::SYCL:
879 #ifdef ENABLE_SYCL
880         if (task->type == Type::FLOAT)
881             sycl_gemm_32bit(task->A,task->B,task->C, task->M, task->N, task
    ->K);
882         if (task->type == Type::HALF)
883             sycl_gemm_16bit(task->A,task->B,task->C, task->M, task->N, task
    ->K);
884 #endif
885         break;
886
887     case BT::OPENVINO:
888 #ifdef ENABLE_OPENVINO
889         if (task->type == Type::FLOAT)
890             ov_gemm_32bit(task->A,task->B,task->C, task->M, task->N, task->
    K);
891         if (task->type == Type::HALF)
892             ov_gemm_16bit(task->A,task->B,task->C, task->M, task->N, task->
    K);
893 #endif
894         break;
895
896     case BT::OPENBLAS:
```

```
897 #ifdef ENABLE_OPENBLAS
898         if (task->type == Type::FLOAT)
899             cpu_gemm_32bit(task->A,task->B,task->C, task->M, task->N, task
    ->K);
900         if (task->type == Type::HALF)
901             cpu_gemm_16bit(task->A,task->B,task->C, task->M, task->N, task
    ->K);
902 #endif
903         break;
904
905     default:
906         cerr << "[SCHEDULER] ERROR handle task\n";
907         exit(EXIT_FAILURE);
908     }
909
910     task->end_time = chrono::high_resolution_clock::now();
911 }
912
913 /* call the init func of the accellerator */
914 inline void init_acc(BT backend_type) {
915     switch (backend_type) {
916     case BT::CUDA:
917 #ifdef ENABLE_CUDA
918         cuda_init();
919 #endif
920         break;
921     case BT::SYCL:
922 #ifdef ENABLE_SYCL
923         sycl_init();
924 #endif
925         break;
926     case BT::OPENVINO:
927 #ifdef ENABLE_OPENVINO
928         ov_init();
929 #endif
930         break;
931     case BT::OPENBLAS:
932 #ifdef ENABLE_OPENBLAS
933         cpu_init();
934 #endif
935         break;
936     default:
937         cerr << "[SCHEDULER] ERROR Change Status\n";
938         exit(EXIT_FAILURE);
939     }
940 }
941
942 /* call the init func of the accellerator */
943 inline void free_acc(BT backend_type) {
944     switch (backend_type) {
945     case BT::CUDA:
946 #ifdef ENABLE_CUDA
947         cuda_free();
948 #endif
949     case BT::SYCL:
950 #ifdef ENABLE_SYCL
951         sycl_free();
952 #endif
953         break;
954     case BT::OPENVINO:
955 #ifdef ENABLE_OPENVINO
```

```
956         ov_free();
957 #endif
958         break;
959     case BT::OPENBLAS:
960 #ifdef ENABLE_OPENBLAS
961 #endif
962         break;
963     default:
964         cerr << "[SCHEDULER] ERROR Change Status\n";
965         exit(EXIT_FAILURE);
966     }
967 }
968
969 inline string get_logic_string(Logic l) {
970     string logic;
971     switch (l) {
972         case Logic::ROUND_ROBIN:
973             logic = "ROUND_ROBIN"; break;
974         case Logic::CUDA_ONLY:
975             logic = "CUDA_ONLY"; break;
976         case Logic::STATIC_PARTITIONING:
977             logic = "STATIC_PARTITIONING"; break;
978         case Logic::LARGE_MATRIX_SPLIT:
979             logic = "LARGE_MATRIX_SPLIT"; break;
980         case Logic::LARGE_MATRIX_SPLIT_CUDA:
981             logic = "LARGE_MATRIX_SPLIT_CUDA"; break;
982         case Logic::STATIC_HETERO_PARTITIONING:
983             logic = "STATIC_HETERO_PARTITIONING"; break;
984         case Logic::DYNAMIC:
985             logic = "DYNAMIC"; break;
986         default:
987             cerr << "[SCHEDULER] ERROR print_logic \n";
988             exit(EXIT_FAILURE);
989     }
990     PRINT("[SCHEDULER] Started with " << logic << " logic \n");
991     return logic;
992 }
993
994 /* get the right csv filename for each accellerators */
995 inline string get_benchmark_filename(BT bt, int type) {
996     string base_path = "bin/csv/";
997     if (!filesystem::exists(base_path)) {
998         filesystem::create_directories(base_path);
999     }
1000
1001     string acc_name = get_acc_string(bt) ;
1002
1003     string type_name;
1004     switch (type) {
1005         case Type::FLOAT: type_name = "FLOAT"; break;
1006         case Type::HALF: type_name = "HALF"; break;
1007         case Type::UINT8: type_name = "UINT8"; break;
1008         default:
1009             cerr << "[SCHEDULER] ERROR type get_benchmark_filename \n";
1010             exit(EXIT_FAILURE);
1011     }
1012
1013     return base_path + acc_name + "_" + type_name + ".csv";
1014 }
```

```
#endif
```

Listing A.6: Scheduler class (scheduler.hpp)

```
#ifndef TASKS_H
#define TASKS_H

#include <cstdint>
#include <cstring>
#include <chrono>
#include <iostream>
#include <random>

#include "config.hpp"

using namespace std;

/* backend_type */
enum BT : size_t {
    CORDINATOR,
    CUDA,
    SYCL,
    OPENVINO,
    OPENBLAS,
    COUNT,
};

enum Type : int {
    FLOAT,
    HALF,
    UINT8,
};

typedef struct {
    void *A;
    void *B;
    void *C;

    Type type; /* 1 float, 2 half, 3 8bit */
    int M;
    int N;
    int K;

    /* benchmarking */
    chrono::high_resolution_clock::time_point start_time;
    chrono::high_resolution_clock::time_point end_time;
} task;

inline float half_to_float(uint16_t h);
inline uint16_t float_to_half(float f);
inline void init_16bit(void* A, void* B, void* C, int M, int N, int K);
inline void init_32bit(void* A, void* B, void* C, int M, int N, int K);
inline void clean_tasks(task* array_task, size_t n_task);

inline task* init_tasks(size_t n_task, int m, int n, int k, Type t) {
        task* array_task = new task[n_task];

        for (int i = 0; i < n_task; i++) {
```

```
55              array_task[i].M = m;
56              array_task[i].N = n;
57              array_task[i].K = k;
58              array_task[i].type = t;
59
60              if (t == Type::FLOAT) {
61                  array_task[i].A = new float[m * k];
62                  array_task[i].B = new float[k * n];
63                  array_task[i].C = new float[m * n];
64
65                  init_32bit(array_task[i].A,array_task[i].B,array_task[i].C,
     m, n, k);
66
67              } else if (t == Type::HALF) {
68                  array_task[i].A = new uint16_t[m * k];
69                  array_task[i].B = new uint16_t[k * n];
70                  array_task[i].C = new uint16_t[m * n];
71
72                  init_16bit(array_task[i].A, array_task[i].B, array_task[i].
    C, m, n, k);
73              }
74          }
75
76          return array_task;
77  }
78
79  inline task* init_hetero_tasks(size_t n_task, Type t) {
80      task* array_task = new task[n_task];
81
82      std::random_device rd;
83      std::mt19937 gen(rd());
84
85      std::vector<int> possible_sizes;
86      possible_sizes.reserve(STEP_TOTAL);
87
88      for (int s = 1; s <= STEP_TOTAL; s++)
89          possible_sizes.push_back(s * STEP_SIZE);
90
91      std::uniform_int_distribution<> dis_idx(0, possible_sizes.size() - 1);
92
93      for (size_t i = 0; i < n_task; i++) {
94          int curr_m = possible_sizes[dis_idx(gen)];
95          int curr_n = possible_sizes[dis_idx(gen)];
96          int curr_k = possible_sizes[dis_idx(gen)];
97
98          //cout << " M : " << curr_m << " N: " << curr_n << " K: " << curr_k
     << "\n";
99
100         array_task[i].M = curr_m;
101         array_task[i].N = curr_n;
102         array_task[i].K = curr_k;
103         array_task[i].type = t;
104
105         size_t A = (size_t)curr_m * curr_k;
106         size_t B = (size_t)curr_k * curr_n;
107         size_t C = (size_t)curr_m * curr_n;
108
109         if (t == Type::FLOAT) {
110             array_task[i].A = new float[A];
111             array_task[i].B = new float[B];
112             array_task[i].C = new float[C];
```

```
113
114             init_32bit(array_task[i].A, array_task[i].B, array_task[i].C,
    curr_m, curr_n, curr_k);
115
116         } else if (t == Type::HALF) {
117             array_task[i].A = new uint16_t[A];
118             array_task[i].B = new uint16_t[B];
119             array_task[i].C = new uint16_t[C];
120
121             init_16bit(array_task[i].A, array_task[i].B, array_task[i].C,
    curr_m, curr_n, curr_k);
122         }
123     }
124
125     return array_task;
126 }
127
128 inline void clean_tasks(task* array_task, size_t n_task) {
129     if (array_task == nullptr) return;
130
131     Type type = array_task->type;
132
133     if (type == Type::FLOAT) {
134         for (int i = 0; i < n_task; i++) {
135             delete[] static_cast<float*>(array_task[i].A);
136             delete[] static_cast<float*>(array_task[i].B);
137             delete[] static_cast<float*>(array_task[i].C);
138         }
139     }
140
141     if (type == Type::HALF) {
142         for (int i = 0; i < n_task; i++) {
143             delete[] static_cast<uint16_t*>(array_task[i].A);
144             delete[] static_cast<uint16_t*>(array_task[i].B);
145             delete[] static_cast<uint16_t*>(array_task[i].C);
146         }
147     }
148
149     delete[] array_task;
150 }
151
152 inline uint16_t float_to_half(float f) {
153     uint32_t x; memcpy(&x, &f, 4);
154     uint32_t s = (x >> 31) & 0x00000001;
155     uint32_t e = (x >> 23) & 0x000000ff;
156     uint32_t m = x & 0x007fffff;
157
158     uint32_t h_e, h_m;
159
160     if (e >= 143) {
161         h_e = 31; h_m = 0;
162     }
163     else if (e < 113) {
164         h_e = 0; h_m = 0;
165     }
166     else {
167         h_e = e - 112;
168         h_m = m >> 13;
169     }
170     return (s << 15) | (h_e << 10) | h_m;
171 }
```

```
172
173 inline float half_to_float(uint16_t h) {
174     uint32_t s = (h >> 15) & 0x00000001;
175     uint32_t e = (h >> 10) & 0x0000001f;
176     uint32_t m = h & 0x000003ff;
177
178     if (e == 0) {
179         if (m == 0) {
180             uint32_t res = s << 31;
181             float f; memcpy(&f, &res, 4); return f;
182         } else {
183             while (!(m & 0x00000400)) { m <<= 1; e -= 1; }
184             e += 1; m &= ~0x00000400;
185         }
186     } else if (e == 31) {
187         if (m == 0) {
188              uint32_t res = (s << 31) | 0x7f800000;
189              float f; memcpy(&f, &res, 4); return f;
190         } else {
191              uint32_t res = (s << 31) | 0x7f800000 | (m << 13);
192              float f; memcpy(&f, &res, 4); return f;
193         }
194     }
195
196     e = e + (127 - 15);
197     m = m << 13;
198     uint32_t res = (s << 31) | (e << 23) | m;
199     float f; memcpy(&f, &res, 4); return f;
200 }
201
202 inline void init_32bit(void* A, void* B, void* C, int M, int N, int K) {
203     float* pA = (float*)A;
204     float* pB = (float*)B;
205     float* pC = (float*)C;
206
207     random_device rd;
208     mt19937 gen(rd());
209
210     uniform_real_distribution<float> dis(-1.0f, 1.0f);
211
212     for (int i = 0; i < M * K; ++i)
213         pA[i] = dis(gen);
214
215     for (int i = 0; i < K * N; ++i)
216         pB[i] = dis(gen);
217
218     for (int i = 0; i < M * N; ++i)
219         pC[i] = 0.0f;
220 }
221
222 inline void init_16bit(void* A, void* B, void* C, int M, int N, int K) {
223     uint16_t* pA = (uint16_t*)A;
224     uint16_t* pB = (uint16_t*)B;
225     uint16_t* pC = (uint16_t*)C;
226
227     random_device rd;
228     mt19937 gen(rd());
229     uniform_real_distribution<float> dis(-1.0f, 1.0f);
230
231     for (int i = 0; i < M * K; ++i) pA[i] = float_to_half(dis(gen));
232     for (int i = 0; i < K * N; ++i) pB[i] = float_to_half(dis(gen));
```

```
233     memset(pC, 0, M * N * sizeof(uint16_t));
234 }
235
236 inline void init_8bit(void* A, void* B, void* C, int M, int N, int K) {
237     int8_t* pA = (int8_t*)A;
238     int8_t* pB = (int8_t*)B;
239     int32_t* pC = (int32_t*)C; /* C 32 bit for the overflows */
240
241     random_device rd;
242     mt19937 gen(rd());
243
244     uniform_int_distribution<int> dis(-127, 127); /* 8bit maximum and
        minimum */
245
246     for (int i = 0; i < M * K; ++i)
247         pA[i] = (int8_t)dis(gen);
248
249     for (int i = 0; i < K * N; ++i)
250         pB[i] = (int8_t)dis(gen);
251
252     memset(pC, 0, M * N * sizeof(int32_t));
253 }
254
255 /* return the accellerator string name */
256 inline string get_acc_string(BT backend_type) {
257     string res;
258     switch (backend_type) {
259         case BT::CUDA:
260             res = "CUDA"; break;
261         case BT::SYCL:
262             res = "SYCL"; break;
263         case BT::OPENVINO:
264             res = "OPENVINO"; break;
265         case BT::OPENBLAS:
266             res = "OPENBLAS"; break;
267         default:
268             cerr << "[SCHEDULER] ERROR get_acc_string \n";
269             exit(EXIT_FAILURE);
270     }
271     return res;
272 }
273
274 #endif
```

Listing A.7: Task definitions (tasks.hpp)

```
1  #ifndef PROFILER_H
2  #define PROFILER_H
3
4  #include <cstdlib>
5  #include <iostream>
6  #include <iomanip>
7  #include <limits>
8  #include <memory>
9  #include <vector>
10 #include <string>
11 #include <fstream>
12 #include <filesystem>
```

```
13 #include <chrono>
14
15 #include "tasks.hpp"
16 #include "config.hpp"
17
18 #ifdef ENABLE_CUDA
19 #include <nvml.h>
20 #endif
21
22 /* total energy consumption in joules */
23 struct EnergyConsumption {
24     double cpu_pkg_joules = 0.0;
25     double cpu_core_joules = 0.0;
26     double nvidia_gpu_joules = 0.0;
27 };
28
29 struct WorkerMetrics {
30     std::chrono::time_point<std::chrono::high_resolution_clock> start_work;
31     std::chrono::time_point<std::chrono::high_resolution_clock> end_work;
32     std::chrono::time_point<std::chrono::high_resolution_clock> last_work;
33
34     double work = 0;
35     double total_idle = 0;
36     double total_work = 0;
37     int tasks_processed = 0;
38
39     void init_last_work() { last_work = std::chrono::high_resolution_clock
       ::now(); }
40     void start_worker() {
41         start_work = std::chrono::high_resolution_clock::now();
42         double idle_us = std::chrono::duration<double, std::micro>(
      start_work - last_work).count();
43         total_idle += idle_us;
44     }
45     void end_worker() {
46         end_work = std::chrono::high_resolution_clock::now();
47         double work_us = std::chrono::duration<double, std::micro>(end_work
       - start_work).count();
48         total_work += work_us;
49         tasks_processed++;
50         last_work = end_work;
51     }
52 };
53
54 struct Metric {
55     std::string name;
56     std::chrono::time_point<std::chrono::high_resolution_clock> start;
57     std::chrono::time_point<std::chrono::high_resolution_clock> stop;
58     double sum = 0.0;
59     double min = std::numeric_limits<double>::max();
60     double max = 0.0;
61     int count = 0;
62
63     double record() {
64         double res = std::chrono::duration<double, std::micro>(stop - start
      ).count();
65         sum += res;
66         if (res < min) min = res;
67         if (res > max) max = res;
68         count++;
69         return res;
```

```
70      }
71      void start_counter() { start = std::chrono::high_resolution_clock::now
        (); }
72      void stop_counter() { stop = std::chrono::high_resolution_clock::now();
         }
73      double avg() const { return count > 0 ? sum / count : 0.0; }
74  };
75
76  class Profiler {
77      Metric fetch{"Fetch"};
78      Metric logic{"Logic"};
79      Metric disp{"Dispatch"};
80
81      std::unique_ptr<WorkerMetrics[]> workers;
82
83      double total = 0;
84      int batch_count = 0;
85
86      bool power_monitoring_active = false;
87
88      /* gpu accumulator */
89      unsigned long long start_nvidia_mj = 0;
90
91      /* rapl start */
92      double start_pkg_uj = 0.0;
93      double start_core_uj = 0.0;
94
95      EnergyConsumption final_energy;
96
97      public:
98      Profiler() : workers(new WorkerMetrics[BT::COUNT]) {}
99
100     ~Profiler() {
101         if(power_monitoring_active) stop_power_monitor();
102     }
103
104     void start_power_monitor() {
105         if (power_monitoring_active) return;
106
107         final_energy = {0,0,0};
108
109        /* starting rapl read */
110 #ifdef ENABLE_INTEL_POWER_PROFILE
111         start_pkg_uj = read_rapl_energy_uj("intel-rapl:0");
112         start_core_uj = read_rapl_energy_uj("intel-rapl:0/intel-rapl:0:0");
113 #endif
114
115        /* starting nvml read */
116 #ifdef ENABLE_CUDA
117         nvmlReturn_t res = nvmlInit();
118         if (res == NVML_SUCCESS) {
119             nvmlDevice_t device;
120             nvmlDeviceGetHandleByIndex(0, &device);
121             unsigned long long energy = 0;
122             if (nvmlDeviceGetTotalEnergyConsumption(device, &energy) ==
    NVML_SUCCESS) {
123                 start_nvidia_mj = energy;
124             } else {
125                 start_nvidia_mj = 0;
126                 std::cerr << "[PROFILER] Nvidia Energy Counter not
    supported on this device.\n";
```

```
127                 exit(EXIT_FAILURE);
128             }
129         }
130 #endif
131         power_monitoring_active = true;
132     }
133 
134     void stop_power_monitor() {
135         if (!power_monitoring_active) return;
136 
137 #ifdef ENABLE_INTEL_POWER_PROFILE
138         double end_pkg_uj = read_rapl_energy_uj("intel-rapl:0");
139         double end_core_uj = read_rapl_energy_uj("intel-rapl:0/intel-rapl
    :0:0");
140 
141         if (end_pkg_uj >= start_pkg_uj)
142             final_energy.cpu_pkg_joules = (end_pkg_uj - start_pkg_uj) / 1e6
    ; // uJ to J
143 
144         if (end_core_uj >= start_core_uj)
145             final_energy.cpu_core_joules = (end_core_uj - start_core_uj) /
    1e6;
146 #endif
147 
148 #ifdef ENABLE_CUDA
149         unsigned long long end_nvidia_mj = 0;
150         nvmlDevice_t device;
151         if (nvmlDeviceGetHandleByIndex(0, &device) == NVML_SUCCESS) {
152             nvmlDeviceGetTotalEnergyConsumption(device, &end_nvidia_mj);
153         }
154 
155         if (end_nvidia_mj >= start_nvidia_mj && start_nvidia_mj > 0) {
156             final_energy.nvidia_gpu_joules = (double)(end_nvidia_mj -
    start_nvidia_mj) / 1000.0; // mJ to J
157         }
158 
159         nvmlShutdown();
160 #endif
161 
162         power_monitoring_active = false;
163     }
164 
165     void start_fetch() {fetch.start_counter();}
166     void start_logic() {logic.start_counter();}
167     void start_dispatch() {disp.start_counter();}
168     void stop_fetch() {fetch.stop_counter();}
169     void stop_logic() {logic.stop_counter();}
170     void stop_dispatch() {disp.stop_counter();}
171 
172     void init_last_work(size_t backend_type) { workers[backend_type].
    init_last_work(); }
173     void start_worker(size_t backend_type) { workers[backend_type].
    start_worker(); }
174     void end_worker(size_t backend_type) { workers[backend_type].end_worker
    (); }
175 
176     void record_sample() {
177         total += fetch.record() + logic.record() + disp.record();
178         batch_count++;
179     }
180 
```

```
181     void print_stats(const std::vector<BT>& bts, task* tasks, size_t
    num_tasks, std::string strategy_name = "ERROR") {
182         if (tasks == nullptr || num_tasks == 0) return;
183 
184         stop_power_monitor();
185 
186         double total_span_ms = 0, avg_lat_ms = 0, min_lat_ms = 0,
    max_lat_ms = 0;
187         calculate_latency_metrics(tasks, num_tasks, total_span_ms,
    avg_lat_ms, min_lat_ms, max_lat_ms);
188 
189         print_latency_info(tasks[0].M, tasks[0].N, tasks[0].K, num_tasks,
    total_span_ms, avg_lat_ms, min_lat_ms, max_lat_ms);
190 
191         print_energy_stats(total_span_ms);
192 
193 #ifdef ENABLE_PROFILING
194         print_scheduler_metrics(bts);
195 #endif
196 
197         save_to_csv(strategy_name, tasks[0].M, tasks[0].N, tasks[0].K,
    num_tasks,
198                     total_span_ms, avg_lat_ms, min_lat_ms, max_lat_ms, bts,
    final_energy);
199     }
200 
201     void print_stats(const std::vector<BT>& bts, const std::vector<task*>&
    sub_tasks, std::string strategy_name = "ERROR") {
202         if (sub_tasks.empty()) return;
203         std::vector<task> temp_tasks;
204         temp_tasks.reserve(sub_tasks.size());
205         for (auto* t : sub_tasks) temp_tasks.push_back(*t);
206         print_stats(bts, temp_tasks.data(), temp_tasks.size(),
    strategy_name);
207     }
208 
209     private:
210     double read_rapl_energy_uj(const std::string &domain) {
211         std::ifstream file("/sys/class/powercap/" + domain + "/energy_uj");
212         double value = 0;
213         if (file >> value) return value;
214         return 0;
215     }
216 
217     void calculate_latency_metrics(task* tasks, size_t num_tasks, double&
    total_span, double& avg, double& min, double& max) {
218         if (num_tasks == 0) return;
219         auto global_start = tasks[0].start_time;
220         auto global_end = tasks[0].end_time;
221         double sum = 0; min = std::numeric_limits<double>::max(); max = 0;
222 
223         for (size_t i=0; i<num_tasks; i++) {
224             if (tasks[i].start_time < global_start) global_start = tasks[i
    ].start_time;
225             if (tasks[i].end_time > global_end) global_end = tasks[i].
    end_time;
226             double curr = std::chrono::duration<double, std::milli>(tasks[i
    ].end_time - tasks[i].start_time).count();
227             sum += curr;
228             if (curr < min) min = curr;
229             if (curr > max) max = curr;
```

```
230         }
231         total_span = std::chrono::duration<double, std::milli>(global_end -
     global_start).count();
232         avg = sum / num_tasks;
233     }
234 
235     void print_latency_info(int M, int N, int K, int num, double total,
    double avg, double min, double max) {
236         std::cout << "==================================\n";
237         std::cout << "Matrix: " << M << "x" << N << "x" << K << " | Tasks:
    " << num << "\n";
238         std::cout << "Total Time: " << total << " ms\n";
239         std::cout << "Latency (ms): Avg " << avg << " | Min " << min << " |
     Max " << max << "\n";
240     }
241 
242     void print_energy_stats(double total_ms) {
243         double seconds = total_ms / 1000.0;
244         if (seconds <= 0.0) seconds = 1.0;
245 
246         std::cout << "\nPower consumption: \nDuration: " << seconds << " s\
    n";
247         std::cout << std::fixed << std::setprecision(2);
248 
249 #ifdef ENABLE_INTEL_POWER_PROFILE
250         double pkg_w = final_energy.cpu_pkg_joules / seconds;
251         double core_w = final_energy.cpu_core_joules / seconds;
252 
253         std::cout << "cpu package: " << std::setw(8) << final_energy.
    cpu_pkg_joules << " J  (" << pkg_w << " W avg)\n";
254         std::cout << "cpu core:    " << std::setw(8) << final_energy.
    cpu_core_joules << " J  (" << core_w << " W avg)\n";
255 #endif
256 
257 #ifdef ENABLE_CUDA
258         double nv_w = final_energy.nvidia_gpu_joules / seconds;
259         std::cout << "cuda gpu:    " << std::setw(8) << final_energy.
    nvidia_gpu_joules << " J  (" << nv_w << " W avg)\n";
260 #endif
261         double total_j = final_energy.cpu_pkg_joules + final_energy.
    nvidia_gpu_joules;
262         std::cout << "total:    " << std::setw(8) << total_j << " J\n";
263 
264     }
265 
266     void print_scheduler_metrics(const std::vector<BT>& bts) {
267         if (batch_count == 0) return;
268 
269         std::cout << "\nScheduler: " << batch_count << " batches\n";
270         print_metric(fetch);
271         print_metric(logic);
272         print_metric(disp);
273         std::cout << std::left << std::setw(10) << "Total avg overhead: "
    << total/batch_count << " us  Total overhead: " << total << " us\n";
274 
275         std::cout << "\nWorkers:\n";
276 
277         for (BT i : bts) {
278             const auto& w = workers[i];
279 
280             double total_time_us = w.total_work + w.total_idle;
```

```
281             double occupancy = (total_time_us > 0) ? (w.total_work /
    total_time_us) * 100.0 : 0.0;
282
283             double avg_idle_us = (w.tasks_processed > 0) ? w.total_idle / w
    .tasks_processed : 0.0;
284             double avg_work_us = (w.tasks_processed > 0) ? w.total_work / w
    .tasks_processed : 0.0;
285
286             double total_idle_ms = w.total_idle / 1000.0;
287             double total_work_ms = w.total_work / 1000.0;
288             double total_time_ms = total_time_us / 1000.0;
289
290             std::string acc_str = get_acc_string(i);
291             std::cout << "[" << acc_str << "]" << "\n";
292             std::cout << "  tasks: " << w.tasks_processed << "\n";
293             std::cout << "  occupancy:          " << std::fixed << std::
    setprecision(2) << occupancy << " %\n";
294             std::cout << "  total idle time: " << total_idle_ms << " ms,
    Avg idle time:   " << avg_idle_us << " us\n";
295             std::cout << "  total work time: " << total_work_ms << " ms,
    Avg work time:   " << avg_work_us << " us\n";
296             std::cout << "  total time:       " << total_time_ms << " ms\n"
    ;
297         }
298     }
299
300     void print_metric(const Metric& m) {
301         std::cout << std::left << std::setw(10) << m.name
302             << " | avg: " << std::setw(6) << std::fixed << std::
    setprecision(2) << m.avg() << " us"
303             << " | min: " << std::setw(6) << m.min << " us"
304             << " | max: " << std::setw(6) << m.max << " us\n";
305     }
306
307     void save_to_csv(std::string strategy, int M, int N, int K, int
    num_tasks,
308             double total_ms, double avg_ms, double min_ms, double max_ms,
309             const std::vector<BT>& bts, const EnergyConsumption& en) {
310
311         std::string filename = csvname;
312
313         std::filesystem::path path(filename);
314         if (path.has_parent_path() && !std::filesystem::exists(path.
    parent_path())) {
315             std::filesystem::create_directories(path.parent_path());
316         }
317
318         bool file_exists = std::filesystem::exists(filename);
319         std::ofstream file(filename, std::ios::app); /* append mode */
320
321         if (!file.is_open()) {
322             std::cerr << "[PROFILER] Error opening CSV file: " << filename
    << std::endl;
323             return;
324         }
325
326         double seconds = total_ms / 1000.0;
327         if (seconds <= 0) seconds = 1.0;
328
329         double pkg_w = en.cpu_pkg_joules / seconds;
330         double core_w = en.cpu_core_joules / seconds;
```

```
331         double nv_w = en.nvidia_gpu_joules / seconds;
332 
333         if (!file_exists || std::filesystem::file_size(filename) == 0) {
334             /* global state */
335             file << "Strategy,M,N,K,Num_Tasks,Total_Time_ms,Total_Time_s,
    Avg_Lat_ms,Min_Lat_ms,Max_Lat_ms";
336 
337             /* power stats */
338             file << ",Pkg_J,Pkg_Avg_W,Core_J,Core_Avg_W,NVGPU_J,NVGPU_Avg_W
    ";
339 
340             /* scheduler metrics */
341             file << ",Sched_Avg_Fetch_us,Sched_Avg_Logic_us,
    Sched_Avg_Disp_us";
342 
343             /* worker metrics */
344             for (int i = 0; i < BT::COUNT; ++i) {
345                 if (i == BT::CORDINATOR) {continue;}
346                 std::string name = get_acc_string((BT)i);
347                 file << "," << name << "_Tasks"
348                     << "," << name << "_Occupancy_%"
349                     << "," << name << "_Tot_Idle_ms"
350                     << "," << name << "_Tot_Work_ms"
351                     << "," << name << "_Avg_Idle_us"
352                     << "," << name << "_Avg_Work_us"
353                     << "," << name << "_Tot_Time_ms";
354             }
355             file << "\n";
356         }
357 
358         /* global stats */
359         file << strategy << "," << M << "," << N << "," << K << "," <<
    num_tasks << ","
360             << total_ms << "," << std::fixed << std::setprecision(3) <<
    seconds << ","
361             << avg_ms << "," << min_ms << "," << max_ms;
362 
363         /* power stats */
364 #ifdef ENABLE_INTEL_POWER_PROFILE
365         file << "," << en.cpu_pkg_joules << "," << pkg_w
366             << "," << en.cpu_core_joules << "," << core_w;
367 #else
368         file << ",0,0,0,0";
369 #endif
370 
371 #ifdef ENABLE_CUDA
372         file << "," << en.nvidia_gpu_joules << "," << nv_w;
373 #else
374         file << ",0,0";
375 #endif
376 
377         /* scheduler metrics */
378         file << "," << fetch.avg() << "," << logic.avg() << "," << disp.avg
    ();
379 
380         /* workers metrics */
381         for (int i = 0; i < BT::COUNT; ++i) {
382             if (i == BT::CORDINATOR) {continue;}
383             bool is_active = false;
384             for(auto b : bts) { if((int)b == i) is_active = true; }
385 
```

```
386             if (is_active) {
387                 const auto& w = workers[i];
388                 double total_time_us = w.total_work + w.total_idle;
389                 double occupancy = (total_time_us > 0) ? (w.total_work /
    total_time_us) * 100.0 : 0.0;
390                 double avg_idle_us = (w.tasks_processed > 0) ? w.total_idle
     / w.tasks_processed : 0.0;
391                 double avg_work_us = (w.tasks_processed > 0) ? w.total_work
     / w.tasks_processed : 0.0;
392                 double total_idle_ms = w.total_idle / 1000.0;
393                 double total_work_ms = w.total_work / 1000.0;
394                 double total_time_ms = total_time_us / 1000.0;
395 
396                 file << "," << w.tasks_processed
397                     << "," << occupancy
398                     << "," << total_idle_ms
399                     << "," << total_work_ms
400                     << "," << avg_idle_us
401                     << "," << avg_work_us
402                     << "," << total_time_ms;
403             } else {
404                 file << ",0,0,0,0,0,0,0,0";
405             }
406         }
407         file << "\n";
408         file.close();
409     }
410 };
411 
412 inline void print_performance_stats(task* tasks, size_t num_tasks) {
413 
414     if (tasks == nullptr || num_tasks == 0) {
415         std::cout << "[ERROR] print_performance_stats\n";
416         exit(EXIT_FAILURE);
417     }
418 
419     auto min_start = tasks[0].start_time;
420     auto max_end = tasks[0].end_time;
421     int M = tasks[0].M;
422     int N = tasks[0].N;
423     int K = tasks[0].K;
424 
425     std::chrono::duration<double, std::milli> first_time = tasks[0].
    end_time - tasks[0].start_time;
426     double min_latency_ms = first_time.count();
427     double max_latency_ms = first_time.count();
428     double total_latency_sum_ms = 0.0;
429 
430     for (size_t i=0; i<num_tasks; i++) {
431 
432         if (tasks[i].start_time < min_start)
433             min_start = tasks[i].start_time;
434 
435         if (tasks[i].end_time > max_end)
436             max_end = tasks[i].end_time;
437 
438         std::chrono::duration<double, std::milli> duration = tasks[i].
    end_time - tasks[i].start_time;
439         double curr_latency_ms = duration.count();
440 
441         total_latency_sum_ms += curr_latency_ms;
```

```
442
443         if (curr_latency_ms < min_latency_ms)
444             min_latency_ms = curr_latency_ms;
445
446         if (curr_latency_ms > max_latency_ms)
447             max_latency_ms = curr_latency_ms;
448     }
449
450     std::chrono::duration<double, std::milli> global_span = max_end -
    min_start;
451     double average_latency_ms = total_latency_sum_ms / num_tasks;
452     std::cout << "================================\n";
453     std::cout << "N Matrix: " << num_tasks << "\n";
454     std::cout << "M : " << M  << " N : " << N << " K : " << K << "\n";
455     std::cout << "\nAvg latency:      " << average_latency_ms << " ms\n";
456     std::cout << "Min latency:      " << min_latency_ms << " ms\n";
457     std::cout << "Max latency:      " << max_latency_ms << " ms\n";
458     std::cout << "\nTotal ms:         " << global_span.count() << " ms\n";
459     std::cout << "\n";
460 }
461 #endif
```

Listing A.8: profiler.hpp

```
1  #ifndef PERFORMANCEMAP_H
2  #define PERFORMANCEMAP_H
3
4  #include "tasks.hpp"
5  #include "config.hpp"
6
7  #include <iostream>
8  #include <tuple>
9  #include <unordered_map>
10 #include <unordered_set> /* for jit times */
11 #include <fstream>
12 #include <vector>
13 #include <string>
14
15 #include <cstdlib>
16 #include <string>
17 #include <sstream>
18 #include <vector>
19 #include <iostream>
20 #include <fstream>
21
22 using namespace std;
23
24 inline vector<string> split(const string &s, char delimiter) {
25     vector<string> tokens;
26     string token;
27     istringstream tokenStream(s);
28
29     while (std::getline(tokenStream, token, delimiter)) {
30         tokens.push_back(token);
31
32     }
33     return tokens;
34 }
```

```
35
36 /* 1048576 limit for each M or N or K */
37 class PerformanceMap {
38
39     struct map_struct {
40         unordered_map<unsigned long long, tuple<double, double>> grid_map;
41
42         long long last_M = -1;
43         long long last_N = -1;
44         long long last_K = -1;
45         double last_result = -1.0;
46         double max_result = -1.0;
47     };
48
49     map_struct map_f;
50     map_struct map_h;
51
52     unordered_set<unsigned long long> jit_cache_f;
53     unordered_set<unsigned long long> jit_cache_h;
54
55     /* which accellerator am i */
56     BT bt;
57
58     long long STEP_SIZE;
59
60 private:
61
62     /* return the successor of val in our step_size */
63     long long round_up(long long val) {
64         if (val == 0) return STEP_SIZE;
65         return ((val + STEP_SIZE - 1) / STEP_SIZE) * STEP_SIZE;
66     }
67
68     /* return the key of val in our step_size */
69     unsigned long long get_key(long long M, long long N, long long K) {
70         long long rM = round_up(M);
71         long long rN = round_up(N);
72         long long rK = round_up(K);
73
74         /* 64 bit  |4bit nil|20bit M|20bit N|20bit K| */
75         return ((unsigned long long)rM << 40) | ((unsigned long long)rN <<
    20) | (unsigned long long)rK;
76     }
77
78     void add_key(long long M, long long N, long long K, double time, double
     jit_time, Type type) {
79         unsigned long long key = get_key(M, N, K);
80         if (type == Type::FLOAT){
81            map_f.grid_map[key] = std::make_tuple(time, jit_time);
82            map_f.max_result = (map_f.max_result > time ) ? map_f.max_result
     : time;
83         }
84         if (type == Type::HALF){
85            map_h.grid_map[key] = std::make_tuple(time, jit_time);
86            map_h.max_result = (map_h.max_result > time ) ? map_h.max_result
     : time;
87         }
88     }
89
90     void add_keys(ifstream* file, Type type) {
91         string line;
```

```
92          getline(*file, line);
93          while (std::getline(*file, line)) {
94              vector<std::string> row = split(line, ',');
95              if (row.size() < 6) continue;
96
97              int M = stod(row[2]);
98              int N = stod(row[3]);
99              int K = stod(row[4]);
100             double time = stod(row[5]);
101             double jit_time = stod(row[6]);
102
103             add_key(M, N, K, time, jit_time, type);
104         }
105     }
106
107     /* open file and init the map */
108     void init_maps(string filename_f, string filename_h) {
109         ifstream file_f(filename_f);
110         ifstream file_h(filename_h);
111
112         if (!file_f.is_open()) {
113             cerr << "[PerformanceMap]: ERROR open file" << filename_f <<" \
    n";
114             exit(EXIT_FAILURE);
115         }
116
117         if (!file_h.is_open()) {
118             cerr << "[PerformanceMap]: ERROR open file" << filename_h <<" \
    n";
119             exit(EXIT_FAILURE);
120         }
121
122         add_keys(&file_f,Type::FLOAT);
123         add_keys(&file_h,Type::HALF);
124     }
125
126 public:
127
128     PerformanceMap(int step_size, BT bt, string filename_f, string
    filename_h) : STEP_SIZE(step_size), bt(bt) {
129         init_maps(filename_f, filename_h);
130     }
131
132     /* return the time, add_jit true = populate the jit cache */
133     double query(long long M, long long N, long long K, Type type, bool
    add_jit) {
134         map_struct* data = nullptr;
135         unordered_set<unsigned long long>* jit_cache = nullptr;
136         double time_ms, jit_ms;
137
138         unsigned long long key = get_key(M, N, K);
139
140         if (type == Type::FLOAT) {
141             jit_cache = &jit_cache_f;
142             data = &map_f;
143         } else {
144             jit_cache = &jit_cache_h;
145             data = &map_h;
146         }
147
148         /* we cache the last results */
```

```
149         if (M == data->last_M && N == data->last_N && K == data->last_K) {
    return data->last_K;}
150
151         data->last_M = M;
152         data->last_N = N;
153         data->last_K = K;
154
155         /* return if we find the key */
156         if (data->grid_map.find(key) != data->grid_map.end()) {
157             time_ms = get<0>(data->grid_map[key]);
158             jit_ms = get<1>(data->grid_map[key]);
159
160             /* add jit_ms only if openvino never see M N K*/
161             if (bt == BT::OPENVINO) {
162
163                 /* mitigate overhead when other accellerator are running */
164                 //time_ms += time_ms * 0.1;
165
166                 if (jit_cache->find(key) == jit_cache->end()){
167                     if (add_jit)
168                         jit_cache->insert(key);
169
170                     time_ms += jit_ms;
171                 }
172             }
173
174             /* add jit_ms only if sycl never see N */
175             if (bt == BT::SYCL) {
176
177                 /* mitigate overhead when other accellerator are running */
178                 //time_ms += time_ms * 0.1;
179
180                 unsigned long long sycl_key = (unsigned long long) N;
181                 if (jit_cache->find(sycl_key) == jit_cache->end()){
182
183                     if (add_jit)
184                         jit_cache->insert(sycl_key);
185
186                     time_ms += JIT_MS_SYCL;
187                 }
188             }
189
190             /* mitigate openblas error in bencharks  */
191             if (bt == BT::OPENBLAS)
192                 time_ms = time_ms * 0.8;
193
194             data->last_K = time_ms;
195             return time_ms;
196         }
197
198         /* if we don't have data on this matrix, just return max time * 2
    */
199         time_ms = data->max_result * 2;
200
201         /* add jit expences */
202         if (bt == BT::SYCL || bt == BT::OPENVINO) {
203             time_ms += JIT_MS_SYCL;
204         }
205
206         /* cache the result */
207         data->last_K = time_ms;
```

```
208
209         return time_ms;
210     }
211 };
212 #endif
```

Listing A.9: performancemap.hpp

```
1  #ifndef CPUGEM_H
2  #define CPUGEM_H
3
4  #include "tasks.hpp"
5  #include "config.hpp"
6  #include <cblas.h>
7  #include <cstdlib>
8  #include <iostream>
9  #include <vector>
10 #include <cstdint>
11 #include <omp.h>
12
13 inline void pin_to_cores(const std::vector<int>& core_ids) {
14 }
15
16 inline void cpu_init() {
17
18     std::vector<int> e_cores = {3,4};
19
20     cpu_set_t cpu;
21     CPU_ZERO(&cpu);
22
23     for (int id : e_cores) {
24         CPU_SET(id, &cpu);
25     }
26
27     pthread_t current_thread = pthread_self();
28     int rc = pthread_setaffinity_np(current_thread, sizeof(cpu_set_t), &cpu
       );
29
30     if (rc != 0) {
31         std::cerr << "[ERROR] Openblas Affinity CPU: " << rc << "\n";
32         exit(EXIT_FAILURE);
33     }
34
35     openblas_set_num_threads(N_CORES);
36     omp_set_num_threads(N_CORES);
37 }
38
39 inline void cpu_gemm_32bit(void* A, void* B, void* C, int M, int N, int K)
       {
40     float* A_ = (float*) A;
41     float* B_ = (float*) B;
42     float* C_ = (float*) C;
43
44     cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
45                 M, N, K,
46                 1.0f,
47                 A_, K,
48                 B_, N,
```

```
                0.0f,
                C_, N);
}

inline void cpu_gemm_16bit(void* A, void* B, void* C, int M, int N, int K)
    {
    uint16_t* pA = (uint16_t*)A;
    uint16_t* pB = (uint16_t*)B;
    uint16_t* pC = (uint16_t*)C;

    std::vector<float> A_f32(M*K);
    std::vector<float> B_f32(K*N);
    std::vector<float> C_f32(M*N);

    /* large overhead but we need it, not all cpus have 16 bit */
    #pragma omp parallel for
    for (int i = 0; i < M * K; ++i) {
        A_f32[i] = half_to_float(pA[i]);
    }

    for (int i = 0; i < K * N; ++i) {
        B_f32[i] = half_to_float(pB[i]);
    }

    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                M, N, K,
                1.0f,
                A_f32.data(), K,
                B_f32.data(), N,
                0.0f,
                C_f32.data(), N);

    #pragma omp parallel for
    for (int i = 0; i < M * N; ++i) {
        pC[i] = float_to_half(C_f32[i]);
    }
}

#endif
```

Listing A.10: OpenBlass Implementation (cpu.hpp)

```
#ifndef SLEEPBUFFER_H
#define SLEEPBUFFER_H

#include <memory>
#include <unistd.h>
#include <atomic>
#include "tasks.hpp"
#include "config.hpp"

using namespace std;

/* basic implementation of a simple ring buffer with blocking function */
class SharedBuffer {
    private:
        unique_ptr<task*[]> data;
        size_t size = 0;
```

```
17
18          alignas(64) std::atomic<size_t> write_i{0};
19          alignas(64) std::atomic<size_t> read_i{0};
20
21      public:
22          /* constructor */
23          SharedBuffer(size_t s) : size(s), data(new task*[s]) {
24          }
25
26          /* destructur */
27          ~SharedBuffer() {
28          }
29
30          /* put one task pointer inside, sleep if full*/
31          void put(task* ele) {
32
33  #ifdef SLEEP
34              while (((write_i + 1) % size) == read_i) {
35                  usleep(PUT_SLEEP);
36              }
37  #else
38              while (((write_i + 1) % size) == read_i) {}
39  #endif
40              data[write_i] = ele;
41              write_i = (write_i + 1) % size;
42          }
43
44          /* get one task pointer sleep if empty, return to the caller after
45           * 5 attempt*/
46          task* get() {
47              int c = 0;
48
49  #ifdef SLEEP
50              while (write_i == read_i){
51                  c++;
52                  usleep(GET_SLEEP);
53                  if (c >= 5) {return nullptr;}
54              }
55  #else
56              while (write_i == read_i){
57                  c++;
58                  if (c >= 50) {return nullptr;}
59              }
60  #endif
61              task* ele = data[read_i];
62              read_i = (read_i + 1) % size;
63
64              return ele;
65          }
66
67          bool is_empty() {
68              if (write_i == read_i)
69                  return true;
70              else
71                  return false;
72          }
73  };
74  #endif
```

Listing A.11: sharedbuffer.hpp

```
#ifndef CONFIG_H
#define CONFIG_H

#include <string>

/********** debug ************/
//#define DEBUG
#define ENABLE_PROFILING           /* profiler for timing the scheduler */
#define ENABLE_INTEL_POWER_PROFILE /* profiling the power consumption with
    intel rapl */

/********** tests ************/
#define M_ 4096
#define N_ 4096
#define K_ 4096
#define N_MATRIX 100

inline int BATCH_SIZE = 30;
inline int BATCH_SIZE_HETERO = 40;
inline std::string csvname = "bin/csv/results.csv";

#define M_split 14000 /* big matrix split size, max is MAX_SIZE/4 */
#define N_split 4000
#define K_split 4000

/********** scheduler ************/
#define MAX_SIZE 4092

#define JIT_MS_SYCL 50  /* mitigate jit time for sycl */

//enum {BATCH_SIZE = 50};                  /* static partitioning batch
    size */
//enum {BATCH_SIZE_HETERO = 40};
enum {SPLIT_MATRIX_ITERATION = 3};

/********** buffers ************/
//#define SLEEP
enum {GET_SLEEP = 500};        /* shared buffer sleep time */
enum {PUT_SLEEP = 500};        /* 5 ms */
enum {BUFFER_LENGHT = 2048};  /* shared buffer length */

/********** benchmark ************/
enum {STEP_SIZE = 512};        /* benchmark for csv step size */
enum {STEP_TOTAL = 8};        /* STEP_SIZE * STEP_TOTAL */

/********** CPU ************/
enum {N_CORES = 2};        /* number of cores for cpu gemm */

#endif
```

Listing A.12: config.hpp

```
#ifndef TEST_H
#define TEST_H

/* this is only for testing pourpuse of the dynamic libraries */
#include "tasks.hpp"
#include "profiler.hpp"
```

```
7  #include <cstdlib>
8  #include <iostream>
9  #include <vector>
10 #include <cmath>
11 #include <iomanip>
12 #include <cstdint>
13 #include <stdlib.h>
14 #include <cstring>
15
16 using namespace std;
17
18 #include <iostream>
19 #include "scheduler.hpp"
20
21 #ifdef ENABLE_OPENVINO
22 #include "ov_wrapper.h"
23 #endif
24
25 #ifdef ENABLE_CUDA
26 #include "cuda_wrapper.h"
27 #endif
28
29 #ifdef ENABLE_SYCL
30 #include "sycl_wrapper.h"
31 #endif
32
33 #ifdef ENABLE_OPENBLAS
34 #include "cpu.hpp"
35 #endif
36
37
38 /***************************** Helpers ******************************/
39
40 inline bool compare_cpu_32bit(void* A_, void* B_, void* C_, int M, int N,
       int K) {
41     float *A = (float*)A_;
42     float *B = (float*)B_;
43     float *C = (float*)C_;
44     vector<float> C_cpu(M * N, 0.0f);
45
46     /* C = A * B */
47     for (int m = 0; m < M; ++m)
48         for (int n = 0; n < N; ++n) {
49             float sum = 0.0f;
50             for (int k = 0; k < K; ++k) {
51                 sum += A[m * K + k] * B[k * N + n];
52             }
53             C_cpu[m * N + n] = sum;
54         }
55
56     /* epsilon because the NPU will intruduce slightly differnce errors in
       the float mul */
57     const float epsilon = 0.03f;; /* NPU fault */
58     bool match = true;
59     float max_diff = 0.0f;
60
61     for (int i = 0; i < M * N; ++i) {
62         float diff = fabs(C_cpu[i] - C[i]);
63         if (diff > max_diff) max_diff = diff;
64
65         /* error */
```

```
66          if (diff > epsilon) {
67              match = false;
68          }
69      }
70
71      /* we print the last 4 element of the matrix */
72      if (!match) {
73          cout << "\n[TESTS] ERROR Mismatch! Max Diff: " << max_diff << "\n";
74
75          int start_col = (N > 4) ? N - 4 : 0;
76
77          cout << "[TESTS] last 4 elem CPU (Reference): ";
78          for (int n = start_col; n < N; ++n) {
79              cout << fixed << setprecision(4) << C_cpu[(M - 1) * N + n] << "
     ";
80          }
81          cout << "\n";
82
83          cout << "[TESTS] last 4 elem NPU (Input C):   ";
84          for (int n = start_col; n < N; ++n) {
85              cout << fixed << setprecision(4) << C[(M - 1) * N + n] << " ";
86          }
87          cout << "\n" << endl;
88      }
89      else
90          PRINT("[TESTS] " <<  M << " " << N << " " << K << " PASSED \n");
91
92      return match;
93  }
94
95  inline bool compare_cpu_16bit(void* A, void* B, void* C, int M, int N, int
      K) {
96      uint16_t* pA = (uint16_t*)A;
97      uint16_t* pB = (uint16_t*)B;
98      uint16_t* pC = (uint16_t*)C;
99
100     const float epsilon = 0.5f;
101     bool match = true;
102     float max_diff = 0.0f;
103
104     for (int m = 0; m < M; ++m) {
105         for (int n = 0; n < N; ++n) {
106             float sum = 0.0f;
107             for (int k = 0; k < K; ++k) {
108                 sum += half_to_float(pA[m * K + k]) * half_to_float(pB[k *
      N + n]);
109             }
110
111             float gpu_val = half_to_float(pC[m * N + n]);
112             float diff = fabs(sum - gpu_val);
113
114             if (diff > max_diff) max_diff = diff;
115
116             if (diff > epsilon && diff > (fabs(sum) * 0.05f)) { // 5%
      tolleranza relativa
117                 match = false;
118             }
119         }
120     }
121
122     if (!match) cout << "\n[TESTS-16] ERROR Mismatch! Max Diff: " <<
```

```
    max_diff << "\n";
123     else PRINT("[TESTS-16] PASSED\n");
124 
125     return match;
126 }
127 
128 inline bool compare_cpu_8bit(void* A, void* B, void* C, int M, int N, int K
    ) {
129     int8_t* pA = (int8_t*)A;
130     int8_t* pB = (int8_t*)B;
131     int32_t* pC = (int32_t*)C; /* output is 32 bit */
132 
133     bool match = true;
134     int32_t max_diff = 0;
135 
136     for (int m = 0; m < M; ++m) {
137         for (int n = 0; n < N; ++n) {
138             int32_t sum = 0;
139             for (int k = 0; k < K; ++k) {
140                 sum += (int32_t)pA[m * K + k] * (int32_t)pB[k * N + n];
141             }
142 
143             int32_t diff = abs(sum - pC[m * N + n]);
144             if (diff > max_diff) max_diff = diff;
145 
146             if (diff != 0) { /* diff 0 for 32 bit output beacause we don't
    lose precision */
147                 match = false;
148             }
149         }
150     }
151 
152     if (!match) cout << "\n[TESTS-08] ERROR Mismatch! Max Diff: " <<
    max_diff << " (Should be 0)\n";
153     else PRINT("[TESTS-08] PASSED\n");
154 
155     return match;
156 }
157 /******************************** Test Accellerators
    **********************************/
158 
159 inline void test_accellerators() {
160     int M = 512;
161     int N = 512;
162     int K = 256;
163 
164     /* ------------------- 32 BIT TEST ------------------- */
165     cout << "\n--- Running 32-bit Tests ---\n";
166     float *A = new float[M*K];
167     float *B = new float[K*N];
168     float *C = new float[M*N];
169 
170     init_32bit(A,B,C,M,N,K);
171 
172     #ifdef ENABLE_OPENVINO
173         ov_init();
174         ov_gemm_32bit(A,B,C,M,N,K);
175         if(!compare_cpu_32bit(A,B,C,M,N,K)) { cout << "OpenVINO 32bit Fail\
    n"; exit(1); }
176         cout << "--- OpenVino Passed\n";
177     #endif
```

```
178
179     #ifdef ENABLE_CUDA
180         cuda_init();
181         cuda_gemm_32bit(A,B,C,M,N,K);
182         if(!compare_cpu_32bit(A,B,C,M,N,K)) { cout << "CUDA 32bit Fail\n";
    exit(1); }
183         cout << "--- Cuda Passed\n";
184     #endif
185
186     #ifdef ENABLE_SYCL
187         sycl_init();
188         sycl_gemm_32bit(A,B,C,M,N,K);
189         if(!compare_cpu_32bit(A,B,C,M,N,K)) { cout << "SYCL 32bit Fail\n";
    exit(1); }
190         cout << "--- Sycl Passed\n";
191     #endif
192
193     #ifdef ENABLE_OPENBLAS
194         cpu_init();
195         cpu_gemm_32bit(A,B,C,M,N,K);
196         if(!compare_cpu_32bit(A,B,C,M,N,K)) { cout << "SYCL 32bit Fail\n";
    exit(1); }
197         cout << "--- OpenBlas Passed\n";
198     #endif
199
200     delete[] A; delete[] B; delete[] C;
201     cout << "--- PASSED 32-bit Tests ---\n";
202
203     /* ------------------- 16 BIT TEST ------------------- */
204     cout << "\n--- Running 16-bit FP16 Tests ---\n";
205     uint16_t *A16 = new uint16_t[M*K];
206     uint16_t *B16 = new uint16_t[K*N];
207     uint16_t *C16 = new uint16_t[M*N];
208
209     init_16bit(A16, B16, C16, M, N, K);
210
211     #ifdef ENABLE_OPENVINO
212         ov_gemm_16bit(A16, B16, C16, M, N, K);
213         if(!compare_cpu_16bit(A16, B16, C16, M, N, K)) { cout << "OpenVINO
    16bit Fail\n"; exit(1); }
214         cout << "--- OpenVino Passed\n";
215     #endif
216
217     #ifdef ENABLE_CUDA
218         cuda_gemm_16bit(A16, B16, C16, M, N, K);
219         if(!compare_cpu_16bit(A16, B16, C16, M, N, K)) { cout << "CUDA 16
    bit Fail\n"; exit(1); }
220         cout << "--- Cuda Passed\n";
221     #endif
222
223     #ifdef ENABLE_SYCL
224         sycl_gemm_16bit(A16, B16, C16, M, N, K);
225         if(!compare_cpu_16bit(A16, B16, C16, M, N, K)) { cout << "SYCL 16
    bit Fail\n"; exit(1); }
226         cout << "--- Sycl Passed\n";
227     #endif
228
229     #ifdef ENABLE_OPENBLAS
230         cpu_gemm_16bit(A16, B16, C16, M, N, K);
231         if(!compare_cpu_16bit(A16, B16, C16, M, N, K)) { cout << "CPU 16bit
     Fail\n"; exit(1); }
```

```
232         cout << "--- OpenBlas Passed\n";
233     #endif
234 
235     delete[] A16; delete[] B16; delete[] C16;
236 
237     cout << "--- PASSED 16-bit Tests ---\n";
238 
239     /* ------------------- 8 BIT TEST ------------------- */
240     /*
241     cout << "\n--- Running 8-bit INT8 Tests ---\n";
242      Input 1 byte, Output 4 byte (int32)
243     int8_t *A8 = new int8_t[M*K];
244     int8_t *B8 = new int8_t[K*N];
245     int32_t *C8 = new int32_t[M*N];
246 
247     init_8bit(A8, B8, C8, M, N, K);
248 
249     #ifdef ENABLE_OPENVINO
250         ov_gemm_8bit(A8, B8, C8, M, N, K);
251         if(!compare_cpu_8bit(A8, B8, C8, M, N, K)) { cout << "OpenVINO 8bit
     Fail\n"; exit(1); }
252         ov_free();
253     #endif
254 
255     #ifdef ENABLE_CUDA
256         cuda_gemm_8bit(A8, B8, C8, M, N, K);
257         if(!compare_cpu_8bit(A8, B8, C8, M, N, K)) { cout << "CUDA 8bit
    Fail\n"; exit(1); }
258         cuda_free();
259     #endif
260 
261     #ifdef ENABLE_SYCL
262         sycl_gemm_8bit(A8, B8, C8, M, N, K);
263         if(!compare_cpu_8bit(A8, B8, C8, M, N, K)) { cout << "SYCL 8bit
    Fail\n"; exit(1); }
264         sycl_free();
265     #endif
266 
267     delete[] A8; delete[] B8; delete[] C8;
268     */
269 
270     cout << "\n[ALL TESTS PASSED]\n" << endl;
271 }
272 /***************************** helper test for main
    ***************************/
273 
274 inline void test_compare_task(task* array_task, size_t n_task) {
275     Type t = array_task->type;
276 
277     if (t == Type::FLOAT)
278         for (int i = 0; i < n_task; i++)
279             compare_cpu_32bit(array_task[i].A, array_task[i].B, array_task[
    i].C, array_task[i].M, array_task[i].N, array_task[i].K);
280 
281     if (t == Type::HALF)
282         for (int i = 0; i < n_task; i++)
283             compare_cpu_16bit(array_task[i].A, array_task[i].B, array_task[
    i].C, array_task[i].M, array_task[i].N, array_task[i].K);
284 
285     if (t == Type::UINT8)
286         for (int i = 0; i < n_task; i++)
```

```
287             compare_cpu_8bit(array_task[i].A, array_task[i].B, array_task[i
    ].C, array_task[i].M, array_task[i].N, array_task[i].K);
288 }
289
290 #ifdef ENABLE_CUDA
291 inline void test_cuda_streaming(int M, int N, int K, size_t num_task, Type
    type) {
292     cout << "\nCuda only \n";
293     task* tasks = init_tasks(num_task, M, N, K, type);
294     cuda_init();
295
296     for (int i=0; i<num_task; i++) {
297         void* A = tasks[i].A;
298         void* B = tasks[i].B;
299         void* C = tasks[i].C;
300         int M = tasks[i].M;
301         int N = tasks[i].N;
302         int K = tasks[i].K;
303
304         if (type == Type::FLOAT) {
305             tasks[i].start_time = chrono::high_resolution_clock::now();
306             cuda_gemm_32bit(A, B, C, M, N, K);
307             tasks[i].end_time = chrono::high_resolution_clock::now();
308         }
309
310         if (type == Type::HALF) {
311             tasks[i].start_time = chrono::high_resolution_clock::now();
312             cuda_gemm_16bit(A, B, C, M, N, K);
313             tasks[i].end_time = chrono::high_resolution_clock::now();
314         }
315
316         if (type == Type::UINT8) {
317             tasks[i].start_time = chrono::high_resolution_clock::now();
318             cuda_gemm_8bit(A, B, C, M, N, K);
319             tasks[i].end_time = chrono::high_resolution_clock::now();
320         }
321     }
322
323     cuda_free();
324
325     //test_compare_task(tasks, num_task);
326     print_performance_stats(tasks, num_task);
327     clean_tasks(tasks,num_task);
328 }
329 #endif
330
331 #ifdef ENABLE_OPENVINO
332 #ifdef ENABLE_SYCL
333 /* test the jit times with different matrixes */
334
335 inline void test_jit_times() {
336     vector<int> dims;
337     vector<double> diffs;
338     chrono::high_resolution_clock::time_point start_time;
339     chrono::high_resolution_clock::time_point end_time;
340
341     chrono::duration<double, std::milli> duration;
342     chrono::duration<double, std::milli> jit_duration;
343
344     double difference = 0.0;
345
```

```
346     ov_init();
347     sycl_init();
348 
349     //dims.push_back(4096);
350     //dims.push_back(3584);
351 
352     for (int d = STEP_SIZE; d <= STEP_SIZE*STEP_TOTAL; d += STEP_SIZE)
353         dims.push_back(d);
354 
355 
356     for (int m : dims)
357         for (int n : dims)
358             for (int k : dims) {
359                 task* task = init_tasks(1, m, n, k, Type::FLOAT);
360 
361                 /* openvino */
362                 start_time = chrono::high_resolution_clock::now();
363                 handle_task(BT::OPENVINO, task);
364                 end_time = chrono::high_resolution_clock::now();
365                 jit_duration = end_time - start_time;
366 
367                 start_time = chrono::high_resolution_clock::now();
368                 handle_task(BT::OPENVINO, task);
369                 end_time = chrono::high_resolution_clock::now();
370                 duration = end_time - start_time;
371 
372                 difference = jit_duration.count() - duration.count();
373                 cout << "ov_jit : " << jit_duration.count()  << " ms | ov :
     " << duration.count() << " ms m " << m << " n " << n << " k "<<  k  <<
    "\n ";
374                 diffs.push_back(difference);
375 
376                 /* sycl */
377                 start_time = chrono::high_resolution_clock::now();
378                 handle_task(BT::SYCL, task);
379                 end_time = chrono::high_resolution_clock::now();
380                 jit_duration = end_time - start_time;
381 
382                 start_time = chrono::high_resolution_clock::now();
383                 handle_task(BT::SYCL, task);
384                 end_time = chrono::high_resolution_clock::now();
385                 duration = end_time - start_time;
386 
387                 difference = jit_duration.count() - duration.count();
388                 cout << "sycl_jit : " << jit_duration.count()  << " ms |
    sycl : " << duration.count() << " ms m " << m << " n " << n << " k "<<
     k  <<"\n ";
389             }
390 
391     task* task_array = init_hetero_tasks(50, Type::FLOAT);
392 
393     for (int i=0; i<50; i++) {
394         start_time = chrono::high_resolution_clock::now();
395         handle_task(BT::OPENVINO, &task_array[i]);
396         end_time = chrono::high_resolution_clock::now();
397         jit_duration = end_time - start_time;
398 
399         start_time = chrono::high_resolution_clock::now();
400         handle_task(BT::OPENVINO, &task_array[i]);
401         end_time = chrono::high_resolution_clock::now();
402         duration = end_time - start_time;
```

```
403
404         difference = jit_duration.count() - duration.count();
405         cout << "ov_jit : " << jit_duration.count()  << " ms | ov : " <<
    duration.count() << " ms m " << task_array[i].M << " n " << task_array[
    i].N << " k "<<  task_array[i].K  <<"\n ";
406         diffs.push_back(difference);
407     }
408
409     double avg = 0;
410     for (auto i: diffs)
411         avg += i;
412     cout << "jit ms: " << avg/diffs.size() << "ms\n";
413
414
415     /*
416     task* task = init_tasks(1, 1046, 4000, 4000, Type::FLOAT);
417
418     start_time = chrono::high_resolution_clock::now();
419     handle_task(BT::SYCL, task);
420     end_time = chrono::high_resolution_clock::now();
421     jit_duration = end_time - start_time;
422
423     start_time = chrono::high_resolution_clock::now();
424     handle_task(BT::SYCL, task);
425     end_time = chrono::high_resolution_clock::now();
426     duration = end_time - start_time;
427
428     difference = jit_duration.count() - duration.count();
429     cout << "sycl_jit : " << jit_duration.count()  << " ms | sycl : " <<
    duration.count() << " ms\n ";
430     */
431
432     ov_free();
433     sycl_free();
434 }
435 #endif
436 #endif
437
438 #endif
```

Listing A.13: tests.hpp

A.1.3 Backend: CUDA

Implementation of the CUDA backend.

```
1  #include "cuda_wrapper.h"
2  #include <stdio.h>
3  #include <cuda_runtime.h>
4  #include <cublas_v2.h>
5  #include <cuda_fp16.h>
6  #include <stdint.h>
7
8  #define CHECK_CUDA(func) {            \
9      cudaError_t e = (func);             \
10     if(e != cudaSuccess)              \
```

```
11          printf ("%s %d CUDA ERROR: %s\n", __FILE__,  __LINE__,
    cudaGetErrorString(e)); \
12 }
13
14 #define CHECK_CUBLAS(func) {                        \
15     cublasStatus_t e = (func);                      \
16     if(e != CUBLAS_STATUS_SUCCESS)                  \
17         printf ("%s %d CUBLAS ERROR: ", __FILE__, __LINE__); \
18 }
19 #define N_STREAM 2
20
21 cublasHandle_t handle;
22 cudaStream_t streams[N_STREAM];
23 void *d_A[N_STREAM];
24 void *d_B[N_STREAM];
25 void *d_C[N_STREAM];
26 int i = 0; /* stream id*/
27
28 void cuda_gemm_32bit_p(float* A, float* B, float* C, int M, int N, int K) {
29
30          cublasSetStream(handle, streams[i]);
31
32          CHECK_CUDA(cudaMemcpyAsync(d_A[i], A, M * K * sizeof(float),
    cudaMemcpyHostToDevice, streams[i]));
33          CHECK_CUDA(cudaMemcpyAsync(d_B[i], B, K * N * sizeof(float),
    cudaMemcpyHostToDevice, streams[i]));
34
35          float a = 1.0f, b = 0.0f;
36
37          cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
38                   N, M, K,
39                   &a,
40                   d_B[i], CUDA_R_32F, N,
41                   d_A[i], CUDA_R_32F, K,
42                   &b,
43                   d_C[i], CUDA_R_32F, N,
44                   CUBLAS_COMPUTE_32F,
45                   CUBLAS_GEMM_DEFAULT_TENSOR_OP);
46
47          CHECK_CUDA(cudaMemcpyAsync(C, d_C[i], M * N * sizeof(float),
    cudaMemcpyDeviceToHost, streams[i]));
48          CHECK_CUDA(cudaStreamSynchronize(streams[i])); /* blocking for
    latency metrics */
49
50          i = (i + 1) % N_STREAM; /* change stream for the next call */
51 }
52
53 void cuda_gemm_16bit_p(__half* A, __half* B, __half* C, int M, int N, int K
    ) {
54      cublasSetStream(handle, streams[i]);
55
56      CHECK_CUDA(cudaMemcpyAsync(d_A[i], A, M * K * sizeof(__half),
    cudaMemcpyHostToDevice, streams[i]));
57      CHECK_CUDA(cudaMemcpyAsync(d_B[i], B, K * N * sizeof(__half),
    cudaMemcpyHostToDevice, streams[i]));
58
59      float a = 1.0f, b = 0.0f;
60
61      cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
62                   N, M, K,
63                   &a,
```

```
64                  d_B[i], CUDA_R_16F, N,
65                  d_A[i], CUDA_R_16F, K,
66                  &b,
67                  d_C[i], CUDA_R_16F, N,
68                  CUBLAS_COMPUTE_32F,
69                  CUBLAS_GEMM_DEFAULT_TENSOR_OP);
70
71      CHECK_CUDA(cudaMemcpyAsync(C, d_C[i], M * N * sizeof(__half),
        cudaMemcpyDeviceToHost, streams[i]));
72      CHECK_CUDA(cudaStreamSynchronize(streams[i]));
73
74      i = (i + 1) % N_STREAM;
75  }
76
77  void cuda_gemm_8bit_p(int8_t* A, int8_t* B, int32_t* C, int M, int N, int K
        ) {
78      cublasSetStream(handle, streams[i]);
79
80      CHECK_CUDA(cudaMemcpyAsync(d_A[i], A, M * K * sizeof(int8_t),
        cudaMemcpyHostToDevice, streams[i]));
81      CHECK_CUDA(cudaMemcpyAsync(d_B[i], B, K * N * sizeof(int8_t),
        cudaMemcpyHostToDevice, streams[i]));
82
83      int32_t a = 1, b = 0;
84
85      cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
86                  N, M, K,
87                  &a,
88                  d_B[i], CUDA_R_8I, N,
89                  d_A[i], CUDA_R_8I, K,
90                  &b,
91                  d_C[i], CUDA_R_32I, N, /* Output : INT32 */
92                  CUBLAS_COMPUTE_32I,
93                  CUBLAS_GEMM_DEFAULT_TENSOR_OP);
94
95      CHECK_CUDA(cudaMemcpyAsync(C, d_C[i], M * N * sizeof(int32_t),
        cudaMemcpyDeviceToHost, streams[i]));
96      CHECK_CUDA(cudaStreamSynchronize(streams[i]));
97
98      i = (i + 1) % N_STREAM;
99  }
100
101 extern "C" {
102     void cuda_init() {
103         CHECK_CUBLAS(cublasCreate(&handle));
104
105         /* reset stream counter */
106         i = 0;
107
108         size_t MAX = (size_t)4096 * 4096 * sizeof(float);
109
110         for (int j = 0; j < N_STREAM; ++j) {
111             CHECK_CUDA(cudaStreamCreate(&streams[j]));
112             CHECK_CUDA(cudaMalloc(&d_A[j], MAX));
113             CHECK_CUDA(cudaMalloc(&d_B[j], MAX));
114             CHECK_CUDA(cudaMalloc(&d_C[j], MAX));
115         }
116     }
117
118     void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K) {
119         cuda_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
```

```
    }

    void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K) {
        cuda_gemm_16bit_p((__half*)A, (__half*)B, (__half*)C, M, N, K);
    }

    void cuda_gemm_8bit(void* A, void* B, void* C, int M, int N, int K) {
        cuda_gemm_8bit_p((int8_t*)A, (int8_t*)B, (int32_t*)C, M, N, K);
    }

    void cuda_free() {
        for (int j = 0; j < N_STREAM; ++j) {
            CHECK_CUDA(cudaFree(d_A[j]));
            CHECK_CUDA(cudaFree(d_B[j]));
            CHECK_CUDA(cudaFree(d_C[j]));
            CHECK_CUDA(cudaStreamDestroy(streams[j]));
        }
        CHECK_CUBLAS(cublasDestroy(handle));
    }
}
```

Listing A.14: CUDA Runner (runner.cu)

```
#ifndef WRAPPER_H
#define WRAPPER_H

#ifdef __cplusplus
extern "C" {
#endif

    void cuda_init();
    void cuda_free();
    void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);

#ifdef __cplusplus
}
#endif

#endif
```

Listing A.15: CUDA Wrapper (cuda_wrapper.h)

A.1.4 Backend: SYCL

Implementation of the SYCL backend.

```
#include <cstdlib>
#include <sycl/sycl.hpp>
#include <oneapi/mkl/blas.hpp>
#include <iostream>
#include <memory>

```

```
7  #define N_STREAMS 2
8
9  using namespace std;
10
11 struct SYCLStream {
12     sycl::queue q;
13     float *d_A_f, *d_B_f, *d_C_f;
14     sycl::half *d_A_h, *d_B_h, *d_C_h;
15
16     SYCLStream(size_t max_elements) : q(sycl::gpu_selector_v, sycl::
       property::queue::in_order()) {
17         d_A_f = sycl::malloc_device<float>(max_elements, q);
18         d_B_f = sycl::malloc_device<float>(max_elements, q);
19         d_C_f = sycl::malloc_device<float>(max_elements, q);
20
21         d_A_h = sycl::malloc_device<sycl::half>(max_elements, q);
22         d_B_h = sycl::malloc_device<sycl::half>(max_elements, q);
23         d_C_h = sycl::malloc_device<sycl::half>(max_elements, q);
24
25         if (q.get_device().get_info<sycl::info::device::name>()  != "Intel(
       R) Arc(TM) Graphics") {
26             std::cout << "Intel GPU NOT PRESENT\n";
27             exit(EXIT_FAILURE);
28         }
29     }
30
31     ~SYCLStream() {
32         sycl::free(d_A_f, q); sycl::free(d_B_f, q); sycl::free(d_C_f, q);
33         sycl::free(d_A_h, q); sycl::free(d_B_h, q); sycl::free(d_C_h, q);
34     }
35 };
36
37 static std::vector<std::unique_ptr<SYCLStream>> streams;
38 static int current_stream = 0;
39 static size_t MAX = (size_t)4096 * 4096;
40
41 void init() {
42     try {
43
44         /* stream persistenti */
45         for(int i=0; i<N_STREAMS; i++) {
46             streams.push_back(std::make_unique<SYCLStream>(MAX));
47         }
48
49     } catch (sycl::exception const& e) {
50         std::cerr << "[SYCL ERROR] Init: " << e.what() << "\n";
51         exit(1);
52     }
53 }
54
55 void sycl_gemm_32bit_p(float *A, float *B, float *C, int M, int N, int K) {
56     SYCLStream& s = *streams[current_stream];
57
58     s.q.memcpy(s.d_A_f, A, M * K * sizeof(float));
59     s.q.memcpy(s.d_B_f, B, K * N * sizeof(float));
60
61     float alpha = 1.0f, beta = 0.0f;
62     oneapi::mkl::blas::row_major::gemm(s.q, oneapi::mkl::transpose::
       nontrans,
63             oneapi::mkl::transpose::nontrans,
64             M, N, K, alpha, s.d_A_f, K, s.d_B_f, N, beta, s.d_C_f, N);
```

```
65
66      s.q.memcpy(C, s.d_C_f, M * N * sizeof(float));
67      s.q.wait();
68
69      current_stream = (current_stream + 1) % N_STREAMS;
70  }
71
72  void sycl_gemm_16bit_p(sycl::half *A, sycl::half *B,
73                                  sycl::half *C, int M, int N, int K) {
74      SYCLStream& s = *streams[current_stream];
75
76      s.q.memcpy(s.d_A_h, A, M * K * sizeof(sycl::half));
77      s.q.memcpy(s.d_B_h, B, K * N * sizeof(sycl::half));
78
79      sycl::half alpha = 1.0f, beta = 0.0f;
80      oneapi::mkl::blas::row_major::gemm(s.q, oneapi::mkl::transpose::
        nontrans,
81              oneapi::mkl::transpose::nontrans,
82              M, N, K, alpha, s.d_A_h, K, s.d_B_h, N, beta, s.d_C_h, N);
83
84      s.q.memcpy(C, s.d_C_h, M * N * sizeof(sycl::half));
85      s.q.wait();
86
87      current_stream = (current_stream + 1) % N_STREAMS;
88  }
89
90  extern "C" {
91      void sycl_init() {
92          init();
93      }
94
95      void sycl_gemm_32bit(void *A, void *B, void *C, int M, int N, int K) {
96          sycl_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
97      }
98
99      void sycl_gemm_16bit(void *A, void *B, void *C, int M, int N, int K) {
100         sycl_gemm_16bit_p((sycl::half*)A, (sycl::half*)B, (sycl::half*)C, M
        , N, K);
101     }
102
103     void sycl_free() {
104         streams.clear();
105     }
106 }
```

Listing A.16: SYCL Runner (runner.cpp)

```
1  #ifndef SYCL_WRAPPER_H
2  #define SYCL_WRAPPER_H
3
4  #ifdef __cplusplus
5  extern "C" {
6  #endif
7
8      void sycl_init();
9      void sycl_gemm_32bit(void *A, void *B, void *C, int M, int N, int K);
10     void sycl_gemm_16bit(void *A, void *B, void *C, int M, int N, int K);
11     void sycl_free();
```

```
12
13 #ifdef __cplusplus
14 }
15 #endif
16
17 #endif
```

Listing A.17: SYCL Wrapper (sycl_wrapper.h)

A.1.5 Backend: OpenVINO

Implementation of the OpenVINO backend.

```
1  #include "ov_wrapper.h" // Il nostro header C
2  #include <openvino/openvino.hpp>
3  #include <openvino/op/matmul.hpp>
4  #include <openvino/op/convert.hpp>
5  #include <cstdlib>
6  #include <iostream>
7
8  /* this library is intended for the Intel NPU */
9  ov::Core core;
10
11 using namespace std;
12
13 /* cache for compiled models */
14 static std::map<tuple<int, int, int>, ov::InferRequest> request_cache_32bit
       ;
15 static std::map<tuple<int, int, int>, ov::InferRequest> request_cache_16bit
       ;
16 static std::map<tuple<int, int, int>, ov::InferRequest> request_cache_8bit;
17 const size_t MAX_MAP_SIZE = 20;
18 size_t map_size_32bit=0;
19 size_t map_size_16bit=0;
20 size_t map_size_8bit=0;
21
22 void cache_32bit(tuple<int, int, int> key) {
23     if (request_cache_32bit.find(key) == request_cache_32bit.end()) {
24         map_size_32bit++;
25         if (map_size_32bit >= MAX_MAP_SIZE) {
26             request_cache_32bit.clear();
27             map_size_32bit = 1;
28         }
29         auto A_ov = std::make_shared<ov::op::v0::Parameter>(ov::element::
       f32, ov::Shape{(size_t)get<0>(key), (size_t)get<2>(key)});
30         auto B_ov = std::make_shared<ov::op::v0::Parameter>(ov::element::
       f32, ov::Shape{(size_t)get<2>(key), (size_t)get<1>(key)});
31
32         auto matmul = std::make_shared<ov::op::v0::MatMul>(A_ov, B_ov);
33         auto result = std::make_shared<ov::op::v0::Result>(matmul);
34
35         auto model = std::make_shared<ov::Model>(result, ov::
       ParameterVector{A_ov, B_ov});
36
37         ov::CompiledModel compiled_model = core.compile_model(model, "NPU")
       ;
```

```
38          ov::InferRequest request = compiled_model.create_infer_request();
39
40          request_cache_32bit[key] = request;
41      }
42  }
43
44  void cache_16bit(tuple<int, int, int> key) {
45      if (request_cache_16bit.find(key) == request_cache_16bit.end()) {
46          map_size_16bit++;
47          if (map_size_16bit >= MAX_MAP_SIZE) {
48              request_cache_16bit.clear();
49              map_size_16bit = 1;
50          }
51          auto A_ov = std::make_shared<ov::op::v0::Parameter>(ov::element::
      f16, ov::Shape{(size_t)get<0>(key), (size_t)get<2>(key)});
52          auto B_ov = std::make_shared<ov::op::v0::Parameter>(ov::element::
      f16, ov::Shape{(size_t)get<2>(key), (size_t)get<1>(key)});
53
54          auto matmul = std::make_shared<ov::op::v0::MatMul>(A_ov, B_ov);
55          auto result = std::make_shared<ov::op::v0::Result>(matmul);
56
57          auto model = std::make_shared<ov::Model>(result, ov::
      ParameterVector{A_ov, B_ov});
58
59          ov::CompiledModel compiled_model = core.compile_model(model, "NPU")
      ;
60          ov::InferRequest request = compiled_model.create_infer_request();
61
62          request_cache_16bit[key] = request;
63      }
64  }
65
66  void cache_8bit(tuple<int, int, int> key) {
67      if (request_cache_8bit.find(key) == request_cache_8bit.end()) {
68          map_size_8bit++;
69          if (map_size_8bit >= MAX_MAP_SIZE) {
70              request_cache_8bit.clear();
71              map_size_8bit = 1;
72          }
73          auto A_ov = std::make_shared<ov::op::v0::Parameter>(ov::element::i8
      , ov::Shape{(size_t)get<0>(key), (size_t)get<2>(key)});
74          auto B_ov = std::make_shared<ov::op::v0::Parameter>(ov::element::i8
      , ov::Shape{(size_t)get<2>(key), (size_t)get<1>(key)});
75
76          /* convert inner tensor from 8bit to 16bit */
77          auto A_conv = std::make_shared<ov::op::v0::Convert>(A_ov, ov::
      element::i16);
78          auto B_conv = std::make_shared<ov::op::v0::Convert>(B_ov, ov::
      element::i16);
79
80          auto matmul = std::make_shared<ov::op::v0::MatMul>(A_conv, B_conv);
81
82          auto result_conv = std::make_shared<ov::op::v0::Convert>(matmul, ov
      ::element::i32);
83          auto result = std::make_shared<ov::op::v0::Result>(result_conv);
84
85          auto model = std::make_shared<ov::Model>(result, ov::
      ParameterVector{A_ov, B_ov});
86
87          ov::CompiledModel compiled_model = core.compile_model(model, "NPU")
      ;
```

```
88          ov::InferRequest request = compiled_model.create_infer_request();
89
90          request_cache_8bit[key] = request;
91      }
92  }
93
94  void ov_init_p() {
95      bool available = false;
96
97      for (auto &d : core.get_available_devices())
98          if (d.find("NPU") != string::npos)
99              available = true;
100
101     if (!available) {
102         cout << "[OPENVINO] NPU not present, aborting \n";
103         exit(EXIT_FAILURE); /* shut down something is wrong */
104     }
105 }
106
107 /* gemm with cached openvino */
108 void ov_gemm_32bit_p(void *A, void *B, void *C, int M, int N, int K){
109     auto key = std::make_tuple(M, N, K);
110     cache_32bit(key);
111
112     ov::InferRequest& infer_request = request_cache_32bit[key];
113
114     ov::Tensor tensor_A(ov::element::f32, ov::Shape{(size_t)M, (size_t)K},
    A);
115     ov::Tensor tensor_B(ov::element::f32, ov::Shape{(size_t)K, (size_t)N},
    B);
116     ov::Tensor tensor_C(ov::element::f32, ov::Shape{(size_t)M, (size_t)N},
    C);
117
118     infer_request.set_input_tensor(0, tensor_A);
119     infer_request.set_input_tensor(1, tensor_B);
120     infer_request.set_output_tensor(0, tensor_C);
121
122     infer_request.infer();
123 }
124
125 void ov_gemm_16bit_p(void *A, void *B, void *C, int M, int N, int K){
126     auto key = std::make_tuple(M, N, K);
127     cache_16bit(key);
128
129     ov::InferRequest& infer_request = request_cache_16bit[key];
130
131     ov::Tensor tensor_A(ov::element::f16, ov::Shape{(size_t)M, (size_t)K},
    A);
132     ov::Tensor tensor_B(ov::element::f16, ov::Shape{(size_t)K, (size_t)N},
    B);
133     ov::Tensor tensor_C(ov::element::f16, ov::Shape{(size_t)M, (size_t)N},
    C);
134
135     infer_request.set_input_tensor(0, tensor_A);
136     infer_request.set_input_tensor(1, tensor_B);
137     infer_request.set_output_tensor(0, tensor_C);
138
139     infer_request.infer();
140 }
141
142 void ov_gemm_8bit_p(void *A, void *B, void *C, int M, int N, int K){
```

```
    auto key = std::make_tuple(M, N, K);
    cache_8bit(key);

    ov::InferRequest& infer_request = request_cache_8bit[key];

    ov::Tensor tensor_A(ov::element::i8, ov::Shape{(size_t)M, (size_t)K}, A);
    ov::Tensor tensor_B(ov::element::i8, ov::Shape{(size_t)K, (size_t)N}, B);
    ov::Tensor tensor_C(ov::element::i32, ov::Shape{(size_t)M, (size_t)N}, C);

    infer_request.set_input_tensor(0, tensor_A);
    infer_request.set_input_tensor(1, tensor_B);
    infer_request.set_output_tensor(0, tensor_C);

    infer_request.infer();
}

extern "C" {
    void ov_init() {
        ov_init_p();
    }

    void ov_gemm_32bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_32bit_p(A, B, C, M, N, K);
    }

    void ov_gemm_16bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_16bit_p(A, B, C, M, N, K);
    }

    void ov_gemm_8bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_8bit_p(A, B, C, M, N, K);
    }

    void ov_free() {
    }
}
```

Listing A.18: OpenVINO Runner (runner.cpp)

```
#ifndef OV_WRAPPER_H
#define OV_WRAPPER_H

#ifdef __cplusplus
extern "C" {
#endif

    void ov_init();

    void ov_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
    void ov_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
    void ov_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);

    void ov_free();

#ifdef __cplusplus
```

```
}
#endif

#endif
```

Listing A.19: OpenVINO Wrapper (ov_wrapper.h)

OpenCV test

```
#ifndef OPENCV_TEST_H
#define OPENCV_TEST_H


#include <cstdlib>
#include <vector>
#ifdef ENABLE_OPENCV
#include <opencv2/opencv.hpp>
#endif

#include "tasks.hpp"
#include "scheduler.hpp"



inline void prepare_task_from_frame(const cv::Mat& frame, const cv::Mat&
    kernel, task& t) {
    int k_sz = kernel.rows;

    int out_h = frame.rows - k_sz + 1;
    int out_w = frame.cols - k_sz + 1;
    int num_patches = out_h * out_w;

    t.type = Type::FLOAT;
    t.M = 1;
    t.K = k_sz * k_sz;
    t.N = num_patches;

    // A: 1 * K
    t.A = new float[t.M * t.K];
    // B: K * N
    t.B = new float[t.K * t.N];
    // C: 1 * N (M * N)
    t.C = new float[t.M * t.N];

    float* A_ptr = (float*)t.A;
    int a_idx = 0;
    for(int i=0; i<k_sz; ++i)
        for(int j=0; j<k_sz; ++j)
            A_ptr[a_idx++] = kernel.at<float>(i, j);

    float* B_ptr = (float*)t.B;
    int patch_idx = 0;

    for (int y = 0; y < out_h; y++)
        for (int x = 0; x < out_w; x++) {
            int k_pixel_idx = 0;
```

```
47              for (int ky = 0; ky < k_sz; ky++) {
48                  for (int kx = 0; kx < k_sz; kx++) {
49                      float pixel_val = frame.at<float>(y + ky, x + kx);
50                      B_ptr[k_pixel_idx * t.N + patch_idx] = pixel_val;
51                      k_pixel_idx++;
52                  }
53              }
54              patch_idx++;
55          }
56  }
57
58  inline cv::Mat result_from_task(const task& t, int original_w, int
        kernel_sz) {
59      int out_h = (t.N / (original_w - kernel_sz + 1));
60      int out_w = (original_w - kernel_sz + 1);
61      cv::Mat result(out_h, out_w, CV_32F, t.C);
62      return result.clone();
63  }
64
65  inline void free_batch(std::vector<task>& tasks) {
66      for(auto& t : tasks) {
67          delete[] (float*)t.A;
68          delete[] (float*)t.B;
69          delete[] (float*)t.C;
70      }
71      tasks.clear();
72  }
73
74  inline void display_video(std::vector<task>& tasks) {
75
76  }
77
78  inline void test_video_filter(Logic l, bool display) {
79      cv::VideoCapture cap("test.mp4");
80
81      if(!cap.isOpened()) {
82          cout << "[OPENCV TEST] ERROR video no present\n";
83          exit(EXIT_FAILURE);
84      }
85
86      double fps = cap.get(cv::CAP_PROP_FPS);
87      if (fps <= 0) fps = 30.0;
88
89      /* edge detection */
90      cv::Mat kernel = (cv::Mat_<float>(3, 3) << -1, -1, -1, -1, 8, -1, -1,
        -1, -1);
91
92      cv::Mat frame, gray, float_img;
93      int i=0;
94
95      bool video_ended = false;
96
97      vector<task> tasks;
98      tasks.reserve(1000);
99
100
101     while(true) {
102         cap >> frame;
103         if (frame.empty()) {
104             break;
105         }
```

```
106
107         cv::cvtColor(frame, gray, cv::COLOR_BGR2GRAY);
108         gray.convertTo(float_img, CV_32F);
109         task t;
110         prepare_task_from_frame(float_img, kernel, t);
111
112         tasks.push_back(t);
113     }
114
115     cout << "[OPENCV TEST] frames done\n";
116
117     AMScheduler scheduler = AMScheduler(1);
118     scheduler.do_tasks(tasks.data(), tasks.size());
119     scheduler.wait();
120     scheduler.print_stats(tasks.data(), tasks.size());
121
122     cout << "[OPENCV TEST] scheduler done \n";
123
124     if (!tasks.empty()) {
125         int out_w = float_img.cols - kernel.rows + 1;
126         int out_h = float_img.rows - kernel.rows + 1;
127         cv::Size frame_size(out_w, out_h);
128
129         cv::VideoWriter writer("test_edge.mp4",
130                                cv::VideoWriter::fourcc('M','P','4','V'),
131                                fps, frame_size, false);
132
133         if (!writer.isOpened()) {
134             std::cout << "[ERROR] video won't open\n";
135         } else {
136             for (auto& t : tasks) {
137                 cv::Mat out = result_from_task(t, float_img.cols, kernel.
    rows);
138
139                 cv::Mat display_frame;
140                 cv::normalize(out, out, 0, 255, cv::NORM_MINMAX);
141                 out.convertTo(display_frame, CV_8U);
142
143                 writer.write(display_frame);
144
145                 if (display) {
146                     cv::imshow("Scheduler Output", display_frame);
147                     if (cv::waitKey(1) == 27) break;
148                 }
149             }
150             std::cout << "[OPENCV TEST] Video saved test_edge.mp4\n";
151         }
152     }
153
154     free_batch(tasks);
155 }
156
157 #endif
```

Listing A.20: OpenCV Video Test (opencv_test.hpp)

A.2 Benchmarks

This section includes the standalone benchmarks developed to study the behavior of the accelerators in the preliminary phase.

A.2.1 CUDA Benchmark

```
#include "include/benchmark/gem_runner.hpp"
#include "include/benchmark/mem_runner.hpp"

int main() {

  printGPUInfo(getGPUInfo(0));

#ifdef GEM
  dim3 blockNaive(16, 16);
  dim3 gridNaive((N_SIZE + blockNaive.x - 1) / blockNaive.x,
                 (M_SIZE + blockNaive.y - 1) / blockNaive.y);

  benchmark_gem(KernelType::NAIVE, M_SIZE, N_SIZE, K_SIZE, blockNaive,
                gridNaive, RUNS);

  dim3 blockTiled(TILE_SIZE, TILE_SIZE);
  dim3 gridTiled((N_SIZE + TILE_SIZE - 1) / TILE_SIZE,
                 (M_SIZE + TILE_SIZE - 1) / TILE_SIZE);

  benchmark_gem(KernelType::TILED, M_SIZE, N_SIZE, K_SIZE, blockTiled,
                gridTiled, RUNS);

  dim3 blockDense(DENSE_BLOCK_N / DENSE_THREAD_X,
                  DENSE_BLOCK_M / DENSE_THREAD_Y);
  dim3 gridDense(N_SIZE / DENSE_BLOCK_N, M_SIZE / DENSE_BLOCK_M);

  if (N_SIZE % DENSE_BLOCK_N != 0)
    gridDense.x++;
  if (M_SIZE % DENSE_BLOCK_M != 0)
    gridDense.y++;

  benchmark_gem(KernelType::DENSE, M_SIZE, N_SIZE, K_SIZE, blockDense,
                gridDense, RUNS);

  benchmark_gem(KernelType::CUBLAS, M_SIZE, N_SIZE, K_SIZE, NULL, NULL,
    RUNS);

  benchmark_gem(KernelType::TENSOR, M_SIZE, N_SIZE, K_SIZE, NULL, NULL,
    RUNS);
#endif

#ifdef MEM
  int N = 1 << 24; /* 16 milioni circ */
  dim3 block(256), grid((N + 255) / 256);

  float alpha = 2.5f;

  benchmark_mem(KernelType::COPY, 2 * sizeof(float), N, block, grid, alpha,
                RUNS);
```

```
48   benchmark_mem(KernelType::SCALE, 2 * sizeof(float), N, block, grid, alpha
       ,
49                 RUNS);
50   benchmark_mem(KernelType::ADD, 3 * sizeof(float), N, block, grid, alpha,
51                 RUNS);
52   benchmark_mem(KernelType::TRIAD, 3 * sizeof(float), N, block, grid, alpha
       ,
53                 RUNS);
54
55 #endif
56
57   return 0;
58 }
```

Listing A.21: CUDA Main (main.cu)

```
1 #include "../include/benchmark/gem_runner.hpp"
2 #include "../include/power.hpp"
3
4 TestResult run_gem(KernelType kernel, const int M, const int N, const int K
      , dim3 blockSize, dim3 gridSize) {
5
6     std::vector<float> h_A(M * K), h_B(K * N), h_C(M * N);
7
8     std::mt19937 gen(42);
9     std::uniform_real_distribution<float> dis(-1.0f, 1.0f);
10
11    for (auto& x : h_A) x = dis(gen);
12    for (auto& x : h_B) x = dis(gen);
13    for (auto& x : h_C) x = 0.0f;
14
15    float *d_A, *d_B, *d_C;
16
17    CHECK_CUDA(cudaMalloc(&d_A, M * K * sizeof(float)));
18    CHECK_CUDA(cudaMalloc(&d_B, K * N * sizeof(float)));
19    CHECK_CUDA(cudaMalloc(&d_C, M * N * sizeof(float)));
20
21    cudaEvent_t start_mem, stop_mem;
22    CHECK_CUDA(cudaEventCreate(&start_mem));
23    CHECK_CUDA(cudaEventCreate(&stop_mem));
24
25    CHECK_CUDA(cudaEventRecord(start_mem)); /* memory timer */
26    CHECK_CUDA(cudaMemcpy(d_A, h_A.data(), M * K * sizeof(float),
      cudaMemcpyHostToDevice));
27    CHECK_CUDA(cudaMemcpy(d_B, h_B.data(), K * N * sizeof(float),
      cudaMemcpyHostToDevice));
28    CHECK_CUDA(cudaMemcpy(d_C, h_C.data(), M * N * sizeof(float),
      cudaMemcpyHostToDevice));
29    CHECK_CUDA(cudaEventRecord(stop_mem));
30
31    float ms_memcpy = 0.0;
32    CHECK_CUDA(cudaEventSynchronize(stop_mem)); /* make sure all the op are
       done */
33    CHECK_CUDA(cudaEventElapsedTime(&ms_memcpy, start_mem, stop_mem));
34
35    cudaEvent_t start_compute, stop_compute;
36    CHECK_CUDA(cudaEventCreate(&start_compute));
37    CHECK_CUDA(cudaEventCreate(&stop_compute));
```

```
38
39      switch(kernel) {
40          case KernelType::NAIVE:
41              simple_gemm_kernel<<<gridSize, blockSize>>>(d_A, d_B, d_C, M, N
    , K);
42              CHECK_CUDA(cudaDeviceSynchronize());
43
44              CHECK_CUDA(cudaEventRecord(start_compute));
45              simple_gemm_kernel<<<gridSize, blockSize>>>(d_A, d_B, d_C, M, N
    , K);
46              CHECK_CUDA(cudaEventRecord(stop_compute));
47              CHECK_CUDA(cudaDeviceSynchronize());
48              simple_gemm_kernel<<<gridSize, blockSize>>>(d_A, d_B, d_C, M, N
    , K);
49              break;
50          case KernelType::TILED:
51              tile_gem_kernel<TILE_SIZE><<<gridSize, blockSize>>>(d_A, d_B,
    d_C, M, N, K);
52              CHECK_CUDA(cudaDeviceSynchronize());
53
54              CHECK_CUDA(cudaEventRecord(start_compute));
55              tile_gem_kernel<TILE_SIZE><<<gridSize, blockSize>>>(d_A, d_B,
    d_C, M, N, K);
56              CHECK_CUDA(cudaEventRecord(stop_compute));
57              CHECK_CUDA(cudaDeviceSynchronize());
58              break;
59          case KernelType::DENSE:
60              dense_gem_kernel<DENSE_BLOCK_M, DENSE_BLOCK_K, DENSE_BLOCK_N,
    DENSE_THREAD_Y, DENSE_THREAD_X>
61                              <<<gridSize, blockSize>>>(d_A, d_B, d_C, M, N,
    K, DENSE_ALPHA, DENSE_BETA);
62              CHECK_CUDA(cudaDeviceSynchronize());
63
64              CHECK_CUDA(cudaEventRecord(start_compute));
65              dense_gem_kernel<DENSE_BLOCK_M, DENSE_BLOCK_K, DENSE_BLOCK_N,
    DENSE_THREAD_Y, DENSE_THREAD_X>
66                              <<<gridSize, blockSize>>>(d_A, d_B, d_C, M, N,
    K, DENSE_ALPHA, DENSE_BETA);
67              CHECK_CUDA(cudaEventRecord(stop_compute));
68              CHECK_CUDA(cudaDeviceSynchronize());
69              break;
70          case KernelType::CUBLAS:
71              {
72                  float alpha = 1.0f, beta = 0.0f;
73                  cublasHandle_t handle;
74                  CHECK_CUBLAS(cublasCreate(&handle));
75
76                  cublasSgemm (handle, CUBLAS_OP_N, CUBLAS_OP_N,
77                      N, M, K, &alpha,
78                      d_B, N, d_A, K, &beta, d_C, N
79                  );
80
81                  CHECK_CUDA(cudaDeviceSynchronize());
82                  CHECK_CUDA(cudaEventRecord(start_compute));
83                  cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
84                              N, M, K, &alpha,
85                              d_B, N, d_A, K, &beta, d_C, N);
86                  CHECK_CUDA(cudaEventRecord(stop_compute));
87                  CHECK_CUDA(cudaDeviceSynchronize());
88                  CHECK_CUBLAS(cublasDestroy(handle));
89                  break;
```

```
90              }
91          case KernelType::TENSOR:
92              {
93                  half *d_A16, *d_B16;
94
95                  /* 16 bit matrix for input */
96                  CHECK_CUDA(cudaMalloc(&d_A16, M * K * sizeof(half)));
97                  CHECK_CUDA(cudaMalloc(&d_B16, K * N * sizeof(half)));
98
99                  /* converto da 32 to 16 */
100                 cudaDeviceSynchronize();
101                 cudaMemcpy(d_A16, d_A, M * K * sizeof(half),
    cudaMemcpyDeviceToDevice);
102                 cudaMemcpy(d_B16, d_B, K * N * sizeof(half),
    cudaMemcpyDeviceToDevice);
103                 cublasHandle_t handle;
104                 CHECK_CUBLAS(cublasCreate(&handle));
105
106                 /* enable tensor core */
107                 CHECK_CUBLAS(cublasSetMathMode(handle,
    CUBLAS_TENSOR_OP_MATH));
108                 const float alpha = 1.0f, beta = 0.0f;
109
110                 CHECK_CUBLAS(cublasGemmEx(handle,
111                                           CUBLAS_OP_N, CUBLAS_OP_N,
112                                           N, M, K,
113                                           &alpha,
114                                           d_B16, CUDA_R_16F, N,
115                                           d_A16, CUDA_R_16F, K,
116                                           &beta,
117                                           d_C, CUDA_R_32F, N,
118                                           CUDA_R_32F,
119                                           /* force tensor core */
120                                           CUBLAS_GEMM_DEFAULT_TENSOR_OP));
121
122                 cudaDeviceSynchronize();
123                 CHECK_CUDA(cudaEventRecord(start_compute));
124                 CHECK_CUBLAS(cublasGemmEx(handle,
125                                           CUBLAS_OP_N, CUBLAS_OP_N,
126                                           N, M, K,
127                                           &alpha,
128                                           d_B16, CUDA_R_16F, N,
129                                           d_A16, CUDA_R_16F, K,
130                                           &beta,
131                                           d_C, CUDA_R_32F, N,
132                                           CUDA_R_32F,
133                                           CUBLAS_GEMM_DEFAULT_TENSOR_OP));
134                 CHECK_CUDA(cudaEventRecord(stop_compute));
135                 cudaDeviceSynchronize();
136
137                 cublasDestroy(handle);
138                 cudaFree(d_A16);
139                 cudaFree(d_B16);
140                 break;
141             }
142             default:
143                 throw std::invalid_argument("Unknown kernel type: " + std::
    to_string(static_cast<int>(kernel)));
144                 break;
145             }
146
```

```
147     float ms_compute = 0.0;
148     CHECK_CUDA(cudaEventElapsedTime(&ms_compute, start_compute,
    stop_compute));
149     double gflops = (2.0 * M * N * K) / (ms_compute * 1e6);
150 
151 #ifdef CHECK_RESULT
152     static std::vector<float> h_C_cpu(M * N);
153     static bool calculated = false;
154     if (!calculated) {
155         std::cout << "------ [DEBUG RESULT]: compute matrix on cpu... ";
156         cpu_gemm(h_A.data(), h_B.data(), h_C_cpu.data(), M, N, K);
157         std::cout << " ...cpu complete \n";
158         calculated = true;
159     }
160 
161     std::vector<float> h_C_gpu(M * N);
162     CHECK_CUDA(cudaMemcpy(h_C_gpu.data(), d_C, M * N * sizeof(float),
    cudaMemcpyDeviceToHost));
163 
164     if (compare_matrix(h_C_gpu.data(), h_C_cpu.data(), M * N)) {
165         std::cout << "------ [DEBUG RESULT] CORRECT for " << getKernelName(
    kernel) << std::endl;
166     } else {
167         std::cerr << "------ [DEBUG RESULT] INCORRECT for " <<
    getKernelName(kernel) << std::endl;
168     }
169 #endif
170 
171     CHECK_CUDA(cudaFree(d_A));
172     CHECK_CUDA(cudaFree(d_B));
173     CHECK_CUDA(cudaFree(d_C));
174     CHECK_CUDA(cudaEventDestroy(start_compute));
175     CHECK_CUDA(cudaEventDestroy(stop_compute));
176 
177     return {ms_compute , gflops, ms_memcpy};
178 }
179 
180 void write_result_csv(const std::string &filename,
181                           const std::string &kernelName,
182                          int M, int N, int K, int runs,
183                          std::vector<RunData> &results)
184 {
185 
186     // Calcola statistiche
187     auto [min_ms_compute_it, max_ms_compute_it] = std::minmax_element(
    results.begin(), results.end(),
188         [](const RunData &a, const RunData &b) { return a.ms_compute < b.
    ms_compute; });
189 
190     auto [min_ms_memcpy_it, max_ms_memcpy_it] = std::minmax_element(results
    .begin(), results.end(),
191         [](const RunData &a, const RunData &b) { return a.ms_memcpy < b.
    ms_memcpy; });
192 
193     auto [min_gflops_it, max_gflops_it] = std::minmax_element(results.begin
    (), results.end(),
194         [](const RunData &a, const RunData &b) { return a.gflops < b.gflops
    ; });
195 
196     double avg_ms_compute = 0.0, avg_ms_memcpy = 0.0, avg_gflops = 0.0,
    avg_power = 0.0;
```

```
197     double min_power = results.front().min_power;
198     double max_power = results.front().max_power;
199 
200     for (const auto &r : results) {
201         avg_ms_compute += r.ms_compute;
202         avg_ms_memcpy += r.ms_memcpy;
203         avg_gflops += r.gflops;
204         avg_power += r.avg_power;
205         min_power = std::min(min_power, r.min_power);
206         max_power = std::max(max_power, r.max_power);
207     }
208 
209     avg_ms_compute /= results.size();
210     avg_ms_memcpy /= results.size();
211     avg_gflops /= results.size();
212     avg_power /= results.size();
213 
214     auto timestamp = std::chrono::system_clock::to_time_t(std::chrono::
    system_clock::now());
215 
216     std::ofstream file(filename, std::ios::app); /* append the file */
217     if (!file.is_open()) {
218         std::cerr << "[ERROR]: can't open " << filename << "\n";
219         return;
220     }
221 
222     /* if the file is empty */
223     file.seekp(0, std::ios::end);
224     if (file.tellp() == 0) {
225         file << "Timestamp,Kernel,M,N,K,Runs,"
226              << "Avg_Compute_ms,Min_Compute_ms,Max_Compute_ms,"
227              << "Avg_MemCpy_ms,Min_MemCpy_ms,Max_MemCpy_ms,"
228              << "Avg_GFLOPS,Min_GFLOPS,Max_GFLOPS,"
229              << "Avg_Power_W,Min_Power_W,Max_Power_W\n";
230     }
231 
232     file << timestamp << ","
233          << kernelName << ","
234          << M << "," << N << "," << K << ","
235          << runs << ","
236          << std::fixed << std::setprecision(3)
237          << avg_ms_compute << "," << min_ms_compute_it->ms_compute << ","
    << max_ms_compute_it->ms_compute << ","
238          << avg_ms_memcpy << "," << min_ms_memcpy_it->ms_memcpy << "," <<
    max_ms_memcpy_it->ms_memcpy << ","
239          << avg_gflops << "," << min_gflops_it->gflops << "," <<
    max_gflops_it->gflops << ","
240          << avg_power << "," << min_power << "," << max_power
241          << "\n";
242 
243     file.close();
244 
245     std::cout << "\nResults (" << kernelName << ")\n";
246     std::cout << "Matrix: " << M << "x" << N << "x" << K << "\n";
247     std::cout << runs << " run(s)\n";
248     std::cout << "Timestamp: " << timestamp << "\n\n";
249     std::cout << "avg compute ms: " << avg_ms_compute << " ms (min " <<
    min_ms_compute_it->ms_compute << ", max " << max_ms_compute_it->
    ms_compute << ")\n";
250     std::cout << "avg memcpy ms : " << avg_ms_memcpy << " ms (min " <<
    min_ms_memcpy_it->ms_memcpy << ", max " << max_ms_memcpy_it->ms_memcpy
```

```
    << ")\n";
251     std::cout << "avg GFLOPS : " << avg_gflops << " GF (min " <<
    min_gflops_it->gflops << ", max " << max_gflops_it->gflops << ")\n";
252     std::cout << "avg Power : " << avg_power << " W (min " << min_power <<
    ", max " << max_power << ")\n";
253     std::cout << "-----------------------------------------\n";
254 }
255
256 void benchmark_gem(KernelType kernel, const int M, const int N, const int K
    , dim3 blockSize, dim3 gridSize, const int runs) {
257     std::cout << "========== Benchmark: " << getKernelName(kernel)
258               << " | " << M << "x" << N
259               << " (K=" << K << "), "
260               << runs << " runs\n";
261
262     double avg_power = 0.0,
263         min_power = 0.0,
264         max_power = 0.0;
265
266     /* thread for gpu power sampling */
267     GpuPowerSampler sampler(0, 100);
268
269     /* results vector for each run */
270     std::vector<RunData> results;
271     results.reserve(runs);
272
273
274     for (int i = 0; i < runs; i++) {
275
276         sampler.start();
277
278         /* running kernel */
279         TestResult r = run_gem(kernel, M, N, K, blockSize, gridSize);
280
281         sampler.stop();
282
283         avg_power = sampler.averagePower();
284         min_power = sampler.minPower();
285         max_power = sampler.maxPower();
286
287         results.push_back({r.ms_compute, r.ms_memcpy, r.gflops, avg_power,
    min_power, max_power});
288
289 #ifdef DEBUG
290         std::cout << "[DEBUG]: Run " << i+1 << ": "
291                   << "Compute " << r.ms_compute << " ms, "
292                   << "MemCpy " << r.ms_memcpy << " ms, "
293                   << r.gflops << " GFLOPS, "
294                   << " | Power (avg/min/max): "
295                   << avg_power << "/" << min_power << "/" << max_power << "
     W\n";
296 #endif
297     }
298
299     write_result_csv("gem_benchmark_results.csv", getKernelName(kernel), M,
     N, K, runs, results);
300 }
```

Listing A.22: GEMM Runner (gem_runner.cu)

```
#include "../include/benchmark/mem_runner.hpp"
#include "../include/power.hpp"

TestResult run_mem(KernelType kernel, size_t bytes_per_elem, const int N,
    dim3 blockSize, dim3 gridSize, float alpha) {
    float *d_A, *d_B, *d_C;
    CHECK_CUDA(cudaMalloc(&d_A, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_B, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_C, N * sizeof(float)));

    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    switch(kernel) {
        case KernelType::COPY:
            copy_kernel<<<gridSize, blockSize>>>(d_C, d_B, N);
            CHECK_CUDA(cudaDeviceSynchronize());

            CHECK_CUDA(cudaEventRecord(start));
            copy_kernel<<<gridSize, blockSize>>>(d_C, d_B, N);
            CHECK_CUDA(cudaEventRecord(stop));
            CHECK_CUDA(cudaDeviceSynchronize());
            break;
        case KernelType::SCALE:
            scale_kernel<<<gridSize, blockSize>>>(d_C, d_B, alpha, N);
            CHECK_CUDA(cudaDeviceSynchronize());

            CHECK_CUDA(cudaEventRecord(start));
            scale_kernel<<<gridSize, blockSize>>>(d_C, d_B, alpha ,N);
            CHECK_CUDA(cudaEventRecord(stop));
            CHECK_CUDA(cudaDeviceSynchronize());
            break;

        case KernelType::TRIAD:
            triad_kernel<<<gridSize, blockSize>>>(d_C, d_B, d_A, alpha, N);
            CHECK_CUDA(cudaDeviceSynchronize());

            CHECK_CUDA(cudaEventRecord(start));
            triad_kernel<<<gridSize, blockSize>>>(d_C, d_B, d_A, alpha, N);
            CHECK_CUDA(cudaEventRecord(stop));
            CHECK_CUDA(cudaDeviceSynchronize());
            break;
        case KernelType::ADD:
            add_kernel<<<gridSize, blockSize>>>(d_C, d_B, d_A, N);
            CHECK_CUDA(cudaDeviceSynchronize());

            CHECK_CUDA(cudaEventRecord(start));
            add_kernel<<<gridSize, blockSize>>>(d_C, d_B, d_A, N);
            CHECK_CUDA(cudaEventRecord(stop));
            CHECK_CUDA(cudaDeviceSynchronize());
            break;
        default:
            throw std::invalid_argument("Unknown kernel type: " +  std::
    to_string(static_cast<int>(kernel)));
            break;
    }

    float ms;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));

```

```
60      double GBs = (bytes_per_elem * (double)N) / (ms * 1e6);
61
62      CHECK_CUDA(cudaFree(d_A));
63      CHECK_CUDA(cudaFree(d_B));
64      CHECK_CUDA(cudaFree(d_C));
65      CHECK_CUDA(cudaEventDestroy(start));
66      CHECK_CUDA(cudaEventDestroy(stop));
67
68      return {ms, GBs};
69  }
70
71  void write_result_csv(const std::string &filename,
72                        const std::string &kernelName,
73                        int runs,
74                        std::vector<RunData> &results)
75  {
76      if (results.empty()) {
77          std::cerr << "[ERROR] results empty\n";
78          return;
79      }
80
81      auto [min_ms_it, max_ms_it] = std::minmax_element(results.begin(),
        results.end(),
82                                                        [](const RunData &a,
        const RunData &b) { return a.ms_compute < b.ms_compute; });
83      auto [min_bw_it, max_bw_it] = std::minmax_element(results.begin(),
        results.end(),
84                                                        [](const RunData &a,
        const RunData &b) { return a.gflops < b.gflops; });
85
86      double avg_ms = 0.0, avg_bw = 0.0, avg_power = 0.0;
87      double min_power = results.front().min_power;
88      double max_power = results.front().max_power;
89
90      for (const auto &r : results) {
91          avg_ms += r.ms_compute;
92          avg_bw += r.gflops;
93          avg_power += r.avg_power;
94          min_power = std::min(min_power, r.min_power);
95          max_power = std::max(max_power, r.max_power);
96      }
97
98      avg_ms /= results.size();
99      avg_bw /= results.size();
100     avg_power /= results.size();
101
102     auto timestamp = std::chrono::system_clock::to_time_t(std::chrono::
        system_clock::now());
103
104     std::ofstream file(filename, std::ios::app);
105     if (!file.is_open()) {
106         std::cerr << "[ERROR]: can't open " << filename << "\n";
107         return;
108     }
109
110     file.seekp(0, std::ios::end);
111     if (file.tellp() == 0) {
112         file << "Timestamp,Kernel,Size,Runs,"
113                 "Avg_Time_ms,Min_Time_ms,Max_Time_ms,"
114                 "Avg_Bandwidth_GBs,Min_Bandwidth_GBs,Max_Bandwidth_GBs,"
115                 "Avg_Power_W,Min_Power_W,Max_Power_W\n";
```

```
116     }
117
118     file << timestamp << ","
119          << kernelName << ","
120          << runs << ","
121          << std::fixed << std::setprecision(3)
122          << avg_ms << "," << min_ms_it->ms_compute << "," << max_ms_it->
    ms_compute << ","
123          << avg_bw << "," << min_bw_it->gflops << "," << max_bw_it->gflops
    << ","
124          << avg_power << "," << min_power << "," << max_power
125          << "\n";
126
127     file.close();
128
129     std::cout << "\nResults (" << kernelName << ")\n";
130     std::cout << runs << " run(s)\n";
131     std::cout << "Timestamp: " << timestamp << "\n\n";
132     std::cout << "avg ms: " << avg_ms << " (min " << min_ms_it->ms_compute
    << ", max " << max_ms_it->ms_compute << ")\n";
133     std::cout << "avg Bandwith : " << avg_bw << " GB/s (min " << min_bw_it
    ->gflops << ", max " << max_bw_it->gflops << ")\n";
134     std::cout << "avg Power : " << avg_power << " W (min " << min_power <<
    ", max " << max_power << ")\n";
135     std::cout << "-----------------------------------------\n";
136 }
137
138 void benchmark_mem(KernelType kernel, size_t bytes_per_elem, const int N,
    dim3 blockSize, dim3 gridSize , float alpha, const int runs) {
139     std::cout << "========== Benchmark: " << getKernelName(kernel)
140               << " | " << N << " elements, " << runs << " runs\n";
141
142     GpuPowerSampler sampler(0, 100);
143
144     std::vector<RunData> results;
145     results.reserve(runs);
146
147     for (int i = 0; i < runs; i++) {
148         unsigned int power_before, power_after;
149
150         sampler.start();
151         TestResult r = run_mem(kernel, bytes_per_elem, N, blockSize,
    gridSize, alpha);
152
153         sampler.stop();
154
155         double avg_power = sampler.averagePower();
156         double min_power = sampler.minPower();
157         double max_power = sampler.maxPower();
158
159         results.push_back({r.ms_compute, r.gflops, avg_power, min_power,
    max_power});
160
161
162 #ifdef DEBUG
163         std::cout << "[DEBUG]: Run " << i + 1 << ": "
164                   << r.ms_compute << " ms, "
165                   << r.gflops << " GB/s, "
166                   << " | Power (avg/min/max): "
167                   << avg_power << "/" << min_power << "/" << max_power << "
     W\n";
```

```
#endif
    }

    write_result_csv("mem_benchmark_results.csv", getKernelName(kernel),
    runs, results);
}
```

Listing A.23: Memory Runner (mem_runner.cu)

```
#include <nvml.h>
#include <thread>
#include <atomic>
#include <vector>
#include <numeric>
#include <chrono>
#include <mutex>
#include <iostream>
#include <limits>

class GpuPowerSampler {
public:
    GpuPowerSampler(unsigned int deviceIndex = 0, int sampleIntervalMs =
    100)
        : deviceIndex_(deviceIndex),
          sampleIntervalMs_(sampleIntervalMs),
          running_(false),
          minPower_(std::numeric_limits<double>::max()),
          maxPower_(0.0)
    {
        nvmlReturn_t result = nvmlInit();
        if (result != NVML_SUCCESS) {
            std::cerr << "NVML init failed: " << nvmlErrorString(result) <<
     std::endl;
        }

        result = nvmlDeviceGetHandleByIndex(deviceIndex_, &device_);
        if (result != NVML_SUCCESS) {
            std::cerr << "NVML device get failed: " << nvmlErrorString(
    result) << std::endl;
        }
    }

    ~GpuPowerSampler() {
        stop();
        nvmlShutdown();
    }

    void start() {
        if (running_) return;
        running_ = true;
        samplerThread_ = std::thread(&GpuPowerSampler::sampleLoop, this);
    }

    void stop() {
        if (!running_) return;
        running_ = false;
        if (samplerThread_.joinable())
            samplerThread_.join();
```

```
47      }
48
49      double averagePower() const {
50          std::lock_guard<std::mutex> lock(mutex_);
51          if (samples_.empty()) return 0.0;
52          double sum = std::accumulate(samples_.begin(), samples_.end(), 0.0)
    ;
53          return sum / samples_.size();
54      }
55
56      double minPower() const {
57          std::lock_guard<std::mutex> lock(mutex_);
58          return samples_.empty() ? 0.0 : minPower_;
59      }
60
61      double maxPower() const {
62          std::lock_guard<std::mutex> lock(mutex_);
63          return samples_.empty() ? 0.0 : maxPower_;
64      }
65
66      const std::vector<double>& samples() const { return samples_; }
67
68  private:
69      void sampleLoop() {
70          while (running_) {
71              unsigned int power_mW = 0;
72              if (nvmlDeviceGetPowerUsage(device_, &power_mW) == NVML_SUCCESS
    ) {
73                  double power_W = power_mW / 1000.0;
74
75                  std::lock_guard<std::mutex> lock(mutex_);
76                  samples_.push_back(power_W);
77                  if (power_W < minPower_) minPower_ = power_W;
78                  if (power_W > maxPower_) maxPower_ = power_W;
79              }
80
81              std::this_thread::sleep_for(std::chrono::milliseconds(
    sampleIntervalMs_));
82          }
83      }
84
85      unsigned int deviceIndex_;
86      int sampleIntervalMs_;
87      nvmlDevice_t device_;
88      std::atomic<bool> running_;
89      std::thread samplerThread_;
90      std::vector<double> samples_;
91      mutable std::mutex mutex_;
92
93      double minPower_;
94      double maxPower_;
95  };
```

Listing A.24: power.hpp

```
1  #ifndef HELP_HPP
2  #define HELP_HPP
3
```

```
4  #include <iostream>
5  #include "config.hpp"
6  #include "types.hpp"
7
8  #define CHECK_CUDA(func) {            \
9      cudaError_t e = (func);             \
10     if(e != cudaSuccess)              \
11         printf ("%s %d CUDA ERROR: %s\n", __FILE__,  __LINE__,
       cudaGetErrorString(e)); \
12 }
13
14 #define CHECK_CUBLAS(func) {                      \
15     cublasStatus_t e = (func);                    \
16     if(e != CUBLAS_STATUS_SUCCESS)                \
17         printf ("%s %d CUBLAS ERROR: ", __FILE__, __LINE__); \
18 }
19
20 inline const char* getKernelName(KernelType type) {
21     switch(type) {
22         case KernelType::NAIVE:     return "Naive";
23         case KernelType::TILED:     return "Tiled";
24         case KernelType::DENSE:     return "Dense";
25         case KernelType::ADD:       return "Add";
26         case KernelType::TRIAD:     return "Triad";
27         case KernelType::SCALE:     return "Scale";
28         case KernelType::COPY:      return "Copy";
29         case KernelType::CUBLAS:    return "Cublas";
30         case KernelType::TENSOR:    return "Tensor";
31         default:                    return "Unknown";
32     }
33 }
34
35 struct GPUInfo {
36     int device_id;
37     std::string name;
38     int compute_capability_major;
39     int compute_capability_minor;
40     int max_threads_per_block;
41     int max_threads_per_multiprocessor;
42     int multiprocessor_count;
43     int warp_size;
44     size_t total_global_mem;
45     size_t shared_mem_per_block;
46     int regs_per_block;
47 };
48
49 inline GPUInfo getGPUInfo(int device_id = -2) {
50     GPUInfo info;
51     cudaDeviceProp prop;
52     CHECK_CUDA(cudaGetDeviceProperties(&prop, device_id));
53
54     info.device_id = device_id;
55     info.name = prop.name;
56     info.compute_capability_major = prop.major;
57     info.compute_capability_minor = prop.minor;
58     info.max_threads_per_block = prop.maxThreadsPerBlock;
59     info.max_threads_per_multiprocessor = prop.maxThreadsPerMultiProcessor;
60     info.multiprocessor_count = prop.multiProcessorCount;
61     info.warp_size = prop.warpSize;
62     info.total_global_mem = prop.totalGlobalMem;
63     info.shared_mem_per_block = prop.sharedMemPerBlock;
```

```
64      info.regs_per_block = prop.regsPerBlock;
65
66      return info;
67  }
68
69  inline void printGPUInfo(const GPUInfo& info) {
70      std::cout << "=== GPU Information ===" << std::endl;
71      std::cout << "Device: " << info.device_id << " - " << info.name << std
        ::endl;
72      std::cout << "Compute Capability: " << info.compute_capability_major
        << "." << info.compute_capability_minor << std::endl;
73      std::cout << "Max Threads per Block: " << info.max_threads_per_block <<
         std::endl;
74      std::cout << "Multiprocessors: " << info.multiprocessor_count << std::
        endl;
75      std::cout << "Warp Size: " << info.warp_size << std::endl;
76      std::cout << "Total Global Memory: " << info.total_global_mem /
        (1024*1024) << " MB" << std::endl;
77      std::cout << "Shared Memory per Block: " << info.shared_mem_per_block
        << " bytes" << std::endl;
78      std::cout << "Registers per Block: " << info.regs_per_block << std::
        endl;
79      std::cout << "========================" << std::endl << std::endl;
80  }
81
82  #ifdef CHECK_RESULT
83  //#include <cmath>
84  //#include <algorithm>
85
86  inline void cpu_gemm(const float* A, const float* B, float* C,
87                       int M, int N, int K) {
88      for (int i = 0; i < M; ++i) {
89          for (int j = 0; j < N; ++j) {
90              float sum = 0.0f;
91              for (int k = 0; k < K; ++k) {
92                  sum += A[i * K + k] * B[k * N + j];
93              }
94              C[i * N + j] = sum;
95          }
96      }
97  }
98
99  inline bool compare_matrix(const float* gpu_C, const float* cpu_C,
100                             int size, float tolerance = 1e-3f) {
101     for (int i = 0; i < size; ++i) {
102         float diff = std::abs(gpu_C[i] - cpu_C[i]);
103         float max_val = std::max(std::abs(gpu_C[i]), std::abs(cpu_C[i]));
104
105         if (diff > tolerance && diff > tolerance * max_val) {
106             std::cerr << "------ [DEBUG RESULT] Mismatch at index " << i
107                       << ": GPU=" << gpu_C[i]
108                       << ", CPU=" << cpu_C[i]
109                       << ", diff=" << diff << std::endl;
110             return false;
111         }
112     }
113     return true;
114 }
115
116
117 #endif
```

```
118
119 #endif
```

Listing A.25: help_func.hpp

A.2.2 OpenVINO Benchmark

```
1  #include <openvino/openvino.hpp>
2  #include <openvino/op/matmul.hpp>
3  #include <iostream>
4  #include <vector>
5  #include <random>
6  #include <string>
7  #include <chrono>
8  //#include <cmath>
9  #include <fstream>
10 #include <thread>
11 #include <atomic>
12 #include <iomanip>
13 #include <sstream>
14 #include <algorithm>
15
16 #define DEBUG 1
17
18 struct PowerSample {
19     double pkg_w, core_w, uncore_w, gpu_w, npu_w;
20     double timestamp;
21 };
22
23 std::atomic<bool> sampling{false};
24 std::vector<PowerSample> samples;
25
26 double read_value(const std::string &path) {
27     std::ifstream file(path);
28     if (!file.is_open()) return -1;
29     double value;
30     file >> value;
31     return value;
32 }
33
34 double read_rapl_energy(const std::string &domain) {
35     std::string path = "/sys/class/powercap/" + domain + "/energy_uj";
36     return read_value(path);
37 }
38
39 void power_monitor_thread() {
40     auto t0 = std::chrono::high_resolution_clock::now();
41
42     double pkg_prev = read_rapl_energy("intel-rapl:0");
43     double core_prev = read_rapl_energy("intel-rapl:0/intel-rapl:0:0");
44     double uncore_prev = read_rapl_energy("intel-rapl:0/intel-rapl:0:1");
45
46     FILE *fp = popen("sudo intel_gpu_top -c -s 100", "r");
47     if (!fp) {
48         std::cerr << "[ERROR] intel_gpu_top\n";
49     }
50
51     char buffer[2048];
```

```
52      double gpu_w = 0.0;
53      bool header_skipped = false;
54
55      while (sampling) {
56          auto now = std::chrono::high_resolution_clock::now();
57          double dt = std::chrono::duration<double>(now - t0).count();
58          t0 = now;
59
60          double pkg_now = read_rapl_energy("intel-rapl:0");
61          double core_now = read_rapl_energy("intel-rapl:0/intel-rapl:0:0");
62          double uncore_now = read_rapl_energy("intel-rapl:0/intel-rapl:0:1")
    ;
63
64          if (pkg_now < 0 || core_now < 0 || uncore_now < 0) {
65              std::cout << "[ERROR] RAPL read failed\n";
66              break;
67          }
68
69          double pkg_w = (pkg_now - pkg_prev) * 1e-6 / dt;
70          double core_w = (core_now - core_prev) * 1e-6 / dt;
71          double uncore_w = (uncore_now - uncore_prev) * 1e-6 / dt;
72
73          pkg_prev = pkg_now;
74          core_prev = core_now;
75          uncore_prev = uncore_now;
76
77          if (fp && fgets(buffer, sizeof(buffer), fp)) {
78              if (!header_skipped) {
79                  header_skipped = true;
80                  continue;
81              }
82              std::string line(buffer);
83              std::stringstream ss(line);
84              std::string token;
85              for (int i = 0; i <= 4; i++)
86                  std::getline(ss, token, ',');
87              try {
88                  gpu_w = std::stod(token);
89              } catch (...) {
90                  gpu_w = 0;
91              }
92          } else {
93              gpu_w = 0;
94          }
95
96          double npu_w = pkg_w - (core_w + uncore_w);
97          if (npu_w < 0.0) npu_w = 0.0;
98
99          double timestamp = std::chrono::duration<double>(now.
    time_since_epoch()).count();
100         samples.push_back({pkg_w, core_w, uncore_w, gpu_w, npu_w, timestamp
    });
101
102         std::this_thread::sleep_for(std::chrono::milliseconds(50));
103     }
104
105     if (fp)
106         pclose(fp);
107 }
108
109 struct PowerStats {
```

```
110     double avg, min, max;
111 };
112
113 PowerStats compute_stats(std::vector<double> &v) {
114     if (v.empty()) return {0, 0, 0};
115     double sum = 0;
116     for (auto &x : v) sum += x;
117     return {
118         sum / v.size(),
119         *std::min_element(v.begin(), v.end()),
120         *std::max_element(v.begin(), v.end())
121     };
122 }
123
124 struct PowerSummary {
125     PowerStats pkg, core, uncore, gpu, npu;
126 };
127
128 PowerSummary compute_power_summary() {
129     std::vector<double> pkg, core, uncore, gpu, npu;
130     for (auto &s : samples) {
131         pkg.push_back(s.pkg_w);
132         core.push_back(s.core_w);
133         uncore.push_back(s.uncore_w);
134         gpu.push_back(s.gpu_w);
135         npu.push_back(s.npu_w);
136     }
137     return {compute_stats(pkg), compute_stats(core), compute_stats(uncore),
      compute_stats(gpu), compute_stats(npu)};
138 }
139
140 void save_results_to_csv(const std::string &filename,
141                          const std::string &device,
142                          int M, int N, int K, int runs,
143                          const PowerSummary &summary,
144                          double avg_time_compute, double avg_time_memcpy,
     double avg_gflops) {
145     std::ofstream file(filename, std::ios::app);
146     if (!file.is_open()) {
147         std::cerr << "[Error]: can't open " << filename << "\n";
148         return;
149     }
150
151     auto timestamp = std::chrono::system_clock::to_time_t(std::chrono::
     system_clock::now());
152
153     file.seekp(0, std::ios::end);
154
155     /* if the file is new, write the intestation */
156     if (file.tellp() == 0) {
157         file << "Timestamp,Device,M,N,K,Runs,"
158                 "Avg_Compute_ms,Avg_MemCpy_ms,Avg_GFLOPS,"
159                 "Pkg_Avg_W,Core_Avg_W,Uncore_Avg_W,GPU_Avg_W,NPU_Avg_W,"
160                 "Pkg_Min_W,Core_Min_W,Uncore_Min_W,GPU_Min_W,NPU_Min_W,"
161                 "Pkg_Max_W,Core_Max_W,Uncore_Max_W,GPU_Max_W,NPU_Max_W\n";
162     }
163
164     /* csv format */
165     file << timestamp << ","
166          << device << ","
167          << M << "," << N << "," << K << ","
```

```
168           << runs << ","
169           << std::fixed << std::setprecision(3)
170           << avg_time_compute << ","
171           << avg_time_memcpy << ","
172           << avg_gflops << ","
173           << summary.pkg.avg << "," << summary.core.avg << "," << summary.
      uncore.avg << "," << summary.gpu.avg << "," << summary.npu.avg << ","
174           << summary.pkg.min << "," << summary.core.min << "," << summary.
      uncore.min << "," << summary.gpu.min << "," << summary.npu.min << ","
175           << summary.pkg.max << "," << summary.core.max << "," << summary.
      uncore.max << "," << summary.gpu.max << "," << summary.npu.max
176           << "\n";
177
178     file.close();
179
180 #ifdef DEBUG
181     std::cout << "\n=== Results ===\n";
182     std::cout << "Device: " << device << "\n";
183     std::cout << "Matrix: " << M << "x" << N << "x" << K << " | Runs: " <<
      runs << "\n";
184     std::cout << "Avg Compute Time: " << avg_time_compute << " ms | Avg
      MemCpy Time: " << avg_time_memcpy << " ms | Avg GFLOPS: " << avg_gflops
       << "\n";
185     std::cout << "Power (W):\n"
186               << "  Package: " << summary.pkg.avg << " (min " << summary.
      pkg.min << ", max " << summary.pkg.max << ")\n"
187               << "  Core:    " << summary.core.avg << " (min " << summary.
      core.min << ", max " << summary.core.max << ")\n"
188               << "  Uncore:  " << summary.uncore.avg << " (min " << summary
      .uncore.min << ", max " << summary.uncore.max << ")\n"
189               << "  GPU:     " << summary.gpu.avg << " (min " << summary.
      gpu.min << ", max " << summary.gpu.max << ")\n"
190               << "  NPU:     " << summary.npu.avg << " (min " << summary.
      npu.min << ", max " << summary.npu.max << ")\n";
191     std::cout << "================================\n\n";
192 #endif
193 }
194
195
196 struct TestResult {
197     double ms_compute;
198     double ms_memcpy;
199     double gflops;
200     bool correct;
201 };
202
203 TestResult run_matmul(ov::Core &core, const std::string &device_name,
204                       int M, int N, int K,
205                       const std::vector<ov::float16> &h_A,
206                       const std::vector<ov::float16> &h_B) {
207     auto A = std::make_shared<ov::op::v0::Parameter>(ov::element::f16, ov::
      Shape{(u_long)M, (u_long)K});
208     auto B = std::make_shared<ov::op::v0::Parameter>(ov::element::f16, ov::
      Shape{(u_long)K, (u_long)N});
209     auto matmul = std::make_shared<ov::op::v0::MatMul>(A, B);
210     auto result = std::make_shared<ov::op::v0::Result>(matmul);
211     auto model = std::make_shared<ov::Model>(result, ov::ParameterVector{A,
       B});
212
213     ov::CompiledModel compiled_model = core.compile_model(model,
      device_name);
```

```
214     ov::InferRequest infer_request = compiled_model.create_infer_request();
215 
216     ov::Tensor tensor_A(ov::element::f16, ov::Shape{(u_long)M, (u_long)K},
    const_cast<ov::float16 *>(h_A.data()));
217     ov::Tensor tensor_B(ov::element::f16, ov::Shape{(u_long)K, (u_long)N},
    const_cast<ov::float16 *>(h_B.data()));
218 
219     /* memcpy or wrapper timing */
220     auto start_memcpy = std::chrono::high_resolution_clock::now();
221     infer_request.set_input_tensor(0, tensor_A);
222     infer_request.set_input_tensor(1, tensor_B);
223     auto end_memcpy = std::chrono::high_resolution_clock::now();
224     double ms_memcpy = std::chrono::duration<double, std::milli>(end_memcpy
     - start_memcpy).count();
225 
226     /* warmup */
227     infer_request.infer();
228 
229     /* computing timing */
230     auto start_compute = std::chrono::high_resolution_clock::now();
231     infer_request.infer();
232     auto end_compute = std::chrono::high_resolution_clock::now();
233 
234     double ms_compute = std::chrono::duration<double, std::milli>(
    end_compute - start_compute).count();
235     double gflops = (2.0 * M * N * K) / (ms_compute * 1e6);
236 
237     return {ms_compute, ms_memcpy, gflops, true};
238 }
239 
240 void benchmark_device(const std::string &device_name, int M, int N, int K,
    int runs) {
241     ov::Core core;
242 
243     bool available = false;
244     for (auto &d : core.get_available_devices())
245         if (d.find(device_name) != std::string::npos)
246             available = true;
247     if (!available) {
248         std::cout << "Device " << device_name << " non disponibile\n";
249         return;
250     }
251 
252 
253 #ifdef DEBUG
254     std::cout << "\n=== Benchmark: " << device_name << " ===\n";
255     std::cout << "Matrix size: " << M << "x" << N << "x" << K << " | Runs:
    " << runs << "\n";
256 #endif
257 
258     std::vector<ov::float16> h_A(M * K), h_B(K * N);
259     std::mt19937 gen(42);
260     std::uniform_real_distribution<float> dis(-1.0f, 1.0f);
261     for (auto &x : h_A) x = ov::float16(dis(gen));
262     for (auto &x : h_B) x = ov::float16(dis(gen));
263 
264     double sum_ms_compute = 0.0, sum_ms_memcpy = 0.0, sum_gflops = 0.0; //
    Modificato
265 
266     samples.clear();
267     sampling = true;
```

```
268     std::thread mon(power_monitor_thread);
269 
270     for (int i = 0; i < runs; i++) {
271         TestResult r = run_matmul(core, device_name, M, N, K, h_A, h_B);
272         sum_ms_compute += r.ms_compute;
273         sum_ms_memcpy += r.ms_memcpy;
274         sum_gflops += r.gflops;
275     }
276 
277     sampling = false;
278     mon.join();
279 
280     double avg_time_compute = sum_ms_compute / runs;
281     double avg_time_memcpy = sum_ms_memcpy / runs;
282     double avg_gflops = sum_gflops / runs;
283     auto summary = compute_power_summary();
284 
285 #ifdef DEBUG
286     std::cout << std::fixed << std::setprecision(2);
287     std::cout << "Average Compute Time: " << avg_time_compute << " ms |
    Average MemCpy Time: " << avg_time_memcpy << " ms | Average GFLOPS: "
    << avg_gflops << "\n";
288     std::cout << "Core avg: " << summary.core.avg << "W (min " << summary.
    core.min << ", max " << summary.core.max << ")\n";
289     std::cout << "Uncore avg: " << summary.uncore.avg << "W (min " <<
    summary.uncore.min << ", max " << summary.uncore.max << ")\n";
290     std::cout << "GPU avg: " << summary.gpu.avg << "W (min " << summary.gpu
    .min << ", max " << summary.gpu.max << ")\n";
291     std::cout << "NPU est.: " << summary.npu.avg << "W (min " << summary.
    npu.min << ", max " << summary.npu.max << ")\n";
292 #endif
293 
294     save_results_to_csv("power_benchmarks.csv", device_name, M, N, K, runs,
     summary, avg_time_compute, avg_time_memcpy, avg_gflops);
295 }
296 
297 int main() {
298     benchmark_device("CPU", 4096, 4096, 2048, 10);
299     benchmark_device("GPU", 4096, 4096, 2048, 10);
300     benchmark_device("NPU", 4096, 4096, 2048, 10);
301     return 0;
302 }
```

Listing A.26: Openvino Main (main.cpp)

```
1 TARGET = benchmark
2 
3 SRC_DIR = src
4 SRC = $(SRC_DIR)/main.cpp
5 
6 CXX = g++
7 CXXFLAGS = -std=c++17
8 
9 LDFLAGS += -lopenvino
10 
11 all: $(TARGET)
12 
13 $(TARGET): $(SRC)
```

```
  $(CXX) $(SRC) -o $(TARGET) $(LDFLAGS)

clean:
  rm -f $(TARGET)

run: $(TARGET)
  ./$(TARGET)
```

Listing A.27: Openvino Makefile

A.2.3 SYCL Benchmark

```
#include <sycl/sycl.hpp>
#include <oneapi/mkl/blas.hpp>
#include <iostream>
#include <vector>
#include <chrono>
#include <fstream>
#include <thread>
#include <atomic>
#include <iomanip>
#include <sstream>
#include <algorithm>
#include <numeric>
#include <cmath>
#include <cstdio>
#include <cstdint> // Per uint16_t

// sycl::half invece di float
using data_t = sycl::half;

struct PowerStats {
    double avg;
    double min;
    double max;
};

PowerStats compute_stats(const std::vector<double>& v) {
    if (v.empty()) return {0.0, 0.0, 0.0};
    double sum = std::accumulate(v.begin(), v.end(), 0.0);
    return { sum / v.size(), *std::min_element(v.begin(), v.end()),
                         *std::max_element(v.begin(), v.end()) };
}

void save_results_to_csv(const std::string& filename,
                         const std::string& device,
                         int M, int N, int K, int runs,
                         double avg_compute_ms, double avg_memcpy_ms,
    double gflops,
                         const PowerStats& gpu_stats) {

    std::ofstream file(filename, std::ios::app);
    if (!file.is_open()) {
        std::cerr << "[ERRORE] CSV\n";
        return;
    }

    file.seekp(0, std::ios::end);
```

```
46      if (file.tellp() == 0) {
47          file << "Timestamp,Device,Mode,Precision,M,N,K,Runs,Avg_Compute_ms,
    Avg_Memcpy_ms,GFLOPS,"
48               << "GPU_Avg_W,GPU_Max_W\n";
49      }
50
51      auto now = std::chrono::system_clock::to_time_t(std::chrono::
    system_clock::now());
52
53      #ifdef ZEROCOPY
54      std::string mode = "ZeroCopy";
55      #else
56      std::string mode = "Explicit";
57      #endif
58
59      // Identifichiamo la precisione
60      std::string prec = (sizeof(data_t) == 2) ? "FP16" : "FP32";
61
62      file << std::put_time(std::localtime(&now), "%Y-%m-%d %H:%M:%S") << ","
63           << device << ","
64           << mode << ","
65           << prec << ","
66           << M << "," << N << "," << K << ","
67           << runs << ","
68           << std::fixed << std::setprecision(3)
69           << avg_compute_ms << ","
70           << avg_memcpy_ms << ","
71           << gflops << ","
72           << gpu_stats.avg << ","
73           << gpu_stats.max << "\n";
74
75      std::cout << ">> Results in: " << filename << "\n";
76  }
77
78  class GpuPowerMonitor {
79  private:
80      std::atomic<bool> sampling;
81      std::vector<double> gpu_samples;
82      std::thread monitor_thread;
83
84      void thread_loop() {
85          FILE* fp = popen("sudo intel_gpu_top -c -s 100", "r");
86          if (!fp) return;
87
88          char buffer[2048];
89          bool header_skipped = false;
90
91          while (sampling) {
92              if (fgets(buffer, sizeof(buffer), fp)) {
93                  if (!header_skipped) {
94                      header_skipped = true;
95                  } else {
96                      std::stringstream ss(buffer);
97                      std::string token;
98                      for (int i = 0; i <= 4; i++) std::getline(ss, token, ',
    ');
99                      try {
100                         double val = std::stod(token);
101                         if(val > 0) gpu_samples.push_back(val);
102                     } catch (...) {}
103                 }
```

```
104             }
105             std::this_thread::sleep_for(std::chrono::milliseconds(50));
106         }
107         pclose(fp);
108     }
109 
110 public:
111     GpuPowerMonitor() : sampling(false) {}
112 
113     void start() {
114         if (sampling) return;
115         gpu_samples.clear();
116         sampling = true;
117         monitor_thread = std::thread(&GpuPowerMonitor::thread_loop, this);
118     }
119 
120     void stop() {
121         if (!sampling) return;
122         sampling = false;
123         if (monitor_thread.joinable()) monitor_thread.join();
124     }
125 
126     PowerStats get_stats() { return compute_stats(gpu_samples); }
127 };
128 
129 namespace mkl = oneapi::mkl;
130 
131 void benchmark(sycl::queue& q, int M, int N, int K, int runs) {
132     #ifdef ZEROCOPY
133     std::cout << "\n=== Benchmark FP16 [ZERO-COPY]: " << M << "x" << N << "
    x" << K << " ===" << "\n";
134     #else
135     std::cout << "\n=== Benchmark FP16 [EXPLICIT]: " << M << "x" << N << "x
    " << K << " ===" << "\n";
136     #endif
137 
138     std::vector<data_t> h_A(M * K, data_t(1.0f));
139     std::vector<data_t> h_B(K * N, data_t(2.0f));
140     std::vector<data_t> h_C(M * N, data_t(0.0f));
141 
142     try {
143         data_t *d_A, *d_B, *d_C;
144 
145         #ifdef ZEROCOPY
146             d_A = sycl::malloc_shared<data_t>(M * K, q);
147             d_B = sycl::malloc_shared<data_t>(K * N, q);
148             d_C = sycl::malloc_shared<data_t>(M * N, q);
149         #else
150             d_A = sycl::malloc_device<data_t>(M * K, q);
151             d_B = sycl::malloc_device<data_t>(K * N, q);
152             d_C = sycl::malloc_device<data_t>(M * N, q);
153         #endif
154 
155         if (!d_A || !d_B || !d_C) {
156             std::cerr << "[ERROR] Memory GPU\n";
157             return;
158         }
159 
160         #ifdef ZEROCOPY
161             std::copy(h_A.begin(), h_A.end(), d_A);
162             std::copy(h_B.begin(), h_B.end(), d_B);
```

```
163         #else
164             q.memcpy(d_A, h_A.data(), M * K * sizeof(data_t));
165             q.memcpy(d_B, h_B.data(), K * N * sizeof(data_t));
166         #endif
167         q.wait();
168 
169         mkl::transpose trans = mkl::transpose::nontrans;
170         data_t alpha = data_t(1.0f);
171         data_t beta = data_t(0.0f);
172 
173         mkl::blas::row_major::gemm(q, trans, trans, M, N, K, alpha, d_A, K,
     d_B, N, beta, d_C, N);
174         q.wait();
175 
176         GpuPowerMonitor monitor;
177         monitor.start();
178 
179         double total_compute_ms = 0.0;
180         double total_memcpy_ms = 0.0;
181 
182         for (int i = 0; i < runs; ++i) {
183 
184             #ifndef ZEROCOPY
185             auto t_copy_start = std::chrono::high_resolution_clock::now();
186 
187             auto e1 = q.memcpy(d_A, h_A.data(), M * K * sizeof(data_t));
188             auto e2 = q.memcpy(d_B, h_B.data(), K * N * sizeof(data_t));
189             sycl::event::wait({e1, e2});
190 
191             auto t_copy_end = std::chrono::high_resolution_clock::now();
192             total_memcpy_ms += std::chrono::duration<double, std::milli>(
    t_copy_end - t_copy_start).count();
193             #endif
194 
195             auto t_compute_start = std::chrono::high_resolution_clock::now
    ();
196 
197             auto e_gemm = mkl::blas::row_major::gemm(q, trans, trans, M, N,
     K, alpha, d_A, K, d_B, N, beta, d_C, N);
198             e_gemm.wait();
199 
200             auto t_compute_end = std::chrono::high_resolution_clock::now();
201             total_compute_ms += std::chrono::duration<double, std::milli>(
    t_compute_end - t_compute_start).count();
202         }
203 
204         monitor.stop();
205 
206         sycl::free(d_A, q);
207         sycl::free(d_B, q);
208         sycl::free(d_C, q);
209 
210         double avg_compute = total_compute_ms / runs;
211         double avg_memcpy = total_memcpy_ms / runs;
212         double gflops = (2.0 * static_cast<double>(M) * N * K) / (
    avg_compute * 1e6);
213         PowerStats p_stats = monitor.get_stats();
214 
215         std::cout << ">> Avg Compute: " << avg_compute << " ms\n";
216         std::cout << ">> Avg Memcpy:  " << avg_memcpy << " ms\n";
217         std::cout << ">> GFLOPS:      " << gflops << "\n";
```

```
218         std::cout << ">> Power:       " << p_stats.avg << " W\n";
219 
220         std::string dev_name = q.get_device().get_info<sycl::info::device::
    name>();
221         save_results_to_csv("benchmark_usm_fp16.csv", dev_name, M, N, K,
    runs, avg_compute, avg_memcpy, gflops, p_stats);
222 
223     } catch (sycl::exception const& e) {
224         std::cerr << "[SYCL ERROR]: " << e.what() << "\n";
225     }
226 }
227 
228 int main() {
229     try {
230         sycl::queue q(sycl::gpu_selector_v);
231         auto dev = q.get_device();
232         std::cout << "Device: " << dev.get_info<sycl::info::device::name>()
     << "\n";
233 
234         if (!dev.has(sycl::aspect::fp16)) {
235             std::cerr << "[ERROR] no FP16!\n";
236             return 1;
237         }
238 
239         benchmark(q, 4096, 4096, 4096, 10);
240 
241     } catch (sycl::exception const& e) {
242         std::cerr << "[ERROR] SYCL\n";
243         return 1;
244     }
245     return 0;
246 }
```

Listing A.28: SYCL Main (main.cpp)

```
1  TARGET = sycl_benchmark
2  BUILD_DIR = build
3  EXECUTABLE = $(BUILD_DIR)/$(TARGET)
4  CXX = icpx
5 
6  SRC_FILES = $(wildcard *.cpp)
7 
8  all: $(EXECUTABLE)
9 
10 $(EXECUTABLE): CMakeLists.txt $(SRC_FILES)
11   mkdir -p $(BUILD_DIR)
12   cd $(BUILD_DIR) && cmake -DCMAKE_CXX_COMPILER=$(CXX) ..
13 
14   $(MAKE) -C $(BUILD_DIR)
15 
16 run: all
17   sudo ./$(EXECUTABLE)
18 
19 clean:
20   rm -rf $(BUILD_DIR)
21 
22 fresh: clean run
23 
```

```
24 .PHONY: all run clean fresh
```

Listing A.29: SYCL Makefile)

Bibliography

[1] Intel Corporation. *Intel Arc Graphics Xe-LPG Architecture*, 2023. https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2023-2/intel-xe-gpu-architecture.html.

[2] Intel Corporation. *Intel Core Ultra Processors (Series 1)*, 2024. https://www.intel.com/content/www/us/en/products/sku/236849/intel-core-ultra-9-processor-185h-24m-cache-up-to-5-10-ghz/specifications.html.

[3] Intel Corporation. Intel oneapi: A unified programming model, 2024. https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html.

[4] Intel Corporation. *Intel's Neural Processing Unit*, 2024. https://intel.github.io/intel-npu-acceleration-library/npu.html.

[5] Intel Corporation. *Openvino Documentation*, 2025. https://docs.openvino.ai/2025/index.htm.

[6] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice Hall, 1988.

[7] Kitware, Inc. *CMake Documentation*, 2025. https://cmake.org/cmake/help/latest/.

[8] Klaus Hinum. *NVIDIA GeForce RTX 4060 Laptop GPU Spec*, 2022. https://www.notebookcheck.net/NVIDIA-GeForce-RTX-4060-Laptop-GPU-Benchmarks-and-Specs.675692.0.html.

[9] NVIDIA Corporation. *Cublas*, 2025. https://docs.nvidia.com/cuda/cublas/index.html.

[10] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2025. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[11] NVIDIA Corporation. *NVIDIA Management Library (NVML)*, 2026. https://developer.nvidia.com/management-library-nvml.

[12] OpenBLAS Team. Openblas github, 2024. https://github.com/OpenMathLib/OpenBLAS.

[13] Steve Rennich. *CUDA C++ Streams and Concurrency*, 2024. https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf.

[14] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 2013.

[15] The Khronos Group. Sycl overview and specification, 2024. https://www.khronos.org/sycl/.

[16] Zhang Xianyi. Openblas, 2025. http://www.openmathlib.org/OpenBLAS/.